DESIGN AND DEVELOPMENT OF FLOOR LEVEL MATERIAL TRANSFER ROBOT

21MHP401L - MAJOR PROJECT REPORT

Submitted by

Lalithesh K (RA2211038010007)

Tejas M K (RA2211038010012)

Preetham M (RA2211038010046)

(Batch No.: MHRP02)

Under the guidance of

Dr. R. Senthilnathan M.E., Ph.D.

(Professor, Department of Mechatronics Engineering)

In Partial Fulfilment for the Degree

of

BACHELOR OF TECHNOLOGY

in

MECHATRONICS ENGINEERING (ROBOTICS)

of

COLLEGE OF ENGINEERING & TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

May 2026

BONAFIDE CERTIFICATE

Certified that this project report titled ***"Design and Development Floor Level Material Transfer Robot"*** is the Bonafide work of **"Lalithesh K (RA2211038010007), Tejas M K (RA2211038010012) and Preetham M (RA2211038010046)"** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. R. Senthilnathan M.E., Ph.D.
GUIDE
Professor
Dept. of Mechatronics Engineering

SIGNATURE

Dr. T. Muthuramalingam, M.E., Ph.D.
Professor & Head of The Department
Dept. of Mechatronics Engineering

Signature of the Internal Examiner

Signature of the External Examiner

ABSTRACT

This project presents the design and development of a floor-level material transfer robot that autonomously delivers files, documents, laboratory supplies, and other general-purpose items between staff rooms, classrooms, and laboratories on a single floor of an institutional building. The robot is built on the DANI differential-drive chassis, equipped with a stereo camera as the primary perception and localization sensor, and powered by an embedded controller with GPU capabilities wrapped under a middleware framework.

The environment is represented as an occupancy grid map, constructed by physically measuring corridor dimensions with a surveyor’s tape and rasterizing the layout into a pixel-accurate PNG where black regions encode obstacles and white regions encode traversable space. The floor map is segmented into tiles; a custom tile-switching plugin monitors the robot's position and initiates a map swap when the robot enters the overlapping region between adjacent tiles, ensuring localization continuity across the full corridor length.

Runtime localization fuses the visual-inertial positional tracking with IMU data, providing continuous pose estimates without a dedicated LiDAR sensor. Navigation is executed using SmacPlanner2D as the global path planner and the DWB (Dynamic Window B) controller as the local trajectory executor. A custom obstacle avoidance plugin extends DWB with depth-map ingestion and context-awareness, enabling the controller to consume ZED depth data and differentiate between obstacle classes during trajectory evaluation.

Obstacle avoidance is driven by the stereo camera's live depth map. To enable context-aware behavior, the ZED SDK's multiclass object detection model is applied to the RGB stream to classify detected obstacles. The stack is structured in way where when a human is identified in the robot's path, the robot stalls and waits for the person to clear, while for all other dynamic obstacles, the Nav2 global planner is triggered to replan around the blockage. This distinction ensures safe cohabitation with building occupants while maintaining delivery throughput.

Task management is handled through a web application that allows staff to submit pickup and drop-off location requests, monitor queue status, and receive delivery confirmations at each stage of the mission. The full software stack and the web backend runs on the embedded controller, with targeted memory optimization applied across the pipeline to sustain reliable operation within these constraints. This work demonstrates a practical, sensor-minimal implementation of autonomous indoor logistics for an institutional campus environment.

ACKNOWLEDGEMENTS

It has been a great honor and privilege to undergo **B.Tech** in **Mechatronics Engineering with specialization in Robotics** at **SRM Institute of Science and Technology**. We are very much thankful to the **Department of Mechatronics, SRM Institute of Science and Technology** for providing all facilities and support to meet our project requirements.

We would like to thank our Head of Department **Dr. T. Muthuramalingam, M.E., Ph.D.** and our project guide **Dr. R. Senthilnathan M.E., Ph.D.** who gave us a big helping hand during the entire project.

The success and final outcome of this project required a lot of guidance and assistance from many people, and we are extremely fortunate that we have got this all along the completion of our project work. Whatever we have achieved so far is only due to such timely guidance and assistance and we would like to thank them for their kind support.

LALITHESH K

TEJAS M K

PREETHAM M

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	I
	ACKNOWLEDGEMENTS	III
	TABLE OF CONTENTS	IV
	LIST OF TABLES	VII
	LIST OF FIGURES	VIII
	LIST OF ABBREVIATIONS	XII
	LIST OF SYMBOLS	XV
1.	**INTRODUCTION**	1
	1.1 INTRODUCTION TO THE BROAD FIELD OF WORK	1
	1.2 OBJECTIVES OF THE PROJECT	5
	1.3 PERSPECTIVE OF THE PROJECT	5
	1.4 CONTEXT AND MOTIVATION	8
	1.5 CONTRIBUTION AND SIGNIFICANCE	10
	1.6 OVERVIEW OF THE PROJECT	11
	1.7 NOTATIONS AND CONVENTIONS	12
	1.8 ORGANIZATION OF THE REPORT	12
2.	**LITERATURE SURVEY**	14
	2.1 PURPOSES OF SURVEY	14

	2.2 SOURCES OF SURVEY	15
	2.3 REVIEW OF ARTICLES	15
	2.4 KEY FINDINGS	19
3.	**SYSTEM HARDWARE**	22
	3.1 HARDWARE BLOCK DIAGRAM	22
	3.2 MECHANICAL DESIGN AND FABRICATION	23
	3.3 SENSORS	29
	3.4 ACTUATORS	33
	3.5 PROCESSING UNIT	35
	3.6 ELECTRICAL INTERFACES AND COMMUNICATION	36
	3.7 POWER SUPPLY CIRCUITRY	40
	3.8 PCB DESIGN	43
	3.9 ROBOT SPECIFICATIONS	46
4.	**SYSTEM SOFTWARE**	50
	4.1 SOFTWARE ARCHITECTURE	50
	4.2 MAP GENERATION AND USAGE	51
	4.3 LOCALIZATION	60
	4.4 PATH PLANNING	66
	4.5 OBSTACLE AVOIDANCE	70
	4.6 CONTROL	74

4.7 TASK ASSIGNMENT AND MANAGEMENT 79

5. SYSTEM INTEGRATION 86

5.1 DEPLOYMENT ENVIRONMENT 86

5.2 MIDDLEWARE 89

5.3 NAVIGATION FRAMEWORK 93

5.4 SYSTEM SUPERVISOR 98

5.5 LAUNCH AND STARTUP CONFIGURATION 104

6. USER INTERFACE DEVELOPMENT AND INTEGRATION 110

6.1 OVERVIEW 110

6.2 SYSTEM ARCHITECTURE 110

6.3 TECHNOLOGY STACK 111

6.4 INTERFACE DESIGN AND FUNCTIONALITY 113

6.5 SUMMARY 116

7. CONCLUSION 118

7.1 CONCLUSION 118

7.2 SCOPE FOR FUTURE WORK 121

REFERENCES XIX

APPENDIX A – SETUP GUIDE XXI

APPENDIX B – DATASHEETS AND MANUALS XXIII

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
TABLE 2.1	COMPARISON OF EXISTING MAP MANAGEMENT APPROACHES WITH THE PROPOSED TILE-BASED APPROACH	19
TABLE 3.1	SUMMARY TABLE REPRESENTING ALL PERIPHERAL INTERFACES WITH THE JETSON NANO	39
TABLE 3.2	ROBOT MECHANICAL SPECIFICATIONS	46
TABLE 3.3	ROBOT ACTUATION SPECIFICATIONS	47
TABLE 3.4	ROBOT SENSING SPECIFICATIONS	47
TABLE 3.5	ROBOT COMPUTATION SPECIFICATIONS	48
TABLE 3.6	ROBOT POWER SYSTEM SPECIFICATIONS	48
TABLE 3.7	ROBOT ELECTRICAL INTERFACE SPECIFICATIONS	49
TABLE 4.1	TILE DIMENSIONS, PHYSICAL COVERAGE, AND NAVIGABLE CELL STATISTICS AT 0.04 M/PX RESOLUTION	56
TABLE 4.2	YAML METADATA PARAMETERS FOR EACH TILE MAP	57

TABLE 4.3 TILE SWITCHING ALGORITHM CONFIGURATION PARAMETERS 58

TABLE 4.4 VELOCITY COMMAND SOURCE PRIORITIES FOR THE PRIORITY-BASED MULTIPLEXER 75

TABLE 4.5 CYTRON SMARTDRIVEDUO-30 SIMPLE SERIAL BYTE ENCODING FOR DIRECTION AND SPEED 77

TABLE 5.1 CYCLONEDDS COMMUNICATION MODES 88

LIST OF FIGURES

TABLE NO.	TITLE	PAGE NO.
FIGURE 1.1	SUBSYSTEM DOMAINS AND THE INTER-DOMAIN LINKAGES THAT DEFINE THE MECHATRONICS ENGINEERING CONTRIBUTION OF THIS PROJECT	8
FIGURE 3.1	HARDWARE BLOCK DIAGRAM REPRESENTING THE INTERACTIONS OF DIFFERENT HARDWARE COMPONENTS WITH EACH OTHER	22
FIGURE 3.2	PHOTOGRAPH OF THE DANI ROBOT	24
FIGURE 3.3	PHOTOGRAPH OF THE BARE DANI ROBOT WILL ALL ITS ELECTRONICS STRIPPED AWAY	24
FIGURE 3.4	ELECTRONIC COMPONENTS REMOVED FROM THE DANI ROBOT CHASSIS	24
FIGURE 3.5	THE ASSEMBLED ROBOT FRAME WITH THE FOUR CARBON FIBRE CORNER COLUMNS AND ACRYLIC DELIVERY TRAY	25
FIGURE 3.6	THE ROBOT POSITIONED ALONGSIDE A PERSON	25
FIGURE 3.7	PHOTOGRAPH OF THE LOWER SHELF	26

FIGURE 3.8	CLOSE-UP PHOTOGRAPH OF THE DRIVE WHEEL WITH THE NEOPRENE SHEET WRAPPED	27
FIGURE 3.9	REAR MECANUM SUPPORT WHEEL	27
FIGURE 3.10	PHOTOGRAPH OF THE CAMERA MOUNT AT THE FRONT-CENTER OF THE TRAY	28
FIGURE 3.11	RENDER OF THE 3D-PRINTED COMPONENTS	29
FIGURE 3.12	PHOTOGRAPH OF THE ZED MINI CAMERA MOUNTED ON THE ROBOT (RIGHT), AND THE ZED MINI PRODUCT PHOTOGRAPH (LEFT)	30
FIGURE 3.13	PHOTOGRAPH OF THE TF MINI PLUS MOUNTED ON THE REAR OF THE CHASSIS	31
FIGURE 3.14	CLOSE-UP PHOTOGRAPH OF THE ENCODER ON THE MOTOR SHAFT	32
FIGURE 3.15	PHOTOGRAPH AND DATASHEET DRAWING OF THE 739530 MOTOR AND ENCODER	33
FIGURE 3.16	PHOTOGRAPH OF THE MDDS30 MOUNTED ON THE ELECTRONICS SHELF	34

FIGURE 3.17	PHOTOGRAPH OF THE EAGLE 101 CARRIER BOARD WITH THE JETSON NANO MOUNTED	36
FIGURE 3.18	PC817 OPTOISOLATOR (LEFT) AND TXS0108E BIDIRECTIONAL LOGIC-LEVEL SHIFTER (RIGHT)	37
FIGURE 3.19	PHOTOGRAPH OF THE LIPO BATTERY PACK	40
FIGURE 3.20	BLOCK DIAGRAM OF THE POWER DISTRIBUTION ARCHITECTURE	42
FIGURE 3.21	ALTIUM DESIGNER SCHEMATIC OF THE CUSTOM SIGNAL-CONDITIONING PCB	43
FIGURE 3.22	SINGLE-LAYER PCB LAYOUT	44
FIGURE 3.23	PHOTOGRAPH OF THE COMPLETED PERF-BOARD PCB	45
FIGURE 3.24	PHOTOGRAPH OF PCB SHOWING THE POWER LINES	45
FIGURE 4.1	BLOCK DIAGRAM REPRESENTING THE SOFTWARE ARCHITECTURE	51
FIGURE 4.2	SOLIDWORKS CAD SKETCH OF THE HI-TECH BLOCK 2ND FLOOR (DXF EXPORT). CORRIDOR OUTLINES	53

FIGURE 4.3	FULL FLOOR OCCUPANCY MAP AFTER ANNOTATION	55
FIGURE 4.4	INDIVIDUAL TESSELLATED TILE MAPS (TILE 1–5).	56
FIGURE 4.5	TESSELLATED MAP TILE LAYOUT WITH OVERLAP REGIONS.	56
FIGURE 4.6	ASYNCHRONOUS TILE SWITCH STATE MACHINE	60
FIGURE 4.7	RECOVERY STATE MACHINE FOR POSE-LOSS RELOCALIZATION	65
FIGURE 4.8	JOB LIFECYCLE STATE MACHINE	82
FIGURE 5.1	STARTUP DEPENDENCY CHAIN ON JETSON NANO	108
FIGURE 6.1	UI ARCHITECTURE	111

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM/DESCRIPTION
2D	Two-Dimensional
3D	Three-Dimensional
AMCL	Adaptive Monte Carlo Localization
CAD	Computer-Aided Design
CPU	Central Processing Unit
DWB	Dynamic Window B (local trajectory controller in Nav2)
DXF	Drawing Exchange Format
GPIO	General-Purpose Input/Output
GPU	Graphics Processing Unit
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
MCP	Map Change Plugin (custom tile-switching plugin)
Nav2	Navigation 2 (ROS 2 navigation stack)
PCB	Printed Circuit Board
PNG	Portable Network Graphics
PWM	Pulse-Width Modulation
RAM	Random Access Memory
RGB	Red Green Blue (colour channels)
RGB-D	Red Green Blue Depth (colour and depth image)

RFID	Radio-Frequency Identification
ROS 2	Robot Operating System 2
SDK	Software Development Kit
SLAM	Simultaneous Localization and Mapping
TTL	Transistor-Transistor Logic
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VIO	Visual-Inertial Odometry
Wi-Fi	Wireless Fidelity (IEEE 802.11)
ZED	Zero Effort Depth (product line by Stereolabs)
ADC	Analogue-to-Digital Converter
API	Application Programming Interface
ARM64	64-bit ARM architecture (AArch64)
CUDA	Compute Unified Device Architecture (NVIDIA GPU programming platform)
DDS	Data Distribution Service (ROS 2 middleware transport)
DNS	Domain Name System
DWA	Dynamic Window Approach (local trajectory planner)
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure

IP	Internet Protocol
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
JST	Japan Solderless Terminal (connector standard)
NAT	Network Address Translation
ORB	Oriented FAST and Rotated BRIEF (feature descriptor)
PETG	Polyethylene Terephthalate Glycol (3D printing filament)
PGM	Portable Gray Map (occupancy grid image format)
PID	Proportional-Integral-Derivative (control algorithm)
PLA	Polylactic Acid (3D printing filament)
QR	Quick Response (code)
RMW	ROS Middleware (abstraction layer for DDS implementations)
SD	Secure Digital (memory card)
SSH	Secure Shell (remote access protocol)
SSL	Secure Sockets Layer (cryptographic protocol)
UI	User Interface
URL	Uniform Resource Locator
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language (configuration file format)

LIST OF SYMBOLS

SYMBOL	DESCRIPTION
x	Robot position along the x-axis in the map frame (m)
y	Robot position along the y-axis in the map frame (m)
θ	Robot heading angle measured counterclockwise from x-axis (rad)
v	Linear velocity of the robot (m/s)
ω	Angular velocity of the robot (rad/s)
r	Drive wheel radius (m)
L	Wheelbase: distance between left and right drive wheels (m)
v_L	Linear velocity of the left drive wheel (m/s)
v_R	Linear velocity of the right drive wheel (m/s)
d	Obstacle distance measured by the depth sensor (m)
ρ	Map resolution (m/px)
N	Number of occupancy grid cells in a map tile (dimensionless)
$p\ (x \mid z, m)$	Posterior probability of robot pose x given observations z and map m
ε	Localization position error magnitude (m)
σ	Standard deviation of pose estimate uncertainty (m or rad)
t	Time (s)
Δt	Discrete time step between control updates (s)
I	Electrical current (A)

V	Voltage (V)
P	Power (W)
η	Power conversion or motor efficiency (dimensionless, 0 to 1)
b	Track width (wheelbase between left and right wheels) (m)
ωL	Angular speed of the left drive wheel (rad/s)
ωR	Angular speed of the right drive wheel (rad/s)
ω_cmd	Commanded wheel angular speed from kinematic transformation (rad/s)
ω_actual	Measured wheel angular speed from encoder (rad/s)
$e(t)$	Wheel speed error: ω_cmd(t) − ω_actual(t) (rad/s)
$u(t)$	Control output voltage applied to motor (V)
K_ff	Feedforward gain in velocity controller (dimensionless, ≈1.16)
K_p	Proportional gain in PI velocity controller
K_i	Integral gain in PI velocity controller
$\Delta ticks$	Change in encoder tick count over one control interval
$ticks\_per\_rev$	Encoder counts per full wheel revolution
u, v	Pixel coordinates in the camera image plane
x_c, y_c, z_c	3D point coordinates in the camera optical frame (m)
f_x, f_y	Horizontal and vertical focal lengths of the camera (px)
c_x, c_y	Principal point (image center) of the camera (px)
$g(n)$	A* exact cost from start node to node n

$h(n)$	A* heuristic estimate from node n to goal (Euclidean distance)
$c(n, m)$	Edge cost from cell n to adjacent cell m in the occupancy grid
α	Obstacle proximity cost scaling factor in A* cost function
v_max	Maximum allowable linear velocity (m/s)
ω_max	Maximum allowable angular velocity (rad/s)
T	DWB trajectory simulation time horizon (s)
w_path	DWB weight for path distance term
w_goal	DWB weight for goal distance term
w_obst	DWB weight for obstacle distance term

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO THE BROAD FIELD OF WORK

The field of autonomous indoor mobile robotics has undergone a remarkable transformation over the past decade. What was once confined to tightly controlled factory floors, automated guided vehicles following magnetic tapes or buried wire tracks, has evolved into a far broader class of systems capable of operating freely within complex, shared, and unpredictable indoor environments. Hospitals, warehouses, university campuses, corporate office buildings, and research laboratories are all active sites of deployment today, and the pace of adoption is accelerating sharply. Market analysts project the global autonomous mobile robot sector to grow from approximately USD 4.3 billion in 2023 to nearly USD 10 billion by 2032, at compound annual growth rates consistently estimated between 15% and 24% depending on the segment. This reflects a broad industrial and institutional recognition that indoor logistics, material handling, and routine transport are among the most tractable early applications of autonomous robotics in shared human environments.

The reasons for this boom are convergent. On the demand side, labor shortages in logistics and service sectors, rising operational costs, and post-pandemic pressure to reduce human contact in critical delivery workflows have pushed organizations and institutions to look for automation solutions. Hospitals have been early and consistent adopters, autonomous robots now deliver medications, laboratory specimens, and sterile supplies in hundreds of facilities worldwide, freeing clinical staff from repetitive transport duties and reducing infection-risk contact chains. Warehouses and fulfilment centers have deployed mobile robots at massive scale to handle picking, sorting, and inventory replenishment tasks that previously required large workforces. Beyond logistics, academic and institutional environments, the deployment context most directly relevant to this work present similar motivating conditions: routine, repetitive, low-stakes transport tasks between defined locations, carried out repeatedly throughout the working day, with no inherent need for human involvement.

On the supply side, a parallel set of advances has made capable indoor robots practically buildable. Depth-sensing cameras and solid-state sensor technologies have

fallen in cost by orders of magnitude over the past decade, making rich environmental perception accessible at consumer and embedded hardware price points. Embedded computing platforms now provide GPU-accelerated inference and multi-core processing within the thermal and power envelope of a small circuit board. Open-source robotics middleware frameworks have matured to the point where foundational capabilities, sensor abstraction, inter-process communication, navigation pipeline architecture, are available as production-grade components ruling out the need for prototypes requiring months of custom engineering. These supply-side developments have collectively lowered the barrier to building and deploying capable mobile robots, enabling university research groups and small companies to field systems that would have required dedicated industrial engineering teams a decade earlier.

At the technical core of any autonomous mobile robot is the problem of navigation: the ability to determine the robot's position within its environment, plan a route to a given destination, and execute that route safely in the presence of other occupants and dynamic obstacles. Indoor navigation is structurally different from outdoor navigation in ways that make it non-trivially harder. Satellite-based global positioning is unavailable indoors, eliminating the simplest source of absolute position information. Indoor environments are geometrically repetitive, long uniformly tiled corridors, and open-plan spaces with few distinctive landmarks, which confound the feature-matching and loop-closure mechanisms that localization algorithms rely on. Lighting conditions vary widely across different zones and times of day, degrading the reliability of vision-based perception. And unlike outdoor spaces, indoor environments are densely shared: people move unpredictably, doors open and close, furniture is rearranged, and corridors are temporarily blocked by deliveries, cleaning equipment, or gatherings. A navigation system that works correctly in an empty building at with no crowd must continue to work correctly during its peak hours.

The dominant technical approach to indoor navigation has evolved through several generations. Early systems used infrastructure-dependent localization: The shift to infrastructure-free navigation, where the robot determines its position purely from onboard sensor data interpreted against a map of the environment, represents the defining technical advancement of the current generation of indoor mobile robots.

Two broad strategies dominate infrastructure-free indoor navigation. The first is Simultaneous Localization and Mapping, or SLAM, in which the robot builds a map of its environment in real time while simultaneously estimating its own position within that map. SLAM is appealing as it requires no prior survey of the environment, the robot can navigate a space it has never visited before, constructing its spatial model on the fly. Research in SLAM has been extraordinarily productive, producing algorithms that work from laser rangefinders, RGB-D cameras, stereo cameras, and combinations thereof, and that handle a range of environment types with increasing robustness. However, SLAM remains computationally demanding, particularly in large or geometrically ambiguous environments where data association errors, mistakenly matching observations from different locations can accumulate into substantial drift. In long, featureless corridors, the identical appearance of consecutive segments creates persistent challenges for loop closure, the mechanism by which SLAM corrects accumulated position error by recognizing previously visited places.

The second strategy is localization against a pre-built static map. Here, the environment is surveyed in advance, either by running a mapping algorithm offline or by constructing the map through direct measurement, and the resulting representation is stored as a fixed reference. During operation, the robot estimates its position by comparing live sensor observations against this stored map. This approach is generally more computationally tractable than SLAM during operation, since the map is not being modified at runtime, and it tends to produce more consistent localization behavior in familiar environments. Its limitation is inflexibility: changes to the environment that are not reflected in the stored map can degrade the localization accuracy, and the map must be re-built if the environment changes significantly.

Obstacle avoidance sits alongside localization as a core challenge of indoor navigation, and it is here that the shared nature of indoor environments introduces the most demanding requirements. Static obstacles like walls, furniture, and fixed equipment can be represented in the map and avoided through path planning. Dynamic obstacles like people, trolleys, doors, and temporarily placed objects, must be detected and responded to in real time. The distinction matters because the appropriate response to a dynamic obstacle depends not just on its presence, but on what it is: a person standing in a corridor warrants a fundamentally different robot behavior than a box left on the floor. A person is likely to move, may be startled or endangered by an

aggressively replanning robot, and is subject to safety norms that a stationary box is not. This has driven significant research interest in semantically aware obstacle handling systems that use object detection and classification to identify what type of obstacle is present and select a behavior accordingly, rather than treating all dynamic obstacles as equivalent geometry to be avoided.

The sensor technologies underpinning modern indoor navigation have also diversified considerably. Laser rangefinders, long the dominant sensor for 2D indoor mapping, remain widely used for their precision and robustness across lighting conditions. Depth cameras, both structured-light and stereo-vision variants have grown substantially in deployment, offering richer 3D environmental information at lower cost and in smaller form factors. Stereo cameras in particular have attracted significant research interest because they provide both depth data for obstacle detection and RGB imagery for object recognition, enabling a single sensor to serve multiple functions within the navigation and perception pipeline. The fusion of visual data with inertial measurement unit readings, which provide high-frequency estimates of linear acceleration and angular velocity, has become a standard approach to robust state estimation: visual odometry drifts over long traversals, while IMU data captures short-timescale motion faithfully, and the combination through sensor fusion yields estimates more reliable than either source alone.

The result of these converging developments is a field that is simultaneously well-established and actively evolving. Core navigation capabilities like localization, path planning, and basic obstacle avoidance are sufficiently mature that they can be assembled from open-source components into functioning systems. The active frontier lies in making these systems robust, efficient, and deployable under real-world constraints: operating on embedded hardware with fixed compute and memory budgets, handling the full diversity of dynamic human environments, maintaining consistent performance over long operational periods, and integrating cleanly with the institutional workflows they are meant to support. It is within this context, a field past its formative research phase but still short of solved engineering that the present work situates itself, addressing the specific challenges of autonomous material transfer in a shared institutional building environment.

1.2 OBJECTIVES

The primary objective of this project is to design and develop an autonomous indoor delivery robot capable of transporting materials between the entrances of staff rooms, classrooms, and laboratories on a single floor level. The robot operates within the corridor space of the floor, navigating autonomously from a pickup point to a destination without manual intervention. The aim is to reduce the burden of routine inter-room transport on building occupants and to demonstrate a fully operational autonomous logistics platform in a busy institutional environment.

To accomplish this, the following specific tasks are to be accomplished:

- To construct an accurate spatial representation of the deployment environment and implement reliable localization using onboard sensors, enabling the robot to determine its position at all times and plan optimal paths to any designated location.
- To enable safe and reliable autonomous navigation in a shared, dynamic indoor environment. The robot must plan collision-free routes and execute them in the presence of building occupants and unpredictable obstacles. Crucially, it must respond to humans differently from other obstacles, the system must be capable of safe coexistence with people sharing the same corridors, not merely avoiding them as generic geometry.
- To develop a user-friendly interface through which building occupants can request deliveries and track mission progress in real time.

These objectives together define the scope of the system developed in this work. They span localization, map management, navigation, human-aware operation, task handling, and user interaction, the full set of capabilities required for a functionally complete indoor delivery robot operating on embedded hardware in a busy building environment.

1.3 PERSPECTIVE OF PROJECT

An autonomous indoor delivery robot is not the product of any single engineering discipline. It draws simultaneously from mechanical design, power electronics, embedded computing, computer vision, perception, localization, navigation, and control domains that each carry their own depth, their own research

communities, and their own body of specialist knowledge. A computer vision researcher could spend a career advancing stereo depth estimation. An embedded systems engineer could focus entirely on optimizing real-time compute scheduling on constrained hardware. A robotics researcher could work exclusively on improving the mathematical foundations of localization algorithms. Each of these is a legitimate and substantial field of work in its own right.

The mechatronics engineer's role in a project of this kind, is fundamentally different. The goal is not to push the frontier of any one of these subsystems. It is to understand each domain well enough to make informed decisions about what to adopt, how to configure it, and critically, how to make it work with everything else. The mechatronics perspective is a systems perspective: the question of interest is not how good the perception system is in isolation, but whether the perception system produces output that the navigation stack can consume reliably, at the right rate, under the compute constraints imposed by the embedded platform, while the motors are drawing transient current loads that introduce noise into the power rail that the sensor electronics share. These are not problems that belong to any single domain. They only appear and can only be solved at the boundaries between domains.

This project spans exactly that class of problem. The mechanical platform constrains what sensors can be mounted and where, which determines the field of view available to the perception system, which in turn determines what the localization algorithm sees. The power architecture determines how much current is available to the computing hardware, which determines how much processing headroom the navigation stack has at peak motor load. The stereo camera produces depth data at a fixed resolution and frame rate, and the embedded processor must run perception, localization, navigation, communication, and control concurrently within a fixed memory budget, a constraint that requires deliberate co-design across the software stack rather than optimizing each component independently. The choice of depth-based obstacle detection connects directly to the drive controller through a software interface that must be correctly specified, timed, and handled for the robot to respond to obstacles safely. Each of these connections requires the engineer to understand both sides of the interface and to design the boundary between them with the whole system's behavior in mind.

This is the nature of mechatronics engineering as a discipline: not the depth of a specialist but the breadth and integration thinking of a systems engineer who works at the intersections. A mechanical engineer hands over a chassis; an electrical engineer hands over a power-regulated drive circuit; a software engineer hands over a navigation algorithm. The mechatronics engineer is responsible for the question that none of these individually answers: does it work as a whole? Does the chassis geometry allow the robot to turn in the corridors it will actually navigate? Does the power system remain stable when the processor is under full load, and the motors are accelerating simultaneously? Does the sensor output reach the navigation stack in time to prevent a collision? These are integration questions and answering them is the primary engineering contribution of this project.

The subsystems this project brings together span stereo vision for depth perception and object detection, visual-inertial odometry for localization, embedded GPU computing for real-time inference and navigation, differential drive actuation for motion, power electronics for stable energy distribution under variable load, and a web-based task management layer for operator interaction. The contribution is the working system, the evidence that these components, drawn from different domains, can be made to function together reliably enough to perform autonomous material delivery in a real building.

The integration of these subsystems produces a closed-loop operational architecture in which sensing, decision-making, and actuation are continuously and mutually dependent. Accurate pose estimation is contingent on consistent actuation and reliable sensor data; path planning is only as good as the pose estimate it receives; and motion execution is directly shaped by planner output. This interdependence across the mechanical, electrical, and software engineering domains is the defining characteristic of a mechatronic system as illustrated in figure 1.1, is precisely this coordinated design that enables the robot to perform autonomous delivery operations reliably within a structured indoor environment.

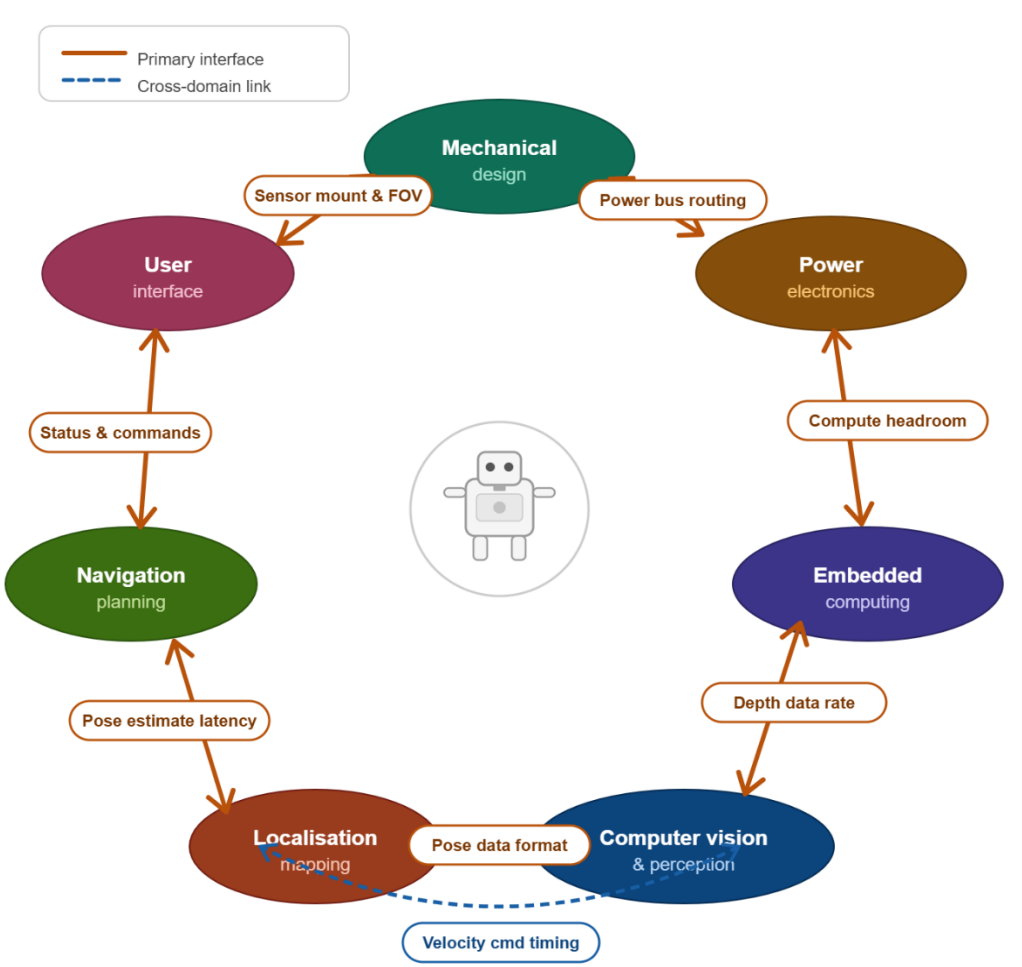

Figure 1.1 Subsystem domains and the inter-domain linkages that define the mechatronics engineering contribution of this project

1.4 CONTEXT AND MOTIVATION

Every engineering system is shaped as much by what it cannot do as by what it can. The floor-level material transfer robot is designed within a specific and deliberate set of operational boundaries, each of which reflects a real constraint of the deployment environment or a considered decision about where to draw the scope of the initial implementation. Understanding these boundaries is essential to understanding the design, not as limitations, but as the conditions that give the system its particular character.

Floor Level: The robot operates on a single floor of the Hi-Tech Block, navigating through corridor spaces and delivering to room entrances. It does not enter rooms, does not climb stairs, and has no elevator integration. This scope reflects the practical reality that a single floor of an institutional building already presents a complete and non-

trivial navigation problem, and that confining the operational domain to a single level eliminates an entire class of complexity that would not add proportional value to the core deliverable.

Terrain: The physical environment imposes its own conditions. The corridors are flat, even, and adequately lit throughout operational hours. Flat surfaces are a prerequisite for reliable visual odometry, stereo depth estimation and pose tracking both degrade on sloped or uneven terrain where the camera's assumed ground plane diverges from reality. Adequate lighting is similarly load-bearing for the perception system: stereo vision depends on ambient illumination to extract depth, and a deployment environment with consistent lighting conditions removes a significant source of variability from the perception pipeline. Stable Wi-Fi coverage across the floor supports the communication link between the robot and the web-based task management system, ensuring that delivery requests and status updates are exchanged reliably throughout the operation.

Payload: Payload is limited to lightweight materials like files, documents, laboratory items, and similar objects within approximately one kilogram. The drive system is sized for corridor maneuvering, not load-bearing transport, and keeping payload within this range ensures the motors can achieve precise movement and controlled stopping without overloading the drivetrain or introducing torque-induced errors. Mechanical stability under load also matters for the sensor platform: an unbalanced payload shifts the robot's center of mass and can introduce vibration or tilt that degrades the quality of depth and IMU data.

Endurance: The robot runs on an onboard battery, making it fully mobile and independent of fixed power infrastructure. The tradeoff is a finite operational runtime bounded by battery capacity accepted as a practical condition of the deployment. Complete mobility and freedom from external power connections is more operationally useful than extended runtime tethered to a charging point, for a system whose task is to move freely between any two locations on the floor.

Safety: Safety in a shared environment is a non-negotiable constraint. The corridors are used by students, staff, and faculty throughout the working day, and the robot must coexist with them without creating hazards or disruptions. When a person is detected in the robot's path, the system stalls and waits for the path to clear rather than attempting to navigate around the individual. This behavior is deliberate: replanning around a

moving person introduces unpredictable trajectory changes at the exact moment when predictability matters most for safety. Stalling is conservative, but it is legible, the person can see what the robot is doing and respond accordingly. For non-human obstacles such as a bag, a dustbin, or a misplaced chair, the robot replans, since these are static objects that can be routed around without the same safety considerations applying.

1.5 CONTRIBUTION AND SIGNIFICANCE

This project makes two major technical contributions to autonomous indoor navigation on embedded hardware.

The first is a pose-triggered tile-switching map management system built on top of the standard Nav2 stack and deployed on an NVIDIA Jetson Nano. Prior zone-based navigation systems trigger map transitions using external fiducial markers whose detection is unreliable under variable lighting and introduces an external dependency the robot cannot control. The system presented here uses only the robot's own real-time pose estimate to detect entry into pre-defined trigger zones and initiate an asynchronous map reload, eliminating the most commonly reported failure mode without modifying the Nav2 codebase or the deployment environment. The result is a 93.8% reduction in the active costmap footprint relative to a single full-floor occupancy grid, which is what allows the full Nav2 stack to run reliably within the Jetson Nano's 4 GB RAM constraint.

The second contribution is a context-aware obstacle response architecture. A custom DWB controller plugin integrates the ZED SDK's multiclass object detection pipeline to distinguish human from non-human obstacles in real time. When a person is detected in the robot's path, the system stalls and waits; when an inanimate obstacle is detected, it triggers global path replanning. This semantic distinction is absent from Nav2's standard obstacle avoidance configuration. The implementation demonstrates that differentiated obstacle handling can be achieved on embedded hardware using only a stereo camera.

The broader significance of this work is its deployment-first orientation. Both contributions are validated not in simulation but in the actual operational environment: the second floor of the Hi-Tech Block at SRMIST KTR, with real building occupants, real lighting variability, and real embedded hardware constraints. The project

demonstrates that a capable, sensor-minimal autonomous delivery platform can be assembled from open-source middleware and commodity embedded hardware, and operated reliably in a shared institutional environment, without custom navigation frameworks or dense sensor arrays.

1.6 OVERVIEW OF THE PROJECT

The project was carried out through a series of structured activities spanning the full development lifecycle of an autonomous indoor delivery robot. These activities are summarised below and are described in detail in the chapters that follow.

Mechanical Design and Fabrication: The physical platform was designed from the ground up to suit the corridor environment of the second floor of the Hi-Tech Block. This included selecting the DANI differential-drive chassis, designing the carbon-fibre and acrylic superstructure to raise the delivery tray to an ergonomic handoff height, and fabricating and assembling all structural components in-house.

Electronics and PCB Design: A custom PCB was designed to integrate the motor drivers, power regulation circuitry, and sensor interfaces. The board consolidates connections between the embedded controller, the motor controllers, and the other components, keeping the wiring harness compact and the power distribution reliable.

Map Generation and Environment Survey: The second floor was manually measured and converted into a 2D occupancy grid. The full floor map was then divided into five overlapping tiles suited to the robot's operating zones, and a YAML-based configuration was authored to define room positions, tile extents, and zone boundaries.

Algorithm Development: The core algorithmic contributions of the project were developed and implemented across three areas: (i) a custom tile-switching Nav2 plugin that monitors the robot's pose and hot-swaps the active map tile at zone boundaries without restarting the navigation stack; (ii) a context-aware obstacle avoidance module that classifies detected objects and applies a differentiated response, i.e. stall-and-wait for people, global replanning for inanimate obstacles; and (iii) a ROS 2 job manager implementing a ten-state delivery state machine with two-stage physical confirmation gating.

System Integration: All hardware and software subsystems were brought together on the robot and deployed in the corridor environment. This included configuring the ROS 2 middleware, setting up Nav2 with the tiled map pipeline, establishing the visual-

inertial odometry localisation chain, and wiring the web application to the ROS 2 stack via the nano agent.

Software Development: A full-stack web application was developed to serve as the user interface and operational backend. The frontend is a browser-accessible HTML page; the backend is a FastAPI application with a PostgreSQL database hosted on Railway. A lightweight agent on the Jetson Nano bridges the cloud backend to the ROS 2 stack via HTTP polling.

Testing and Evaluation: The system was tested through a series of trials in the actual deployment corridor. Tests covered autonomous navigation across tile boundaries, obstacle detection and avoidance response times, end-to-end delivery cycle completion, web interface responsiveness, and system behaviour under edge cases such as confirmation timeouts and job cancellation during active navigation.

1.7 NOTATIONS AND CONVENTIONS

The following conventions are used consistently throughout this report.

Coordinate frames follow the ROS 2 standard: the *x*-axis points forward, the *y*-axis points left, and the *z*-axis points upward. Subscript notation distinguishes between reference frames: the subscript '*odom*' denotes the odometry frame, '*map*' denotes the map frame, and '*base*' denotes the robot base frame.

Units follow SI conventions throughout. Distances are reported in metres (*m*), angular quantities in radians (*rad*), map resolution in metres per pixel (*m/px*), and time durations in seconds (*s*) or milliseconds (*ms*) as appropriate to the context.

Acronyms and abbreviations are defined at first use in the text and are collected in the List of Abbreviations. Mathematical symbols are defined at first use and are listed in the List of Symbols. Where a symbol appears in an equation, its definition is also restated in the surrounding text for readability.

1.8 ORGANIZATION OF THE REPORT

The report is structured to follow the natural sequence of the project: from motivation and prior work, through the design and build of the system, to its integration, deployment, and evaluation.

Chapter 1 (this chapter) establishes the field context, defines the project objectives and operational scope, and states the two principal technical contributions.

Chapter 2 presents the literature survey. It is organised around three specific questions that shaped the survey: what map representation strategies exist for large indoor environments, runtime map switching implemented in ROS 2, and existing approaches which satisfy all the constraints of this system simultaneously. The chapter closes with the key finding that motivated the approach taken in the current work.

Chapter 3 covers the system hardware: mechanical design and chassis fabrication, sensor selection, actuation, embedded computing platform, power architecture, and PCB design.

Chapter 4 describes the system software: the software architecture, map generation and tile management pipeline, localisation using visual-inertial odometry, global and local path planning, the context-aware obstacle avoidance implementation, and the task assignment web application.

Chapter 5 presents system integration: how the hardware and software components are brought together in the deployment environment, including the ROS 2 middleware configuration, Nav2 setup, the system supervisor, and the full-stack launch configuration.

Chapter 6 documents the user interface and its integration with the robot's navigation and task management backend, including the full operational stage sequence from robot idle to delivery complete.

Chapter 7 presents the project conclusions and outlines directions for future work, including multi-floor navigation, improved localisation under lighting variation, and multi-robot coordination.

CHAPTER 2

LITERATURE SURVEY

This chapter surveys existing research that informed the design decisions made in this project. The survey was conducted not as a general review of mobile robotics, but as a targeted investigation into three specific technical problems encountered during the early design phase: how large indoor maps should be represented on resource-constrained hardware, whether runtime map switching has been attempted in ROS 2 and what failure modes arose, and whether any existing approach could satisfy all the constraints of this system without requiring a custom framework or external fiducial infrastructure. The sources, selection criteria, and findings are organized accordingly.

2.1 PURPOSES FOR THE SURVEY

The map representation problem arose directly from the deployment environment. The second floor of the Hi-Tech Block at SRMIST spans approximately 60 m × 45 m. Represented as a single 2D occupancy grid at 0.04 m/cell, this produces a map of 1,466,724 cells, of which only 11.7% are navigable corridor space. The remaining 88.3% are walls and room interiors the robot will never enter. Running Nav2's costmap and global planner over the full grid means allocating and computing over cells that serve no navigational purpose, on a Jetson Nano with 4 GB RAM and a shared CPU-GPU pipeline that has no spare capacity for unnecessary load.

The obvious fix was to break the map into smaller pieces and load only the piece the robot currently needs, which immediately raises a question the team had no ready answer to: how do other systems handle this, and what goes wrong when they try? That question drove the literature survey. The survey was not a general review of mobile robotics; it was focused specifically on map representation strategies and runtime map management for embedded 2D navigation systems. Three questions shaped it:

The survey was conducted with three specific questions in mind:

- What map representation and memory management strategies exist for large indoor environments, and how do they perform on embedded platforms?
- Have others implemented runtime map switching or zone-based navigation in ROS 2, and what failure modes did they encounter?

- Is there an existing approach that simultaneously satisfies all the constraints of this system: 2D occupancy grid, standard Nav2, embedded hardware, no custom framework, no external fiducial infrastructure?

2.2 SOURCES FOR THE SURVEY

Sources were drawn from IEEE Xplore, ScienceDirect, arXiv, MDPI, and Springer. Search terms were kept narrow: occupancy grid memory optimization, runtime map loading in ROS 2, zone-based and submap navigation, visual-inertial odometry for indoor localization, and long-term SLAM on embedded platforms. Material transport robots were included only to establish what failure modes practitioners report in real deployments, not as a primary research area.

Results were filtered for two criteria: relevance to 2D navigation on resource-constrained hardware, and whether the work was tested on a physical platform rather than simulation only. Backward citation tracing from Sousa et al.'s 2023 systematic review [4] recovered the foundational map representation works (OctoMap [2], ColMap [1], Gonzalez-Arjona [3]). The ten papers retained cover the period 2013 to 2025; recent works from 2024 and 2025 were prioritized for the map switching and semantic zone categories, where the field is still moving.

2.3 REVIEW OF ARTICLES

The articles are grouped by the specific question they address. The first group establishes the operational context for indoor delivery robots. The second and third groups address map switching and hierarchical navigation directly. The fourth group covers map compression and segmentation strategies. The final two groups address the sensor and localization components. Each group ends with an assessment of what it leaves unresolved.

2.3.1 Autonomous Indoor Mobile Robots for Material Transport

Leong and Ahmad (2023) surveyed a decade of autonomous load-carrying robot development across indoor settings and identified three recurring failure modes: navigation through doorways and corners, payload stability under acceleration, and sensor-actuator interference during motion. These problems appear consistently regardless of platform choice or navigation framework. Lackner (2024) examined the same space from an industrial deployment perspective and added a fourth: graceful degradation. Peak lab performance does not predict field reliability. Systems that

perform well in controlled conditions frequently fail during long operational runs when unexpected states accumulate sensor drift, battery voltage drop, transient network loss. Do and Dang (2025) implemented a working warehouse transport robot and confirmed that robustness against dynamic obstacles and irregular layouts is the primary unsolved challenge in practice. Brorsson et al. (2025) extended this to multi-robot factory settings, where real-time coordination under load is the bottleneck.

What these papers collectively establish is the operational baseline: indoor delivery robots are a solved problem only in controlled environments. In real corridors with people, varying light, and limited compute, the margin for waste is small. Running a planner over a map three times larger than necessary is exactly the kind of waste this baseline says will cause failures in the field.

2.3.2 Map Switching and Zone-Based Navigation

Two groups implemented zone-based map switching in ROS 2 and both encountered the same failure pattern. The ETH Zurich Autonomous Systems Lab (2022–2023) triggered tile transitions using AprilTag detection on TurtleBot platforms. Switching worked in steady-state, but three specific problems emerged: AMCL diverged after some reloads because the initial pose estimate for the new map was poorly conditioned; timing gaps between the map server update and the costmap rebuild caused the planner to route on stale data; and unreliable tag detection in variable lighting could stall the robot at a zone boundary indefinitely. The TU Munich group (2023) used ArUco markers in an industrial setting and encountered the same three issues, plus a fourth: premature switching when the robot was near a zone boundary but not yet committed to crossing it. Their mitigation required layered filtering and explicit validation logic, adding latency and complexity without fully resolving any of the root causes.

Horelican (2022) [6] provided the quantitative framing. Evaluating Nav2 in a live industrial facility, he measured that large high-resolution maps create processing bottlenecks that cannot be compensated by tuning elsewhere in the Nav2 stack. The bottleneck is structural: the planner's data structures scale with map size, and no parameter adjustment changes that. His conclusion is that map size must be managed as a first-class engineering constraint rather than a fixed input to the system.

The common thread across all three works is that the transition mechanism was the weak point, not the zone decomposition idea itself. Fiducial markers introduce an external dependency that the robot cannot control. A switching mechanism that depends entirely on the robot's own pose and heading, with no external sensing required, would eliminate the most common failure modes.

2.3.3 Multi-Floor and Hierarchical Navigation

The University of Bonn (2021–2022) applied the same per-zone loading idea across building floors: each floor is an independent map, reloaded when the robot reaches a transition point marked with a QR code. The memory benefit was confirmed. Two problems appeared that did not exist in the single-floor zone switching work: localization drift accumulated over the long stretches between transition markers, and global planner resets at floor transitions introduced navigation pauses that grew longer with map complexity.

Yun and Yoo (arXiv, 2025) [9] proposed a more principled decomposition strategy. Rather than cutting by floor or corridor geometry, they partition the environment by semantic category, corridors, rooms, labs, and load only the zones relevant to the current route. In principle this is a better fit for irregular building layouts than geometry-based cuts. In practice the approach has two constraints that rule it out for this project: it requires manual semantic annotation for every new deployment environment, and the entire system is built around 3D mapping in RTAB-Map, not the 2D occupancy grid format that we plan to use. Adapting it to a 2D Nav2 deployment would require replacing the core map representation and the localization backend.

2.3.4 Map Compression and Segmentation

Four papers address the map size problem through compression or alternative representation, and all four hit the same wall from a different direction.

Hornung et al.'s OctoMap (2013) [2] achieves up to 97% memory reduction over a full 3D voxel grid through octree pruning. Fisher et al.'s ColMap (2021) [1] improves on this in 2D with an N^2-tree and run-length encoding, reaching 12–15 times better compression at equivalent resolution. Both are genuine improvements on uncompressed grids. Neither solves the runtime loading problem: the compressed structure still represents the entire environment, and Nav2 still builds its costmap across all of it. Compression reduces the bytes stored, not the cells the planner reasons over.

Baek and Park (2025) [8] take a different approach with RC-Map: partitioning free space into rectangles so the planner operates on a rectangle graph rather than a cell grid. The efficiency gains are real, planning computation drops to 1.5% and memory to 3% of conventional grid costs. The cost is Nav2 incompatibility: RC-Map requires a custom planner that cannot be swapped in for the standard Nav2 global planner without modifying the navigation stack.

Gonzalez-Arjona et al. (2013) [3] simplified the representation further, using binary occupancy rather than probabilistic values to fit within FPGA memory. This works for their specific hardware target but still keeps the entire floor in memory and does not generalize to runtime partial loading.

Wang et al. (2025) [10] address a related but distinct problem: stitching locally built submaps into globally accurate maps during the mapping phase. Their system is designed for map construction, not runtime navigation on a pre-built map. It is not applicable here.

The pattern is consistent: compression and alternative representation strategies reduce the resource cost of a global map, but none of them support loading only a portion of that map at runtime while keeping full compatibility with the standard Nav2 stack.

2.3.5 Vision-Based Perception for Navigation

Vega-Torres et al. (2024) [7] demonstrated tightly coupled visual-inertial fusion aligned to a reference map, achieving pose convergence in 11–21 seconds versus 42–80 seconds for standard AMCL. Their platform uses 3D LiDAR rather than a stereo camera, but the localization result is directly relevant: fusing camera and IMU data produces faster, more stable pose estimates than AMCL alone in corridor environments. The documented degradation modes, poorly lit corridors, textureless wall surfaces are exactly the conditions this project operates in and require physical validation, not simulation.

The broader stereo vision literature identifies three consistent indoor failure modes: repetitive textures (identical tiles, uniform corridor paint) confuse feature-based tracking; reflective floors produce spurious depth returns that corrupt the local costmap; and running stereo depth estimation alongside full Nav2 is compute-heavy on embedded hardware. These are known, bounded problems. They do not make stereo-

based localization infeasible indoors, but they set the conditions under which the system must be tested.

2.3.6 Long-Term Localization and SLAM

Sousa et al. (2023) [4] synthesized 142 SLAM and localization papers and identified two strategies that consistently appear in large-scale deployments: map sparsification (representing only the parts of the environment that carry localization information) and runtime map swapping (loading and unloading map segments as the robot moves). They found that most systems implement one or the other in forms that do not generalize, the techniques are environment-specific or require manual tuning per deployment. No surveyed system implemented both in a form that was simultaneously compatible with a standard navigation stack and deployable without modification across new environments. This is the clearest statement in the literature of the gap this project fills.

Schaefer et al. (2021) [5] implemented sparse landmark-based localization from LiDAR pole features, achieving 0.284 m mean accuracy with orders of magnitude less memory than a dense grid. The approach is practically incompatible with indoor corridors: the consistent vertical structures (poles, lamp posts) that it relies on for landmark extraction are absent in corridor environments. It is relevant here as an example of what sparsification looks like when it works, and as a confirmation that the specific sparsification strategy has to match the environment.

2.4 KEY FINDINGS

The surveyed works converge on three findings, each of which shaped a specific design decision in this project. The table below maps each work to the relevant criterion.

Table 2.1: Comparison of existing approaches with the proposed approach

Area / Paper	Their Approach	This Project's Approach
OctoMap (Hornung et al., 2013)	Octree-based 3D global map; up to 97% compression over full 3D grid; entire map kept in memory.	Pre-built 2D tiles; only the active tile is in memory at any time, independent of total floor size.

ColMap (Fisher et al., 2021)	2D N^2-tree with run-length encoding; 12-15x better than OctoMap; still a single global structure.	No global structure maintained at runtime; tile replacement keeps active costmap bounded.
RC-Map (Baek & Park, 2025)	Partitions free space into rectangles; planning at 1.5% of grid cost; requires custom planner.	Standard Nav2 planner used on a reduced-size occupancy grid tile; no custom planning code needed.
Submap Joining (Wang et al., 2025)	Joins locally built submaps into a globally optimized map; suited for map construction phase.	Map is pre-built offline; onboard sensors are used only for localization within the existing map.
Semantic Zones (Yun & Yoo, 2025)	Divides environment by semantic labels (corridor, room); loads relevant zones in RTAB-Map.	Tile boundaries follow corridor geometry, not semantic labels; works within standard Nav2.
ETH Zurich Multi-Zone (2022-2023)	Zone switching triggered by AprilTag detection; tile maps in ROS 2 on TurtleBot.	Switching triggered purely by robot pose and heading; no external fiducial infrastructure required.
TU Munich Sub-Map (2023)	ArUco-triggered map reload with filtering to prevent premature switching events.	Heading-aware trigger zones with switch cooldown handle directionality without marker detection.
Nav2 Industrial Eval (Horelican, 2022) [6]	Single high-resolution global map; identifies map size as primary bottleneck in Nav2.	Active costmap reduced by 93.8% through tessellation, directly addressing the identified bottleneck.
SLAM2REF (Vega-Torres et al., 2024)	Tightly coupled LiDAR-inertial localization aligned to a reference map; 6-DoF pose estimation.	Visual-inertial odometry from ZED Mini used for 2D pose estimation within a pre-built floor map.
Long-Term SLAM Survey (Sousa et al., 2023)	Identifies runtime map swapping and sparsification as key strategies; notes lack of integrated solutions.	Implements runtime tile swapping in a fully integrated Nav2 system on an embedded platform.

Three findings follow directly from the table.

- First, compression is not the same as exclusion. OctoMap and ColMap reduce the bytes stored, but the Nav2 costmap is still built over the entire environment. The planner still allocates data structures proportional to total map size. Compression makes the problem smaller; it does not make it bounded. What the project needed was a mechanism to exclude the regions the robot is not navigating from memory entirely, not a smaller representation of all of them.
- Second, external switching triggers are the primary failure mode in every zone-based system surveyed. AprilTags and ArUco markers create a dependency on environmental infrastructure that the robot cannot verify or recover from if detection fails. The robot's own pose and heading are always available from the navigation stack. A trigger mechanism built on pose and heading alone has no external dependency to fail.
- Third, no surveyed approach satisfies all five criteria simultaneously: 2D occupancy grid format, standard Nav2 compatibility, runtime partial loading, embedded platform deployment, and no custom framework modification. OctoMap and ColMap fail on partial loading. RC-Map fails on Nav2 compatibility. Semantic zones fail on 2D format. The fiducial-based systems satisfy partial loading but introduce an uncontrolled external dependency. This is the gap the tile-based switching system addresses.

One distinction is worth stating explicitly. The sensors in this project are used only for localization within a pre-built map and not for building the map. The floor map was constructed offline from physical measurements and CAD drawings. The ZED Mini's visual-inertial odometry gives the robot its pose estimate within that fixed map at runtime. This separates this project from all the SLAM-based approaches in section 2.3.6, where sensors are used to build and update the map simultaneously with navigation. The map is static; the robot only navigates within it.

CHAPTER 3

SYSTEM HARDWARE

3.1 HARDWARE BLOCK DIAGRAM

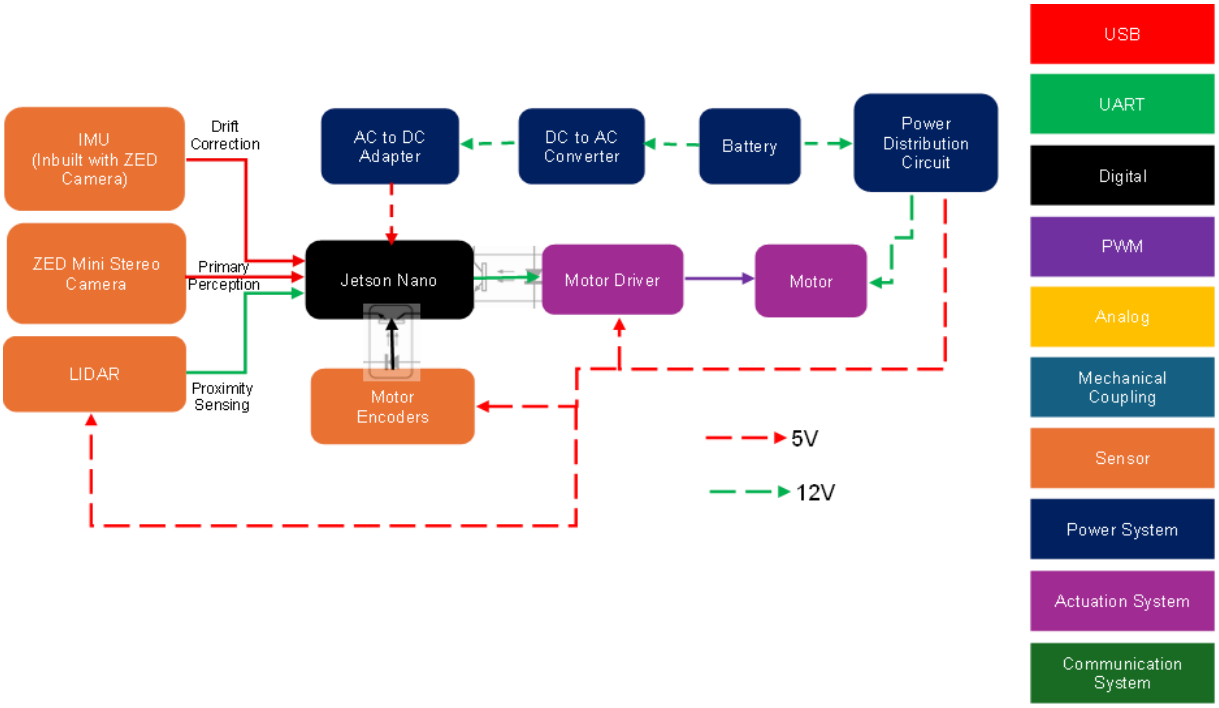

Figure 3.1 Hardware block diagram representing the interactions of different hardware components with each other

The hardware block diagram presents the system as six interconnected functional layers: power, actuation, sensing, compute, control, and communication. Together these define the physical architecture of the robot before any software is considered.

At the base is a differential-drive chassis (derived from the DANI robot platform) driven by two independently controlled DC motors. Low-level motor control is handled by a Cytron Smart Duo 30 motor driver, which receives velocity commands directly from the Jetson Nano. Wheel encoder signals are routed back to the Jetson Nano, giving the compute unit direct access to odometry feedback. The onboard battery powers the drive system directly on its primary rail. A separate buck converter steps the supply voltage down to 5 V for the sensor and peripheral electronics. The Jetson Nano is protected from power transients on the 5 V rail by an opto-isolator, and a logic-

level shifter translates the 5 V logic of peripheral components down to the 3.3 V logic level of the Jetson's GPIO.

The primary perception unit is a ZED stereo camera, which provides RGB imagery, and depth maps, these data are used for visual-inertial odometry via its onboard IMU, forward obstacle detection, and object classification. A single point LiDAR is mounted to cover the robot's rear arc, providing proximity detection during reverse manoeuvres to prevent rearward collisions that the forward-facing ZED cannot observe.

All onboard processing is handled by the NVIDIA Jetson Nano, which acts as the central compute unit. It receives sensor data from both the ZED camera and the wheel encoders, issues commands to the motor driver, and maintains the Wi-Fi communication link to the web-based task management backend through which delivery requests are received and mission status is reported.

The sections that follow examine each of these layers in detail, mechanical design and fabrication (3.2), sensors (3.3), actuators (3.4), the controller (3.5), electrical interfaces and communication (3.6), power supply circuitry (3.7), PCB design (3.8), and finally the robot specifications (3.9).

3.2 MECHANICAL DESIGN AND FABRICATION

The mechanical design of the robot addresses two requirements: keeping the overall structure light enough to maximize usable payload and raising the delivery tray to a height that makes it visible and accessible in a corridor environment. The following subsections describe the base chassis, the elevated frame, the tray assembly, the electronics mounting arrangement, the drive and support wheel configuration, and the sensor mount.

3.2.1 Base Chassis

The robot is built on the frame of the DANI differential-drive robot platform. The chassis was stripped of its original onboard electronics and used purely as the structural and drivetrain base. The DANI frame provides the motor mounting geometry, the wheel axle positions, and the lower structural envelope within which the drive and support wheels are arranged. No modifications were made to the chassis frame itself;

all additions are mounted onto it through 3D-printed interface brackets described in section 3.2.6.

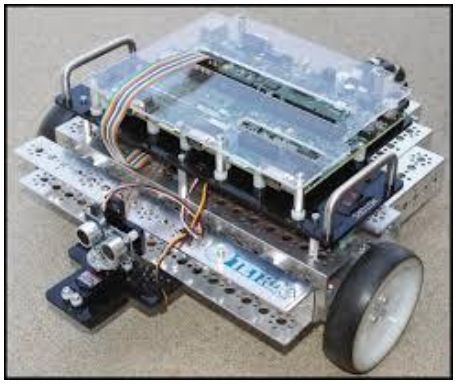

Figure 3.2 Photograph of the DANI Robot
Source: "Mobile Robotics Experiments with DaNI" Manual from National Instruments [11]

Figure 3.3 Photograph of the Bare DaNI Robot will all its electronics stripped away

Figure 3.4 Electronic components removed from the DaNI Robot chassis

3.2.2 Elevated Frame and Delivery Tray

The delivery application requires the tray to be visible to corridor occupants and easily reachable by a standing person. To meet this, the tray platform is elevated to 800 mm from the ground. Four vertical carbon fibre tubes of 1 mm wall thickness, one at each corner of the chassis, form the supporting columns for the elevated structure.

Carbon fibre was selected over aluminium extrusion or steel rod to minimise the structural mass contributed by the frame itself. Since the drive motors were fixed by the base platform and could not be selected for a target payload, reducing every other source of mass was the primary means of expanding the usable payload budget. The frame contributes negligible weight relative to its stiffness, allowing the motors' available torque to be directed toward payload rather than structure. The total robot mass, including all electronics and structural additions, is 6 kg.

The delivery tray at the top is fabricated from 3 mm acrylic sheet, cut by laser to a tray profile. The individual acrylic pieces are assembled using a toothed interlocking (kerf-and-tab) joinery method, where matching teeth along the sheet edges producing a rigid tray form along adhesive to hold it in place. The laser-cut tab-and-slot approach also ensures dimensional repeatability across the joint lines. The resulting tray provides a flat, smooth surface suitable for carrying files, documents, and small laboratory items up to 1 kg.

Figure 3.6 The robot positioned alongside a person.

Figure 3.5 The assembled robot frame

3.2.3 Electronics Mounting Shelf

The electronics are distributed across two horizontal shelves, both fabricated from 1 mm carbon fibre sheet and clamped to the vertical corner columns via 3D-printed brackets. Both shelves are positioned close to the chassis, keeping the centre of mass low to maintain stability under load and during motion.

The lower shelf is the primary electronics deck. Its top face carries the Cytron Smart Duo 30 motor driver and the NVIDIA Jetson Nano, mounted side by side on standoffs. The underside of the same shelf is used as a mounting face for the custom PCB, which handles signal routing and peripheral interfacing. Using both faces of the lower shelf maximizes the usable mounting area without requiring additional height in the frame.

The upper shelf, positioned immediately above the lower one, carries the power electronics, the DC-DC inverter and associated power conversion components. Both shelves share the same 1 mm carbon fiber sheet construction and 3D-printed column clamps, keeping the combined structural addition to a minimum contribution to the overall robot mass.

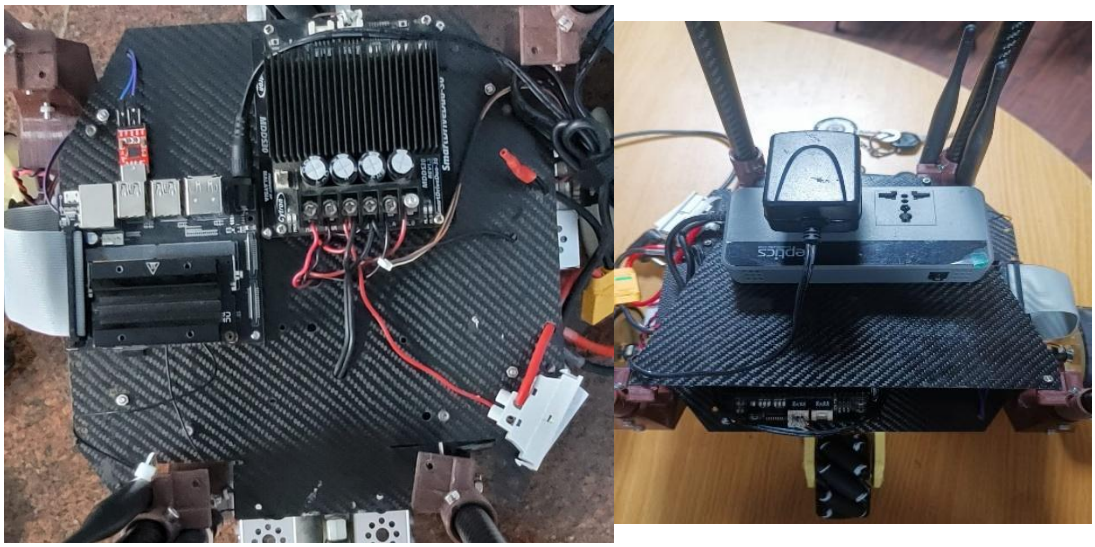

Figure 3.7 Photograph of the lower shelf showing the motor driver and Jetson on the top face; and the upper shelf showing the inverter and power electronics

3.2.4 Drive Wheels and Support Wheels

The DANI chassis drives through two rear traction wheels, one on each side, forming the differential pair. The original wheel surface was found to produce insufficient grip on the tiled corridor floor, causing slip during acceleration and turning. A layer of neoprene sheet was bonded to the outer circumference of each drive wheel to increase the contact friction coefficient. This modification eliminated slip without altering the wheel diameter or the kinematic model of the drive system.

Longitudinal stability is provided by two passive support wheels. At the front, a single omni wheel is mounted at the centreline, always in contact with the floor, functioning as a passive castor that rolls freely in any direction. This gives the robot a tricycle-type support geometry: one front contact point and two rear drive contacts. At the rear, a mecanum wheel is mounted as a passive support wheel. The rear support wheel is positioned 1 mm above the floor surface under normal level conditions, it does not bear load on flat ground. This deliberate clearance prevents the support wheel from lifting the rear drive wheels off the surface on smooth floors; the wheel only makes contact when the surface dips slightly, which was observed to cause rearward toppling tendency during testing. The mecanum wheel at the rear was added as a corrective measure after this instability was identified, and its angled rollers allow it to yield laterally without resisting the differential steering geometry of the drive wheels.

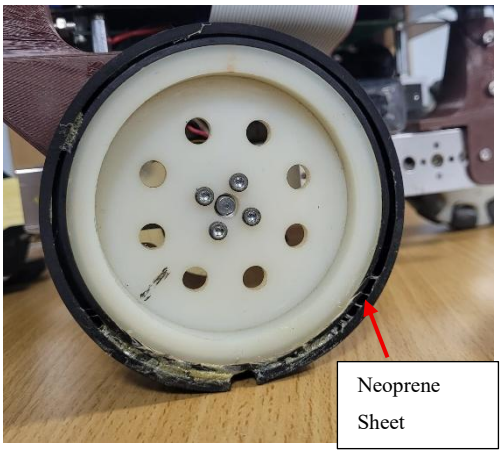

Figure 3.8 Close-up photograph of the drive wheel with the neoprene sheet wrapped

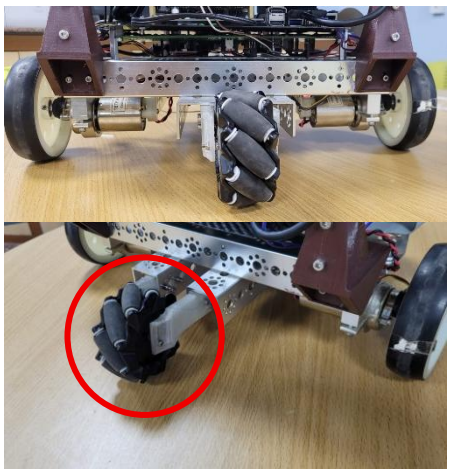

Figure 3.9 Rear mecanum support wheel

3.2.5 Camera Mount

The ZED stereo camera is mounted at the front-center of the tray platform, positioned inside the tray edge to protect it from impact in the event of a collision. The mount is a 3D-printed bracket that allows the camera angle to be set anywhere between −30° and +30° relative to the horizontal, with a locking mechanism to fix the chosen angle. For the deployed configuration, the camera is set at a 15° downward tilt from horizontal. This tilt angles the depth field of view toward the floor plane ahead of the robot, ensuring that low obstacles and floor-level features fall within the usable depth range while retaining sufficient forward coverage for corridor-distance obstacle detection. The mount bracket was printed with the part oriented at 45° to the build plate on a *Z*-axis 3D printer to maximize layer-line strength along the thin sections of the bracket arm, where bending loads are highest.

Figure 3.10 Photograph of the camera mount at the front-centre of the tray

3.2.6 3D-Printed Components and Material Selection

All 3D-printed parts on the robot, column clamps, electronics shelf brackets, rear support wheel bracket, and the camera mount, are printed in PETG (Polyethylene Terephthalate Glycol). The choice of PETG over PLA or PLA+ was made after an initial iteration of mounts printed in PLA+. PLA+ parts showed surface degradation and dimensional creep under the sustained mild loads and ambient temperatures of the deployment environment. PETG offers higher impact resistance, better long-term dimensional stability, and a smoother surface finish, making it the preferred material for structural brackets intended for extended operational life.

All parts are printed at 80% infill density with a gyroid infill pattern. The gyroid pattern was selected over rectilinear or grid patterns because it distributes stress isotropically in all three axes, avoiding the directional weak planes that rectilinear infill introduces along the print axis. At 80% infill, the combination of gyroid geometry and PETG material produces parts with sufficient compressive and shear strength for the mounting loads involved while keeping part mass low.

The camera mount bracket and the rear support wheel mount are printed with the part oriented at 45° to the build plate. Both are thin-walled structures where the bending load is applied transverse to what would otherwise be a weak layer boundary if printed upright. Rotating the print orientation to 45° ensures that no single layer boundary runs perpendicular to the primary load direction, distributing the load across multiple layer interfaces and significantly increasing resistance to delamination failure.

Figure 3.11 Render of the 3D-printed components

3.3 SENSORS

The robot's sensing suite is composed of four components, each selected to serve a specific role within the navigation and safety architecture. The ZED Mini stereo camera is the primary sensor, providing depth, visual odometry, and object detection from a single unit. The ZED's integrated IMU is fused internally within the ZED SDK's visual-inertial pipeline. A TF Mini Plus single-point LiDAR provides dedicated rear proximity sensing for safe reverse manoeuvres. Quadrature wheel encoders built into the drive motors supply wheel-speed feedback for odometry. The following subsections describe each sensor, its interface to the Jetson Nano, and its specific role in the system.

3.3.1 ZED Mini Stereo Camera

The ZED Mini is a passive stereo camera that computes depth by comparing the disparity between two horizontally separated image sensors spaced 63 *mm* apart. It produces a calibrated depth map at up to 720*p* resolution and outputs RGB imagery concurrently on the same USB 3.0 connection to the Jetson Nano. The operating range relevant to corridor navigation is 0.1 *m* to approximately 15 *m*, with depth accuracy degrading at longer ranges; at the corridor distances of interest (0.3 *m* to 5 *m*), the depth data is sufficiently accurate for both obstacle detection and local cost-map population.

Beyond depth and RGB, the ZED Mini's onboard SDK runs a visual-inertial odometry (VIO) pipeline that fuses the stereo visual feed with the camera's built-in six-axis IMU. This pipeline outputs a continuous 6-DoF pose estimate, position and orientation, which the Jetson receives directly over USB without any additional sensor fusion hardware. The IMU is used exclusively within this SDK pipeline; it is not read independently by the system.

The ZED SDK also runs a multi-class object detection model on the RGB stream in real time. This is the mechanism by which the robot distinguishes between human and non-human obstacles, driving the behavior difference described in section 3.1, stall for a person, replan for an inanimate object. Both the depth output and the object detection output are consumed by the Jetson over the same USB 3.0 link.

The camera is mounted at the front-center of the tray platform at 800 *mm* height, tilted 15° downward as described in section 3.2.5. This height and tilt places the stereo

Figure 3.12 Photograph of the ZED Mini camera mounted on the robot (right), and the ZED Mini product photograph showing the dual-lens arrangement (left)

depth field of view to cover the corridor space from approximately 300 *mm* ahead of the robot out to 10 *m*, which is the operationally relevant range for obstacle detection at walking-speed approach rates.

3.3.2 TF Mini Plus Single-Point LiDAR

The TF Mini Plus is a time-of-flight single-point LiDAR with a measurement range of 0.1 *m* to 12 *m* and a beam divergence of approximately 3.6°. It outputs distance measurements at up to 400 *Hz* over a UART serial interface, which connects directly to a USB to TTL module connected to the Jetson Nano.

The sensor is mounted rear-facing on the robot chassis. Its sole function is proximity detection during reverse maneuvers: when the robot is commanded to reverse, the TF Mini Plus distance output is monitored, and the reverse motion is halted if an obstacle is detected within a defined safety threshold. The ZED Mini, being forward-facing, cannot observe the space directly behind the robot; the TF Mini Plus fills this blind spot. Its single-point beam is sufficient for this purpose since reverse motion is always along the robot's longitudinal axis and the sensor is aligned accordingly.

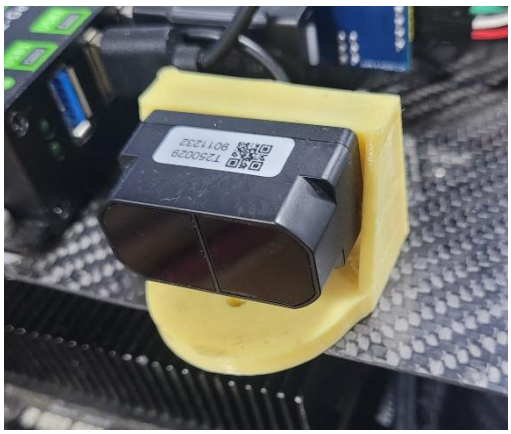

Figure 3.13 Photograph of the TF Mini Plus mounted on the rear of the chassis

3.3.3 Quadrature Wheel Encoders

Each drive motor is fitted with a quadrature incremental encoder built into the motor assembly, mounted directly on the motor shaft. Quadrature encoding uses two sensing channels offset by 90° in phase; by comparing the phase relationship of the two channel signals, both the speed and direction of rotation of each wheel can be

determined unambiguously. The encoder signals are connected directly to GPIO pins on the Jetson Nano and read via interrupt-driven counting in software.

The encoder data serves exclusively as feedback for the PID speed controller of each drive motor. The controller reads the encoder-derived wheel velocity and adjusts the PWM command to the Cytron motor driver to maintain the target speed set by the navigation stack. The encoders are not used for position odometry; localisation is handled entirely by the ZED Mini's visual-inertial odometry pipeline.

Figure 3.14 Close-up photograph of the encoder on the motor shaft

3.3.4 ZED Mini Integrated IMU

The ZED Mini includes a factory-calibrated six-axis IMU (three-axis accelerometer and three-axis gyroscope) rigidly fixed to the camera body. The IMU data is consumed exclusively by the ZED SDK's internal VIO fusion pipeline running on the Jetson; it is not accessed as a standalone sensor stream by the robot's software. Within the VIO pipeline, the IMU provides high-frequency inertial measurements that bridge the gaps between lower-frequency visual keyframe updates, improving pose estimate continuity during fast motion and reducing drift during segments where visual tracking is momentarily unreliable. Because the IMU is factory-calibrated and its fusion is handled entirely within the SDK, no external calibration or IMU driver is required on the Jetson.

3.4 ACTUATORS

The robot's sole actuator system is a differential drive arrangement of two brushed DC motors, one per drive wheel. Speed and direction of each motor are independently controlled by the Cytron SmartDriveDuo-30 (MDDS30) dual-channel motor driver. The following subsections describe the motors and the driver.

3.4.1 Drive Motors

The drive motors are the 739530 brushed DC motors supplied with the DANI robot platform, rated at 12 V. The key operating parameters from the manufacturer's datasheet are as follows: no-load speed of 150 ± 10% rpm, load speed of 137.5 ± 10% rpm at a load torque of 3.9 kg·cm, load current of 0.91 A (1.37 A maximum), and no-load current of 0.34 A (0.68 A maximum). The motors include integrated quadrature encoders on the motor shaft, used for PID speed feedback as described in section 3.3.3.

The rated speed of approximately 150 rpm at 12 V, combined with the wheel diameter of the DANI platform, yields a maximum forward speed appropriate for shared corridor operation, fast enough to complete deliveries in reasonable time, slow enough that the robot can decelerate to a stop within the detection range of the ZED Mini's depth field of view. The load torque specification of 3.9 kg·cm per motor, with two motors driving the platform, provides sufficient drive force for the 6 kg robot mass on level tiled floors, with margin for the neoprene traction layer and payload up to 1 kg.

Figure 3.15 Photograph and datasheet drawing of the 739530 motor and encoder

3.4.2 Motor Driver — Cytron SmartDriveDuo-30 (MDDS30)

The Cytron SmartDriveDuo-30 (model MDDS30) is a dual-channel brushed DC motor driver designed specifically for differential drive mobile robots. It drives both motors from a single board, accepting motor supply voltages from 7 V to 35 V and delivering up to 30 A continuous current per channel (80 A peak for up to one second). The onboard MOSFETs switch at 18 kHz, above the audible frequency range, eliminating motor whine during operation. The board incorporates thermal protection, current-limit protection, and battery over- and under-voltage indicators.

In this system, the MDDS30 operates in Serial Simplified mode. In this mode the driver accepts commands over its UART interface on the IN1 pin; IN2, AN1, and AN2 are unused. The Jetson Nano communicates with the driver through a USB-to-TTL serial adapter, presenting a virtual serial port to Jetson's software. This eliminates the need for a logic-level shifter on this interface and allows the motor driver to be addressed through the standard Linux serial device interface.

Commands follow the Serial Simplified protocol: each motor is addressed by a single byte, where bit 7 selects the channel (0 = left motor, 1 = right motor), bit 6 sets direction (0 = clockwise, 1 = counter-clockwise), and bits 0–5 encode speed as an integer from 0 (stop) to 63 (full speed). The PID controller running on the Jetson reads the encoder feedback (section 3.3.3) and writes updated speed commands to the serial port each control cycle to drive the actual wheel speed to the value commanded by the navigation stack.The MDDS30's 30 A continuous rating per channel is substantially above the motors' 1.37 A maximum current draw, providing a large margin against

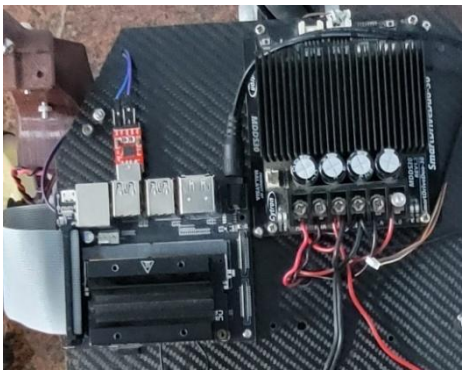

Figure 3.16 Photograph of the MDDS30 mounted on the electronics shelf

transient overloads during acceleration or obstacle encounters and ensuring the driver operates well within its thermal envelope in normal use.

3.5 PROCESSING UNIT

All onboard computation is handled by the NVIDIA Jetson Nano system-on-module, mounted on the Tanna TechBiz Eagle 101 carrier board. The Jetson Nano is the compute foundation for the entire robot: it runs the navigation stack, processes ZED camera data, executes object detection inference, manages all sensor interfaces, drives the motor controller, and maintains the Wi-Fi link to the task management backend, all concurrently within a 4 GB RAM, 20W power envelope.

3.5.1 NVIDIA Jetson Nano Module

The Jetson Nano module integrates a 128-core NVIDIA Maxwell GPU and a quad-core ARM Cortex-A57 CPU on a single 70 × 45 mm system-on-module. It provides 4 GB of LPDDR4 RAM shared between CPU and GPU, and 16 GB of eMMC flash storage. The Maxwell GPU supports CUDA, cuDNN, and TensorRT, enabling hardware-accelerated deep learning inference directly on the module without an external accelerator. The module is rated at 10W in low-power mode and up to 20W in full-performance mode; the system operates in 20W mode to sustain the concurrent workloads of the full software pipeline.

The decision to use the Jetson Nano rather than a general-purpose single-board computer (e.g. a Raspberry Pi) was driven by the GPU inference requirement. The ZED SDK's onboard object detection model and the visual-inertial odometry pipeline both require GPU acceleration to run at the frame rates needed for real-time navigation. A CPU-only platform cannot sustain these workloads within the latency budget of the navigation loop.

3.5.2 Tanna TechBiz Eagle 101 Carrier Board

The Jetson Nano module is mounted on the Tanna TechBiz Eagle 101 carrier board, an India-manufactured carrier designed specifically for the Jetson Nano B01 module form factor. The Eagle 101 exposes the module's connectivity through its I/O complement: 4× USB 3.0 ports, 1× Gigabit Ethernet, a Micro SD card slot, GPIO

header, and a camera connector. The Eagle 101 includes an inbuilt Wi-Fi module, so no USB port is consumed for wireless connectivity. In this system, three of the USB 3.0 ports are occupied: the ZED Mini stereo camera, the USB-to-TTL serial adapter for the MDDS30 motor driver, and a powered USB hub. The TF Mini Plus LiDAR's USB-to-UART adapter connects through the powered hub rather than directly to the carrier board, keeping the remaining Jetson USB ports free. The powered hub's independent power supply ensures that the LiDAR adapter does not draw current from the Jetson's USB rail. The Gigabit Ethernet port is available as a wired fallback connection.

The Eagle 101's compact footprint and standard Jetson Nano connector layout mean no modification to the module or its software environment is required, the carrier board is electrically transparent to the Jetson Nano's Linux and JetPack software stack.

Figure 3.17 Photograph of the Eagle 101 carrier board with the Jetson Nano Mounted

3.6 Electrical Interfaces and Communication

This section documents the complete set of electrical interfaces between the Jetson Nano and all peripheral systems on the robot. Each interface is described by its physical connection type, signal direction, and any conditioning circuitry in the signal path. Two protection components are common to all GPIO-based interfaces and are described first.

3.6.1 GPIO Signal Conditioning: Optoisolator and Logic-Level Shifter

All signals exchanged between the Jetson Nano and external hardware through its GPIO pins pass through two conditioning stages in series: an optoisolator and a bidirectional logic-level shifter.

The optoisolator electrically isolates the Jetson's GPIO circuitry from the external device. Isolation is necessary because the motor drive system operates at battery voltage with high transient currents; voltage spikes induced on shared ground or power lines during motor switching events could otherwise propagate into the Jetson's GPIO input stages and cause latch-up or permanent damage. By coupling signals optically across a galvanic barrier, the optoisolator prevents any conducted electrical transient from reaching the Jetson, regardless of the magnitude of the disturbance on the external side.

The logic-level shifter translates signal voltages between the 5 V logic level of the external peripheral hardware and the 3.3 V GPIO logic level of the Jetson Nano. Driving a 5 V signal directly into a 3.3 V GPIO input exceeds the Jetson's absolute maximum input voltage rating and will damage the device over time; the shifter ensures that no GPIO pin ever sees more than 3.3 V regardless of the external logic level. The shifter is bidirectional, supporting both inbound signals (encoder pulses arriving at the Jetson) and any outbound GPIO signals without requiring separate components for each direction.

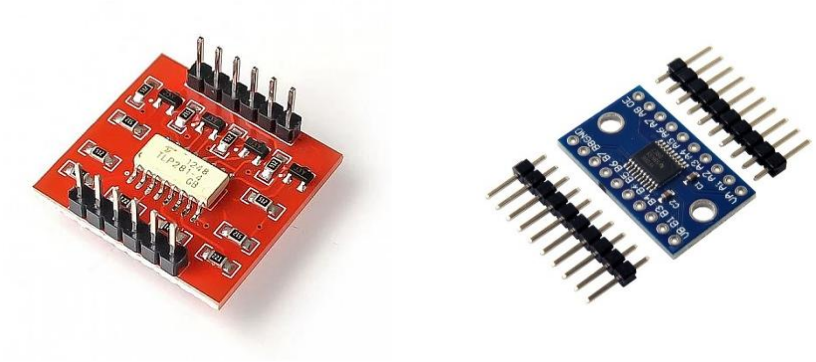

Figure 3.18. PC817 optoisolator (left) and TXS0108E bidirectional logic-level shifter (right).

3.6.2 ZED Mini Stereo Camera (USB 3.0)

The ZED Mini connects to one of the Eagle 101's USB 3.0 ports via a single USB 3.0 cable. All data, stereo video frames, depth maps, VIO pose estimates, and SDK control traffic transits over this single connection. The USB 3.0 interface provides sufficient bandwidth (up to 5 Gbit/s) to sustain the ZED's full-resolution stereo stream without throttling. The USB bus also powers the camera; no separate power supply connection is required. The USB interface is inherently galvanically isolated from the Jetson's internal circuitry by the USB transceiver, so no additional signal conditioning is applied.

3.6.3 MDDS30 Motor Driver (USB-to-TTL Serial)

The Jetson communicates with the Cytron MDDS30 motor driver through a USB-to-TTL serial adapter. The adapter presents a virtual COM port to the Jetson's Linux operating system, through which the motor control software writes Simple Serial protocol bytes. The MDDS30's IN1 UART pin receives the serial signal at TTL level. Because the electrical connection between the Jetson and the MDDS30 is made through the USB transceiver inside the adapter rather than through a direct GPIO pin, the USB interface provides inherent isolation and no optoisolator or level-shifter is required on this path.

3.6.4 TF Mini Plus LiDAR (USB-to-UART via Powered Hub)

The TF Mini Plus outputs distance data over a UART serial interface at 5 V logic. A USB-to-UART adapter converts this to a USB connection, which plugs into the downstream port of the powered USB hub. The hub's upstream USB 3.0 port connects to the Jetson, presenting the LiDAR as a second virtual COM port. The powered hub supplies 5 V to the adapter and sensor from its own regulated supply, drawing directly from the battery (12 V input through the hub's internal regulation) independently of the custom PCB power rails. This arrangement serves two purposes: it prevents the LiDAR from drawing current from the Jetson's USB rail, and it isolates the serial device's supply from the PCB's regulated 5 V rail. Because the USB-to-UART adapters share their supply with the serial line's reference ground, drawing them from a separate battery-referenced supply prevents switching noise on the PCB rail from coupling into the serial signal lines and causing communication errors. As with the

motor driver path, the USB transceiver in the adapter provides inherent electrical isolation from the Jetson; no separate optoisolator is required.

3.6.5 Wheel Encoders (GPIO, Interrupt-Driven)

The quadrature encoder outputs of both drive motors connect directly to GPIO pins on the Jetson Nano via the Eagle 101's 40-pin GPIO header. Each motor provides two encoder channel signals (A and B), for a total of four GPIO lines. Because the encoder electronics operate at 5 V and the Jetson's GPIO is rated for 3.3 V input, each encoder signal passes through the optoisolator and logic-level shifter conditioning chain described in section 3.6.1 before reaching the Jetson pin. The Jetson's software reads encoder pulses via hardware interrupt handlers, capturing each rising and falling edge of both channels to compute direction and speed.

3.6.6 Wi-Fi Communication (Inbuilt Module)

The Eagle 101 carrier board includes an inbuilt Wi-Fi module, which provides the robot's wireless network interface without consuming a USB port. The Jetson communicates with the web-based task management backend exclusively over this Wi-Fi link, sending and receiving HTTP requests to a cloud-hosted server. This is the only interface that carries traffic beyond the robot's local hardware boundary; all other interfaces described above are internal to the robot.

3.6.7 Interface Summary

Table 3.1 summarizes all interfaces between the Jetson Nano and the robot's peripheralvsystems, including the physical connection type, signal direction, and whether the GPIO conditioning chain applies.

Table 3.1 Summary table representing all interfaces with the Jetson Nano

Peripheral	Interface Type	Direction	Signal Conditioning
ZED Mini Stereo Camera	USB 3.0	Bidirectional	None
MDDS30 Motor Driver	USB-to-UART	Output	None

TF Mini Plus Single Point LiDAR	USB-to-UART	Input	None
Powered USB Hub	USB 3.0	Bidirectional	None
Wheel Encoders (×2)	GPIO	Input	Optoisolator + logic-level shifter

3.7 POWER SUPPLY CIRCUITRY

The robot's power system distributes energy from a single 3S LiPo battery (11.1 *V* nominal, 10,000 *mAh*, 25*C*) to five load groups: the drive motors, the Jetson Nano compute unit, the custom PCB peripherals, the powered USB hub, and the onboard inverter. Each load group is supplied through an independently regulated path derived from the battery, ensuring that high-current transients on one branch do not disturb the supply rails of another.

3.7.1 Battery

The 3S LiPo pack provides a nominal voltage of 11.1 *V* (12.6 *V* fully charged, 9.9 *V* at the 3.3 *V* per-cell minimum discharge threshold). At 10,000 *mAh* capacity and a 25*C* discharge rating, the pack can deliver up to 250 *A* instantaneously, far exceeding the peak demand of any load in the system. In practice, peak current draw is dominated

Figure 3.19 Photograph of the LiPo battery pack

by motor acceleration events and the inverter's startup transient, both of which are brief and well within the battery's sustained discharge capability. The battery is the single power source for the entire robot; no supplementary supply or onboard charging circuitry is fitted.

3.7.2 Drive Motor Supply (Direct Battery)

The Cytron MDDS30 motor driver receives its motor supply voltage directly from the battery terminals, with no intermediate regulation. The MDDS30 is rated for 7–35 V motor supply input and incorporates its own internal regulation for its logic and gate-drive circuitry. Direct connection to the battery rail is therefore both permissible and intentional: it delivers maximum available voltage to the motors for peak torque and avoids introducing a series regulator whose voltage drop would reduce the motor supply and limit top speed. The MDDS30's onboard over-voltage, under-voltage, and current-limit protection circuits provide the necessary safeguards on this branch.

3.7.3 Jetson Nano Supply (5 V via Inverter)

The Jetson Nano requires a 5 V, 4 A regulated supply on its barrel jack input. This is provided by a 5 V 4 A barrel jack power adapter, plugged into a 200 W DC-AC inverter that derives its input from the battery rail. The inverter converts the battery's DC voltage to 230 V AC, and the power adapter rectifies and regulates this back to 5 V DC for the Jetson.

Two alternatives were considered for supplying the Jetson: feeding it directly from the battery through a DC-DC step-down converter or routing it through an AC adapter via an inverter. Direct battery supply was ruled out because a LiPo battery's terminal voltage is not stable it varies from 12.6 V fully charged to approximately 9.9 V at minimum discharge, and the Jetson Nano requires a regulated 5 V input; an unregulated or poorly regulated supply risks brownout or damage to the module. Designing a dedicated DC-DC converter with the correct output voltage, current rating, and connector to power the Jetson reliably would constitute a separate sub-system design effort that does not contribute to the project's core objectives. The inverter-plus-adapter route uses off-the-shelf components that together provide a well-regulated, thermally protected 5 V supply without requiring custom power electronics design. The

efficiency penalty of the DC-AC-DC conversion path is accepted given the 10,000 mAh battery capacity.

3.7.4 Custom PCB Supply (Buck Converters: 5 *V* and 3.3 *V*)

The custom PCB is powered by two independent buck converters, both drawing from the battery rail. The first steps the battery voltage down to 5 *V*, supplying the peripheral electronics on the PCB, sensors and other 5 *V* logic devices. The second steps down to 3.3 *V*, supplying the logic-level shifters described in section 3.6.1. Using a dedicated 3.3 *V* buck converter rather than deriving 3.3 *V* from the 5 *V* rail through a linear regulator avoids the thermal dissipation of a linear drop and provides a cleaner regulated rail for the signal-level circuitry.

The two buck converters are independent branches from the battery rather than a cascade (battery → 5 *V* → 3.3 *V*), ensuring that load transients on the 5 *V* peripheral rail do not propagate into the 3.3 *V* logic rail and vice versa.

3.7.5 Powered USB Hub Supply (Direct Battery, 12 *V*)

The powered USB hub draws its supply directly from the battery at battery voltage (nominally 11.1–12.6 *V*), through the hub's own internal voltage regulation down to its operating voltage. As described in section 3.6.3, supplying the hub independently from the battery rather than from the PCB's regulated 5 *V* rail prevents

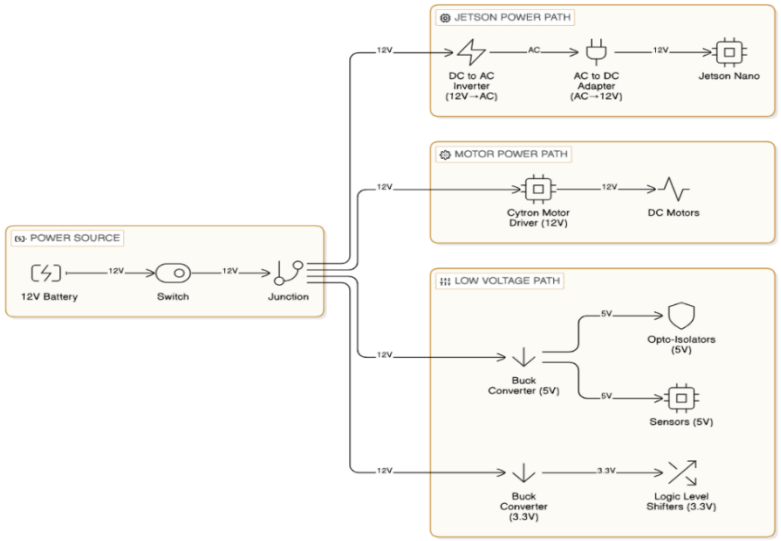

Figure 3.20 Block diagram of the power distribution architecture

switching noise on the PCB supply from coupling into the serial signal lines of the USB-to-UART adapters connected to the hub's downstream ports.

3.8 PCB Design and Fabrication

The custom PCB serves as the signal-conditioning and connectivity hub between the Jetson Nano and all GPIO-connected peripherals. Its functions are: voltage regulation (5 V and 3.3 V rails for peripheral devices), signal isolation (optoisolators on all GPIO lines), logic-level translation (5 V to 3.3 V level-shifting for Jetson GPIO compatibility), and breakout connectivity (headers and JST connectors for all current and future peripheral connections).

3.8.1 Design

The PCB schematic was designed in Altium Designer. The design is single-sided, i.e. all active components and traces are on one layer, with a small number of deliberate wire jumpers to resolve the trace overlaps that arise from a single-layer routing constraint. Rather than proceeding to a double-layer PCB fabrication run, which would have required sending Gerber files to an external manufacturer and adding lead time, the single-sided approach was chosen so that the board could be hand-assembled on perf-board immediately. Wire jumpers were used only where trace overlaps could not be resolved by re-routing, and their number was minimized during the Altium layout review before committing to the perf board. The design includes provisions

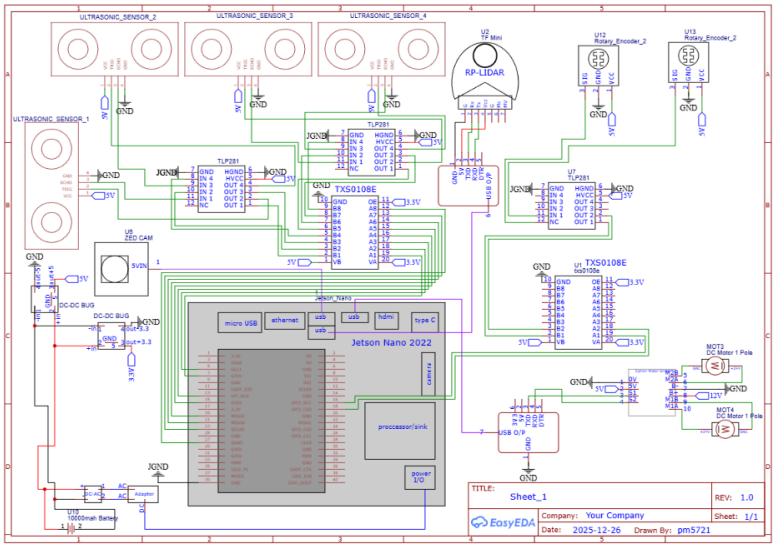

Figure 3.21. Altium Designer schematic of the custom signal-conditioning PCB

beyond the current sensor set to support future expansion. Dedicated pin headers are present for ultrasonic distance sensors (for cliff detection), a servo output, and an additional UART port. These headers are populated on the PCB but unused in the current system.

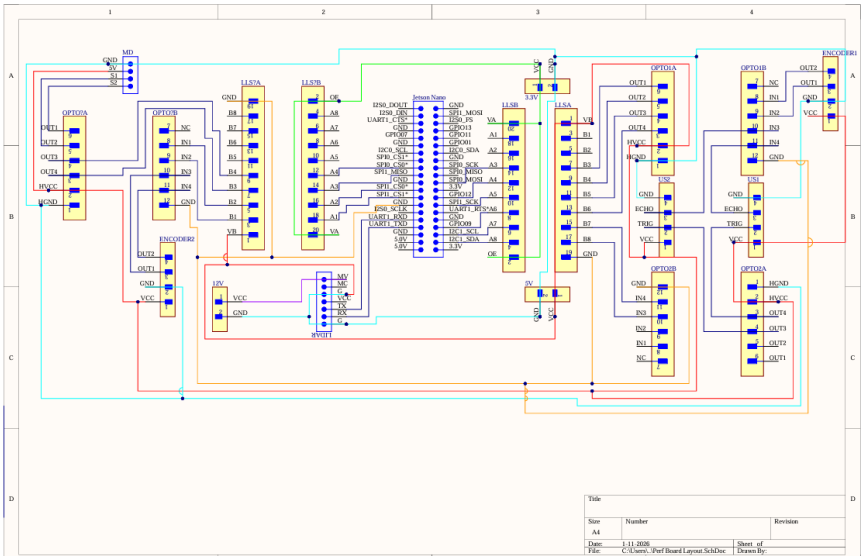

Figure 3.22. Single-layer PCB layout

3.8.2 Fabrication

The PCB was hand-soldered onto a single-sided perf-board. Through-hole components are used throughout: the buck converters, optoisolators, and logic-level shifter modules are all mounted to female header pins soldered directly to the perfboard, with connections between them made using solder lead routed along the perf-board grid. Wire jumpers bridge the trace overlaps identified during the layout phase.

All components that interface with external modules, optoisolators, level shifters, and the buck converters' output connections are connected through soldered pin headers rather than being hardwired. This allows any individual component to be removed and replaced from its header without disturbing the rest of the board, which proved useful during debugging and component substitution.

3.8.3 Connector Scheme

The Jetson Nano GPIO header is connected to the PCB via a 40-pin dual-row flat ribbon cable. The ribbon cable connects the Jetson's 40-pin GPIO connector (on the Eagle 101 carrier board) to a matching 40-pin dual-row header soldered to the PCB. All

GPIO signals from the PCB, encoder inputs, and any future GPIO signals are routed through the ribbon cable to Jetson. The ribbon cable was chosen over individual jumper wires to keep the GPIO bundle compact and to reduce the risk of misconnection during assembly and reassembly.

External peripherals connect to the PCB through JST connectors for signals and power. The encoder connections from each motor use JST connectors on the PCB header side, which mate with connectors crimped to the encoder cable leads. This plug-and-socket scheme allows the electronics shelf to be removed from the robot frame for servicing without cutting any wires.

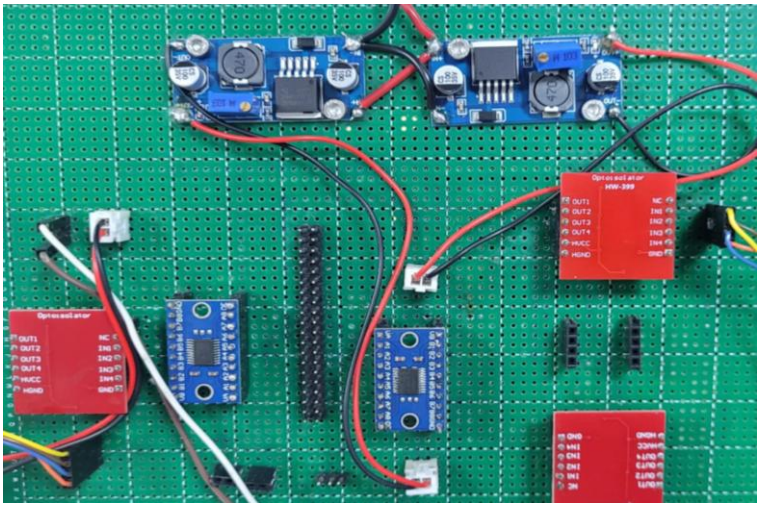

Figure 3.23 Photograph of the completed perf-board PCB

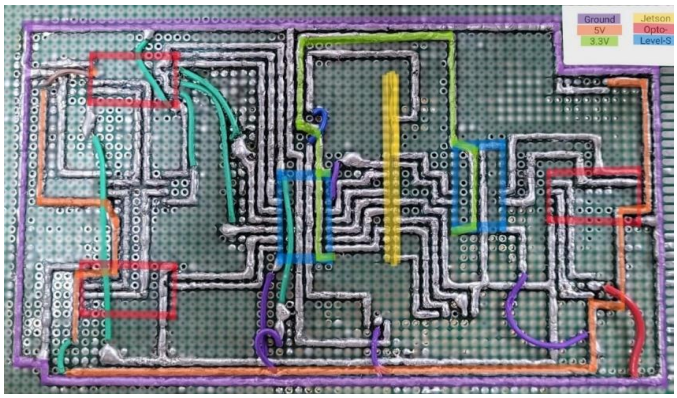

Figure 3.24 Photograph showing the power lines

3.9 ROBOT SPECIFICATIONS

Table 3.2 consolidates the key specifications of the robot across all hardware subsystems. Detailed descriptions and design rationale for each parameter are provided in the referenced subsections.

Table 3.2 Robot Mechanical Specifications

Parameter	Specification	Reference
Overall height	800 mm	Section 3.2.2
Total robot mass (with all electronics)	6 kg	Section 3.2.2
Payload capacity	1 kg	Section 3.2.2
Drive configuration	Differential drive — 2 rear traction wheels	Section 3.2.4
Front support	Passive omni wheel castor (always in contact)	Section 3.2.4
Rear support	Passive mecanum wheel, 1 mm above floor	Section 3.2.4
Traction enhancement	Neoprene sheet bonded to drive wheel rim	Section 3.2.4
Elevated frame	4× carbon fibre tubes, 1 mm wall, corner-mounted	Section 3.2.2
Tray material	Laser-cut acrylic, tab-and-slot assembly	Section 3.2.2
Electronics shelf	1 mm carbon fibre sheet, dual-sided mounting	Section 3.2.3

Structural mounts	PETG 3D-printed, 80% gyroid infill	Section 3.2.6

Table 3.3 Robot Actuation Specifications

Parameter	Specification	Reference
Drive motors	Tetrix 739530, 12 V DC brushed, 150 rpm no-load	Section 3.4.1
Motor load torque	3.9 kg·cm per motor at rated load	Section 3.4.1
Motor driver	Cytron MDDS30, 30 A continuous / 80 A peak per channel	Section 3.4.2
Motor driver interface	Simple Serial mode via USB-to-TTL adapter	Section 3.4.2

Table 3.4 Robot Sensing Specifications

Parameter	Specification	Reference
Primary perception	ZED Mini stereo camera, USB 3.0	Section 3.3.1
ZED depth range	0.1 m – 15 m	Section 3.3.1
Rear proximity	TF Mini Plus LiDAR, 0.1 m – 12 m, UART	Section 3.3.2
Wheel encoders	Quadrature, built into drive motors, GPIO interrupt	Section 3.3.3

Encoder function	PID motor speed feedback only (not odometry)	Section 3.3.3
IMU	ZED Mini integrated 6-axis IMU (VIO pipeline only)	Section 3.3.4

Table 3.5 Robot Computation Specifications

Parameter	Specification	Reference
Processor module	NVIDIA Jetson Nano, 128-core Maxwell GPU, quad-core ARM A57	Section 3.5.1
RAM	4 GB LPDDR4 (shared CPU/GPU)	Section 3.5.1
Carrier board	Tanna TechBiz Eagle 101 (Jetson Nano B01)	Section 3.5.2
Wireless	Inbuilt Wi-Fi module on Eagle 101	Section 3.5.2
USB peripherals	ZED Mini (USB 3.0), MDDS30 TTL adapter, powered hub	Section 3.5.2

Table 3.6 Robot Power System Specifications

Parameter	Specification	Reference
Battery	3S LiPo, 11.1 V nominal, 10,000 mAh, 25C	Section 3.7.1

Motor driver supply	Direct battery (11.1–12.6 V)	Section 3.7.2
Jetson supply	200 W inverter + 5 V 4 A AC adapter	Section 3.7.3
PCB 5 V rail	Buck converter from battery	Section 3.7.4
PCB 3.3 V rail	Buck converter from battery	Section 3.7.4
Powered USB hub supply	Direct battery, 12 V input	Section 3.7.5

Table 3.7 Robot Electrical Interface Specifications

Parameter	**Specification**	**Reference**
GPIO conditioning	Optoisolator + bidirectional logic-level shifter (all GPIO)	Section 3.6.1
Encoder interface	GPIO interrupt via optoisolator + 5V→3.3V level shifter	Section 3.6.3
PCB connectivity	40-pin ribbon cable to Jetson GPIO; JST connectors for peripherals	Section 3.8.3

CHAPTER 4

SYSTEM SOFTWARE

4.1 SOFTWARE ARCHITECTURE

The system is organized around a unidirectional command pipeline that begins at the operator interface and terminates at the motor actuators, with a closed feedback path running in the opposite direction through the localisation and health-monitoring subsystems. At the top of the pipeline, the User Interface accepts a delivery request specifying a pick-up room and a destination room and forwards it as a structured task descriptor to the Task Manager. The Task Manager maintains a queue of pending jobs, resolves each job into an ordered sequence of navigation waypoints derived from the floor layout, and issues goal poses to the Path Planner one at a time. The Path Planner computes a collision-free global path through the active occupancy grid tile from the robot's current pose to the goal pose, then hands the path to the Path Tracker. The Path Tracker executes the path in real time by continuously computing the velocity commands required to follow it, passing those commands to the Motion Controller. The Motion Controller translates the velocity commands into per-wheel speed targets using a differential-drive kinematic model and applies closed-loop PID control against quadrature encoder feedback to track those targets at the actuator level. Running in parallel throughout execution, the Localisation module integrates stereo-visual-inertial odometry to maintain a continuous estimate of the robot's pose on the active map tile; this estimate is fed back to both the Path Tracker and the Obstacle Handler. A block diagram of these functional modules and their signal interconnections is provided in Figure 4.1.

The Obstacle Handler monitors the robot's projected trajectory corridor for detected obstacles and either waits for clearance or triggers the Path Planner to recompute an alternative route, raising an alert via the User Interface and the on-board speaker in either case.The remainder of this chapter addresses each algorithmic subsystem in turn. Section 4.2 covers the map preparation pipeline and usage algorithm. Section 4.3 describes the perception algorithms for pose estimation and obstacle detection. Section 4.4 covers global path planning, and local path tracking. Section 4.5 describes the differential-drive motion control and PID design. The integration of these

subsystems into a running software stack, middleware selection, node topology, and deployment configuration is addressed in chapter 5.

4.2 MAP GENERATION AND USAGE

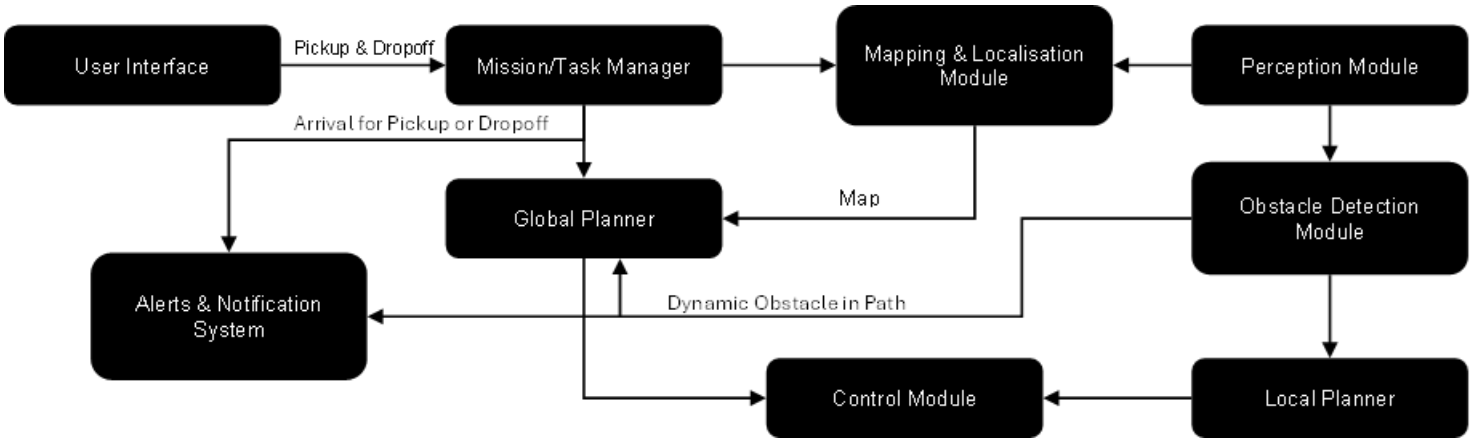

Figure 4.1 Software Architecture of the System

This section describes the end-to-end pipeline for generating the tessellated occupancy grid maps used by the navigation system. The pipeline proceeds through five stages: physical measurement, CAD drawing in SolidWorks, export and conversion to a raster image, resolution calibration and occupancy annotation, and tessellation into overlapping tiles with metadata. The pipeline is executed entirely offline; the resulting tile files are static assets loaded at runtime.

4.2.1 Motivation for a Tiled Map Approach

The second floor of the Hi-Tech Block spans a navigable corridor network of approximately 63.8 *m* × 36.8. A monolithic occupancy grid at the 0.04 *m/px* resolution required for corridor-width navigation produces a 1,596 × 919 *pixel* map of 1,466,724 cells, of which only 11.7% are navigable. The remaining 88.3% are walls and room interiors that *m* the robot will never traverse. Representing the entire floor as a single map allocates full costmap resources over all of it regardless, a structural inefficiency that cannot be resolved by tuning the planner configuration.

The literature survey in Chapter 2 identified this as a fundamental constraint rather than a configuration problem. Horelican (2022) demonstrated that large high-resolution maps create processing bottlenecks that no parameter adjustment can eliminate, because the planner's data structures scale with map size. Compression approaches such as OctoMap and ColMap reduce the bytes stored but do not change

the number of cells the planner reasons over at runtime; the costmap is still built across the entire environment. What was needed was a mechanism to exclude the regions the robot is not currently navigating from memory entirely, not a smaller representation of all of them.

A tessellated approach addresses this directly. The floor is partitioned offline into five overlapping tiles, each covering one navigable zone, so that at runtime only the active tile is loaded into the active costmap. This reduces the active costmap by 93.8% relative to the monolithic map, keeping planner data structures bounded regardless of total floor size, with no modification to the core path-planning algorithm required.

The tile switching mechanism is driven entirely by the robot's own pose and heading, with no dependence on external fiducial markers. Zone-based systems in the literature that relied on AprilTag or ArUco detection encountered consistent failure modes, AMCL divergence on reload, timing gaps between map server updates and costmap rebuilds, and indefinite stalls when marker detection failed. A pose-and-heading trigger has no external dependency to fail and is always available from the navigation stack. The following subsections describe the offline pipeline through which these tiles are built and the algorithm for switching the active tile.

4.2.2 Map Generation Pipeline

The map generation pipeline converts physical floor measurements into the set of tessellated occupancy grid tiles used by the navigation stack at runtime. The pipeline is executed entirely offline, before deployment, so that no map-building computation runs on the Jetson Nano during operation. It proceeds through five sequential stages: physical measurement of the corridor geometry, CAD reconstruction of the floor layout in SolidWorks, export to a raster PNG image, resolution calibration and occupancy annotation of that image, and finally tessellation into five overlapping tiles with associated metadata. Each stage produces a well-defined artefact that feeds directly into the next; the output of the final stage, the tile image files and their JSON metadata are the static assets loaded by the navigation stack at startup. The subsections below describe each stage in turn.

4.2.2.1 Physical Measurement

All corridor widths, room doorway positions, wall offsets, and junction dimensions were measured manually using a surveyor's measuring tape at floor level along the principal corridor axes of the Hi-Tech Block 2nd floor. This manual approach was adopted because no pre-existing architectural drawings with sufficient metric accuracy were available for the specific floor layout.

4.2.2.2 CAD Drawing and DXF Export

Using the recorded measurements, a 2D sketch was constructed in SolidWorks capturing the outline of all navigable corridor regions, wall boundaries, and doorway positions. Room interiors are excluded as they fall outside the robot's operational domain. The sketch was then converted to a SolidWorks Drawing sheet, which establishes the 2D sheet layout with proper scale and projection. Two exports were produced from this drawing: a DXF (Drawing Exchange Format) file for documentation and archival purposes, and a PNG raster image used as the direct input to the map generation pipeline.

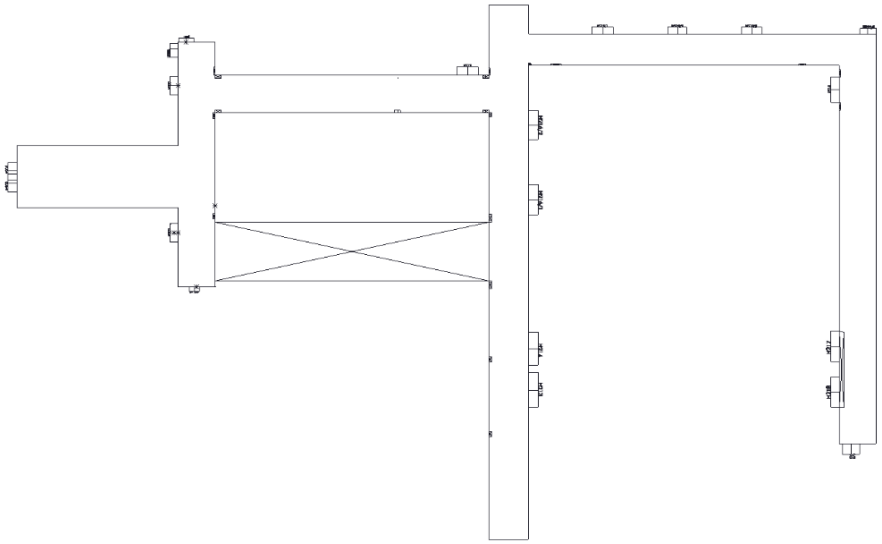

Figure 4.2 SolidWorks CAD sketch of the Hi-Tech Block 2nd floor(DXF export)

4.2.2.3 DXF to PNG Conversion

The PNG was exported directly from the SolidWorks Drawing via File → Export → Portable Network Graphics at 1:1 scale. Exporting at 1:1 scale from the drawing sheet ensures the pixel grid corresponds directly to the metric dimensions of the sketch, preserving the aspect ratio without any post-export rescaling. The exported

PNG was subsequently converted to grayscale using a Python script, producing a single-channel image suitable for occupancy annotation in the following step.

4.2.2.4 Resolution Calibration

The map resolution, defined as the physical distance represented by each pixel, was determined using the following relation:

$$\textit{Resolution (m/px) = Physical Distance (m) / Pixel Distance (px)} \quad \textit{.....(eq-4.1)}$$

This approach, known as pixel-to-metric calibration, establishes a direct correspondence between the pixel coordinate system of the raster image and the metric coordinate system of the real environment. A reference corridor segment of known physical length 2.3 m was identified in the environment, and its corresponding pixel span was measured as 55 pixels in the exported PNG, yielding:

$$\textit{Resolution} = 2.3\ \text{m} / 55\ \text{px} \approx 0.04\ \text{m/px} \quad \textit{.....(eq-4.2)}$$

This value was adopted uniformly across all five tiles rather than calibrating each tile independently. Uniform resolution ensures that the coordinate systems of adjacent tiles are metrically compatible, a prerequisite for accurate tile origin alignment and for seamless localisation continuity during tile transitions. The value of 0.04 m/px (40 mm per cell) is also consistent with standard occupancy grid resolutions reported in the literature for indoor corridor navigation.

4.2.2.5 Occupancy Annotation

The grayscale PNG was annotated as a binary occupancy map: navigable regions (corridors) painted white (pixel value 255, free space) and all non-navigable regions (walls, room interiors) painted black (pixel value 0, occupied). Annotation was performed manually using a raster image editor (Microsoft Paint), with corridor boundaries traced from the underlying CAD geometry. Since the CAD sketch serves as the geometric reference, the accuracy of annotation is bounded by the precision of the original measurements rather than the choice of editing tool.

A critical observation from this step is that only 11.7% of the 1,466,724 total cells in the monolithic whole-floor map are navigable. The remaining 88.3% represent walls and room interiors, occupied space that the robot will never traverse, yet a monolithic approach allocates full costmap resources for all of it. This spatial inefficiency directly motivates the tessellation strategy described in Section 4.2.2.6.

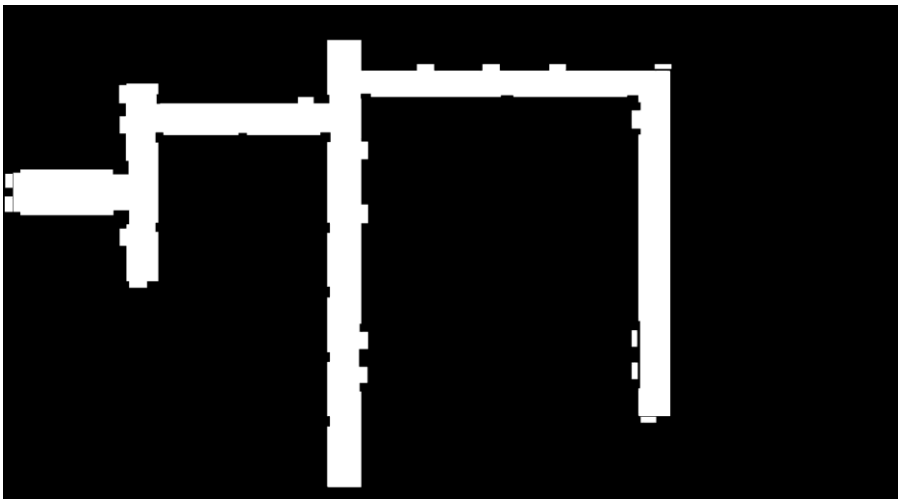

Figure 4.3 Full Floor 0ccupancy Map after Annotation

4.2.2.6 Map Tessellation

4.2.2.6.1 Turning Point Detection

The floor layout contains multiple direction changes, T-junctions and L-junctions where corridor axes change. These turning points serve as natural tile boundaries. They were detected by analysing the distribution of navigable pixels along horizontal and vertical scan lines across the floor plan image. At straight segments, scan lines perpendicular to the corridor axis return a consistent navigable width; at junctions, the navigable cross-section changes significantly, producing a local peak in the spatial frequency of direction changes. These peaks identified candidate split locations, verified against the physical layout.

4.2.2.6.2 Overlap Offset and Tile Boundaries

At each identified turning point, the cut line is duplicated at a fixed offset of 40 pixels (1.6 m at 0.04 m/px) ahead of the original, creating two boundary lines per junction. The region between these two lines forms the overlap zone shared by the two adjacent tiles, i.e. Tile A extends to the far boundary and Tile B begins at the near boundary, such that both tiles include the overlap region in their coverage. This symmetric duplication ensures that the overlap zone is spatially embedded within both tiles rather than belonging exclusively to either one. The overlap serves two purposes: it defines the trigger zone within which the tile switch is evaluated, and it provides sufficient spatial margin to absorb the navigation artifacts arising from the asynchronous switch window. The resulting five tiles cover the full navigable floor with pairwise overlapping regions at each tile boundary.

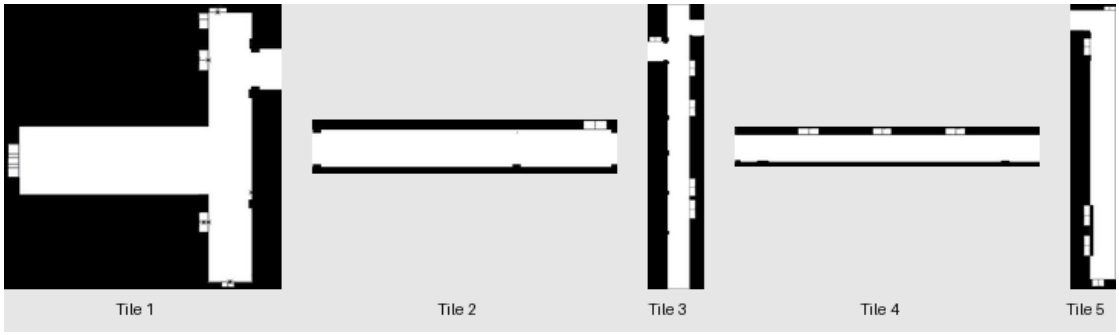

Figure 4.4 Individual tessellated tile maps (Tile 1-5).

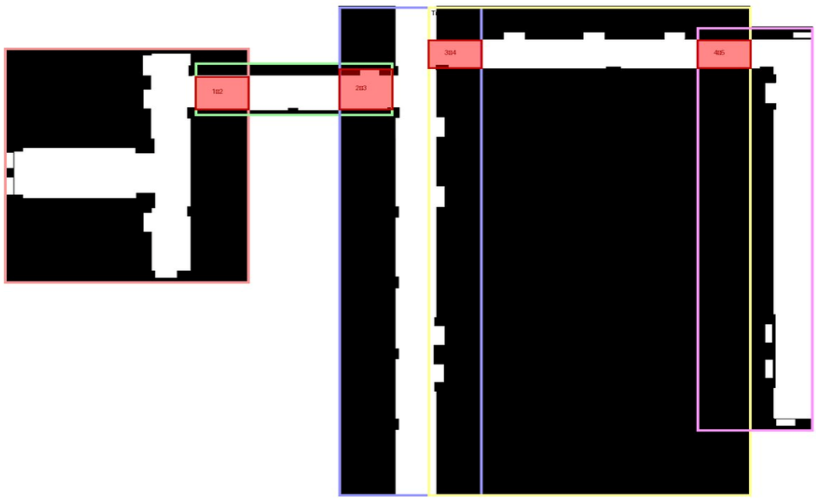

Figure 4.5 Tessellated map tile layout with overlap regions.

Table 4.1 Tile Dimensions, Physical Coverage, and Navigable Cell Statistics at 0.04 m/px resolution.

Tile	Dimensions (px)	Physical Size (m)	Total Cells	Navigable Cells	Navigable %	Descriptions
Tile 1	360×399	14.4×16.0	143,640	44,542	31.0%	Main lab corridor (H201–H208)
Tile 2	291×90	11.6×3.6	26,190	17,390	66.4%	East connector corridor

Tile 3	212×831	8.5×33.2	176,172	59,848	34.0%	Central corridor (H212–H215)
Tile 4	476×96	19.0×3.8	45,696	24,837	54.4%	North connector (SR1–SR3)
Tile 5	171×686	6.8×27.4	117,306	43,275	36.9%	East corridor (H216–H219)
Whole Map	1596×919	63.8×36.8	1,466,724	171,826	11.7%	Full floor (monolithic)

4.2.2.7 PGM Conversion and YAML Metadata

Each annotated tile PNG was converted to a grayscale raster format compatible with the navigation stack's map loader. For each tile, an accompanying metadata file specifies the map image path, resolution (0.04 *m/px*), and origin — the position of the tile's bottom-left corner in the global world coordinate frame expressed as [*x, y, θ*]. Tile origins were determined by measuring the physical offset of each tile's bottom-left corner from a common reference point and verified by visual overlay to confirm alignment of corridor features across tile boundaries.

Table 4.2 YAML Metadata Parameters for each Tile Map

Tile	Map File	Resolution (m/px)	Origin [x, y] (m)	Occ. Thresh	Free Thresh
Tile 1	tile1.pgm	0.04	[−9.8, −1.0]	0.65	0.196
Tile 2	tile2.pgm	0.04	[0.98, 10.36]	0.65	0.196
Tile 3	tile3.pgm	0.04	[9.03, −15.44]	0.65	0.196
Tile 4	tile4.pgm	0.04	[14.31, 12.6]	0.65	0.196
Tile 5	tile5.pgm	0.04	[33.3, −11.02]	0.65	0.196

4.2.3 Map Switching Algorithm

The map switching algorithm is the primary contribution of this work. It enables a robot to navigate across a tessellated environment by dynamically loading and unloading map tiles at runtime without interrupting the active navigation pipeline. The algorithm is implemented as a custom action node integrated into the navigation executor as a plugin.

4.2.3.1 Integration with the Navigation Executor

The navigation executor uses a hierarchical execution graph to orchestrate high-level navigation logic. The tile-check node is inserted into the standard reactive sequence and evaluated on every executor cycle alongside the path-following node. It is implemented as a synchronous action node that always returns *SUCCESS*, ensuring the reactive sequence continues evaluating the path-follower uninterrupted. Internally, a four-state asynchronous state machine handles the switch process across multiple execution cycles without blocking. Tile state is persisted to a file, ensuring correct tile restoration if the executor is restarted mid-session.

4.2.3.2 Tile Configuration

All tile definitions, spatial bounds, neighbour relationships, and trigger zone coordinates are specified in a centralized configuration file with three sections: settings (global parameters), tiles (individual tile metadata), and connections (overlap regions and switch geometry between adjacent tile pairs). At initialization, the algorithm parses this configuration and constructs a list of trigger zone descriptors, one per direction per connection, used for per-tick zone lookup.

Table 4.3 Tile Switching Algorithm Configuration Parameters

Parameter	Value	Description
Resolution	0.04 m/cell	Map grid resolution, consistent across all tiles
Number of tiles	5	Total tessellated tiles covering the floor
Switch cooldown	0.5 s	Minimum interval between consecutive tile switches

Post-switch delay	0.3 s	Wait for costmap rebuild after clear
LoadMap timeout	5.0 s	Maximum wait for map-load service response
ClearCostmap timeout	8.0 s	Maximum wait for costmap-clear service response
Tile persistence	Persistent file on disk	Tile ID persisted across executor restarts

4.2.3.3 Switching Cues: Position and Heading

4.2.3.3.1 Spatial Containment

The robot's current position (x, y) is obtained from the pose estimate produced by the localization module. A trigger zone is an axis-aligned bounding box *[x_min, x_max, y_min, y_*max] in world coordinates. The robot is within a trigger zone if and only if: $x\_min \leq x \leq x\_max$ *and* $y\_min \leq y \leq y\_max$.

4.2.3.3.2 Heading-Based Directional Disambiguation

Spatial containment alone is insufficient to determine switch direction. An overlap region is shared by exactly two tiles; without heading information the algorithm cannot distinguish a robot transitioning from Tile $A \rightarrow B$ versus $B \rightarrow A$. The robot's yaw angle is mapped to one of four cardinal headings **(**$+x, -x, +y, -y$) using 45-degree angular thresholds. Each TriggerZone carries a required heading. A switch from Tile $A \rightarrow B$ is triggered only when the robot is inside the overlap box AND travelling in the heading associated with the $A \rightarrow B$ direction, correctly handling robots reversing through overlap zones.

4.2.3.4 Asynchronous Switch State Machine

Once a valid trigger is detected, the algorithm initiates a tile switch through a four-state asynchronous state machine. Asynchronous (non-blocking) calls to the map-loading and costmap-clearing services are critical: a blocking call would stall the entire executor cycle loop and halt navigation. Instead, asynchronous request handles are

dispatched and polled across successive execution cycles with zero-timeout checks, allowing the path-following node to continue executing throughout the switch process.

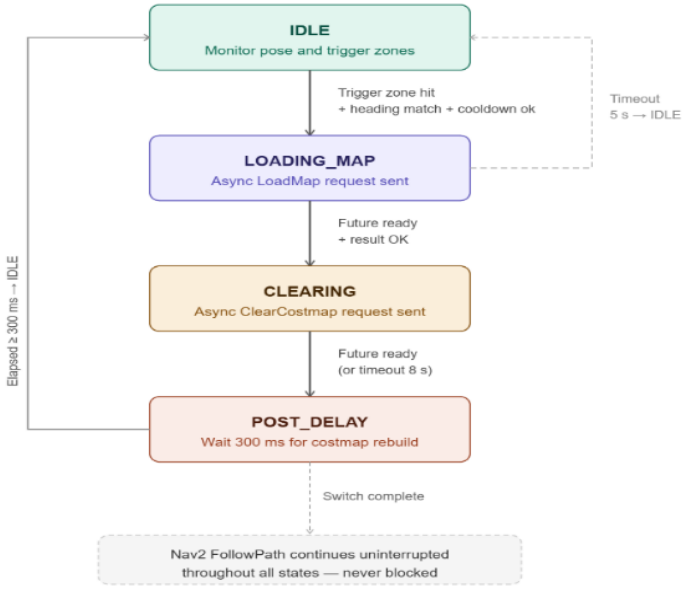

Figure 4.6 Asynchronous Tile Switch State Machine

4.2.4 Switching Artifacts and Mitigation

A known artifact of this design is that the robot operates briefly on a stale costmap during the switch window (~100–300 ms), as the tile-check node always returns SUCCESS and the path-following node continues executing. This is mitigated by the overlap zone design: both tiles share the overlap region by definition, so the outgoing tile's costmap remains geometrically valid within the overlap for the entire switch duration. The 300 ms POST_DELAY ensures the new costmap is fully populated before the next planning cycle. The 500 ms cooldown prevents oscillatory switching when the robot decelerates near a trigger zone boundary.

4.3 LOCALIZATION

The localization subsystem continuously estimates the robot’s position and orientation within the pre-built occupancy grid map. Unlike wheel odometry, which drifts unboundedly on slippery tiled floors, or pure visual odometry, which fails on textureless walls, the system fuses stereo vision with inertial measurements. This visual-inertial approach provides a drift-bounded pose estimate using only the onboard

ZED stereo camera and its integrated IMU, requiring no external infrastructure. When the estimate becomes unreliable (e.g., after a tracking loss), a secondary ArUco-based relocalization routine recovers the robot's pose by detecting fiducial markers placed at known locations in the environment. The following subsections describe the core algorithms for continuous pose estimation, marker-based relocalization, and safe recovery behaviour.

4.3.1 Motivation for Visual-Inertial Odometry over Wheel Odometry

Wheel odometry computes pose change by integrating encoder pulses. Its error grows linearly with travelled distance because each wheel slip or uneven floor surface adds a small, uncorrected error. On the corridor's polished tiles, slip is frequent: a 50 m delivery would accumulate several metres of drift, unacceptable for reaching doorways accurately.

Pure visual odometry tracks features between successive camera frames. It drifts more slowly than wheel odometry but still accumulates error, and it fails outright when the camera sees a plain white wall or a uniformly tiled floor, common in the deployment environment.

Visual-inertial odometry (VIO) combines the complementary strengths of both sensors:

- The camera provides low-drift, low-frequency pose corrections by observing visual features.
- The IMU measures acceleration and angular velocity at high frequency (30 Hz), filling the gaps between camera frames and providing short-term motion estimates that are drift-free over intervals of a few seconds.

The fusion produces a pose estimate that is more accurate and more robust than either sensor alone. The ZED Mini camera integrates a factory-calibrated IMU and runs the entire VIO pipeline internally; the robot's computer simply reads the resulting 6-DoF pose (*x, y, z, roll, pitch, yaw*) over USB. This removes the need for external sensor fusion code or additional hardware.

4.3.2 Visual-Inertial Odometry Pipeline

The VIO pipeline executed inside the ZED SDK performs the following steps for each stereo frame:

1. **Feature extraction:** ORB (Oriented FAST and Rotated BRIEF) features are detected in the left image. Features are chosen at points with sufficient texture contrast to be tracked reliably.

2. **Stereo matching:** Each left-image feature is matched to the right image to compute its 3D position (triangulation). This produces a sparse 3D point cloud in the camera's coordinate frame.

3. **IMU pre-integration:** Inertial readings (accelerometer, gyroscope) between consecutive stereo frames are integrated to form a relative motion constraint. Pre-integration expresses this relative motion in a way that can be combined with visual measurements without re-integrating the IMU data every time the visual estimate changes.

4. **Visual-inertial alignment:** A nonlinear optimization solves for the camera pose (position and orientation) that best satisfies both sets of constraints:

    a. **Visual reprojection error:** The difference between where a tracked feature appears in the image and where it is predicted to appear given the current pose estimate.

    b. **IMU pre-integration error:** The difference between the motion measured by the IMU and the motion implied by the pose change between two frames.

    c. The optimization runs at each key frame and produces the final 6-DoF pose estimate.

5. **Pose output:** The estimated pose is transformed from the camera's optical frame to the world frame (aligned with gravity) and made available as the robot's odometry.

Because the IMU provides continuous measurements between visual frames, the pose estimate is smooth and does not jump even when visual tracking momentarily loses features. When tracking recovers, the visual information corrects any drift that accumulated during the tracking loss.

4.3.3 ArUco-Based Relocalization

Although VIO is robust, it can still fail if the camera is covered, if lighting drops below usable levels, or if the robot is moved while the camera is off (e.g., at startup). In these cases, the pose estimate diverges from the true position. To recover, the robot uses ArUco markers placed at known locations on the corridor walls. The relocalization algorithm proceeds as follows:

1. **Marker detection:** The grayscale camera image is searched for square ArUco patterns using adaptive thresholding, contour detection, and perspective correction. For each candidate, the inner binary code is read and matched against the dictionary. If the number of corrected bit errors is below a threshold of 6 bits, the marker is accepted.

2. **Pose estimation:** For a detected marker of known physical size (0.28 m in this case), the four corner points in the image are used to solve the Perspective-n-Point (PnP) problem. Given the camera's intrinsic matrix K and distortion coefficients, the algorithm finds the rotation matrix R and translation vector t that minimise the reprojection error:

$$min \ \Sigma_{i=1}^{4} \| u_i - \pi(R X_i + t) \|^2 \ldots\ldots (eq\text{-}4.3)$$

   where u_i are the observed 2D corner coordinates, X_i are the known 3D corner positions of the marker in its own coordinate frame, and π is the camera projection function. The solution yields the camera's pose relative to the marker.

3. **Coordinate transformation:** Because each marker's pose in the world frame (position and orientation) is known from the configuration file, the robot's absolute pose (world frame) is computed by composing the marker-to-camera

transform with the world-to-marker transform. The result is a corrected 6-DoF pose estimate.

4. **Odometry reset:** The corrected pose is fed back into the VIO pipeline as a new reference. The pipeline reinitialises its internal state so that subsequent odometry estimates are relative to this corrected pose instead of the drifted one.

The frequency of relocalization is deliberately throttled (e.g., 3 Hz) to avoid destabilising the VIO filter. Markers farther than a `maximum_distance` (e.g., 5 m) are ignored because the PnP solution becomes unreliable at large ranges.

4.3.4 Recovery State Machine

When pose loss is detected, either by the robot's supervisor or by a manual trigger, the relocalization manager executes a finite state machine to recover the robot

without human intervention,The transition from SPINNING to REPOSITIONING occurs only after the robot has turned through at least 370° (full rotation + a small margin) to ensure it has looked in every direction. The number of repositioning moves between spins is also bounded (e.g., 3) before incrementing the full-retry counter.

4.3.5 Obstacle-Aware Recovery

Relocalisation manoeuvres (spinning and driving forward) are performed while the robot's pose estimate may be inaccurate. To prevent collisions with walls or other obstacles, the recovery algorithm continuously monitors the ZED's depth map. The procedure is:

1. **Depth sampling:** The depth image (32-bit float, metres) is cropped to a vertical strip in the centre (width ≈ 60 pixels) and the middle third in height (to ignore the floor and ceiling). Every fourth pixel is evaluated.

2. **Minimum distance:** The smallest valid depth value in the sampled region is stored as `d_min`.

3. **Safety thresholds:** Two distances are defined:

a. **safety_distance:** if `d_min` falls below this during forward motion, the robot stops and enters avoidance.

b. **critical_distance:** if `d_min` falls below this during any motion (including spinning), avoidance is triggered immediately.

4. **Avoidance manoeuvre:** The robot performs a two-phase escape:

   - **Phase 0 – back up:** reverse at low speed (e.g., 0.2 m/s) until `avoidance_backup_dist` (e.g., 0.2 m) has been travelled or a timeout (3 s) expires.

   - **Phase 1 – turn away:** rotate in place at the same angular velocity used for spinning until `avoidance_turn_angle` (e.g., 1 rad ≈ 57°) has been achieved or a timeout (5 s) expires.

5. **Resume:** After the avoidance manoeuvre, the robot returns to the state it was in before the interruption (SPINNING or REPOSITIONING). The accumulated spin angle counter is reset to ensure the robot does not incorrectly conclude that a full rotation has already been completed.

This depth-aware behaviour prevents the robot from driving into a wall when its pose estimate is wrong. It also allows the robot to recover from being placed too close to a wall at startup, because the avoidance routine will back away and turn before attempting to spin.

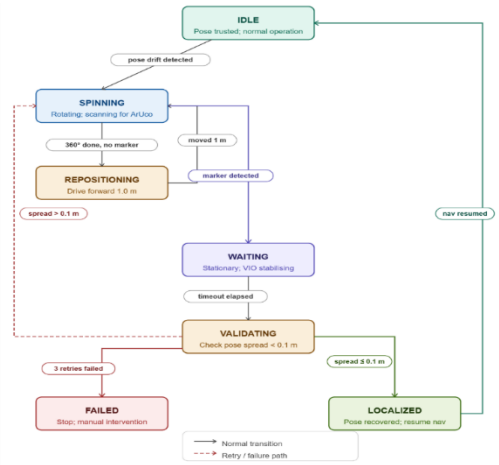

Figure 4.7. Recovery State Machine for Pose-Loss Relocalization

4.4 PATH PLANNING

Path planning transforms a desired destination into a collision-free trajectory that the robot can execute in real time. The planning system is decomposed into two complementary layers: a global planner that computes a discrete, obstacle-avoiding path through the static environment, and a local planner that continuously generates velocity commands to follow that path while reacting to dynamic obstacles. The integration of these two layers with the tile management system is also described.

4.4.1 Global Path Planning

The global planner operates on the occupancy grid of the currently active tile (Section 4.2). The occupancy grid is a discrete *2D* array where each cell is labelled as *free*, *occupied*, or *unknown*. The robot's pose is given by its center cell, and the goal is the cell corresponding to the requested room entrance. The planning problem reduces to finding a sequence of adjacent free cells from start to goal that minimises a cost function, while avoiding occupied cells.

The algorithm used is *A** (A-star) with grid-based connectivity. *A** maintains two scores for each cell:

- g(n): the exact cost of the cheapest known path from the start cell to cell n.
- h(n): a heuristic estimate of the cost from n to the goal (Euclidean distance, multiplied by a scalar).

The cost of moving from cell n to an adjacent neighbour m is:

$$c(n, m) = d(n, m) \times (1 + \alpha \cdot occupancy(m) \ldots\ldots (eq\text{-}4.4)$$

where $d(n, m)$ is the Euclidean distance between cell centres (1 or √2 for cardinal or diagonal moves). The term occupancy(m) is a continuous cost value derived from the grid: free cells have cost 0, cells near obstacles have an inflated cost (computed by a distance transform), and occupied cells are effectively infinite. The scaling factor α weighs the cost of proximity to obstacles against path length, encouraging the planner to keep a safe clearance from walls and furniture.

Expansion uses the Moore neighbourhood (8-connected cells). The heuristic is admissible because the straight-line Euclidean distance is always less than or equal to any actual path length, guaranteeing that *A** finds the optimal path with respect to the cost function.

The output of the global planner is a sequence of waypoints, each with a desired orientation (the robot's heading when approaching that point). The final waypoint includes the required final yaw, so that the robot arrives facing the room entrance rather than sideways.

If the occupancy grid is large (e.g., a full tile), the *A** search is run only once per change of the *goal* state. Intermediate replanning is triggered by the local planner when the global path becomes invalid (e.g., when a dynamic obstacle is blocking it for an extended period). Replanning uses the same A* algorithm but reuses the existing costmap which is updated continuously from the depth sensor.

4.4.2 Local Trajectory Planning

The local planner runs at a higher frequency and is responsible for converting the global path into smooth, collision-free velocity commands (v_x, ω) that respect the robot's kinematic and dynamic limits. The chosen method is the Dynamic Window Approach (DWA), which searches the space of admissible velocity commands over a short time horizon.

At each control cycle, the DWA performs the following steps:

1. Velocity space sampling – The robot's current linear and angular velocities (v, ω) define a centre point in velocity space. The algorithm samples a set of candidate velocities within a window defined by:

   The robot's acceleration limits:

   $$v \in [v\_current - \dot{v}\_max \cdot \Delta t,\ v\_current + \dot{v}\_max \cdot \Delta t] \ldots\ldots (eq\text{-}4.5)$$

   and analogously for ω.

   The global limits: $|v| \leq v\_max, \backslash\ |\omega| \leq \omega\_max$.

The sampled velocities are discretised (e.g., 10 values for linear speed, 30 for angular speed), producing a finite set of candidate control pairs.

2. Trajectory simulation – For each candidate velocity, the robot's motion is simulated forward over a fixed time horizon T (typically 3–4 s) using the kinematic model of a differential-drive robot:

$$x(t + \Delta t) = x(t) + v \cos \theta(t) \Delta t \ldots\ldots (eq\text{-}4.6)$$

$$y(t + \Delta t) = y(t) + v \sin \theta(t) \Delta t \ldots\ldots (eq\text{-}4.7)$$

$$\theta(t + \Delta t) = \theta(t) + \omega \Delta t \ldots\ldots (eq\text{-}4.8)$$

The simulation proceeds in small time steps (e.g., 0.1 s) to compute the entire predicted trajectory.

3. Cost evaluation – Each simulated trajectory is scored by a weighted sum of several criteria, normalised by the maximum possible value in the set:

$$cost = w\_path \cdot path\_dist + w\_goal \cdot goal\_dist + w\_obst \cdot obstacle\_dist + w\_occ \cdot oscillation + w\_align \cdot goal\_align$$

- Path distance – the average distance from the trajectory to the nearest point on the global path. Lower is better.
- Goal distance – the distance from the trajectory's end point to the global goal (Euclidean).
- Obstacle distance – the minimum distance from any point on the trajectory to any obstacle in the costmap. The cost is high if this distance falls below the robot's clearance radius.
- Oscillation – penalises trajectories that repeatedly change direction, preventing the robot from jittering.
- Goal alignment – penalises trajectories that end with the robot facing away from the goal orientation (only active near the final destination).

Each term is scaled by its weight (e.g., *w_path = 8.0, w_goal = 8.0, w_obst = 0.5*). The trajectory with the lowest total cost is selected.

4. Command execution – The first velocity command (v, ω) of the selected trajectory is sent to the low-level controller. At the next cycle, the process repeats, always starting from the robot's current true state (obtained from localisation).

The dynamic window ensures that only velocities reachable within one control cycle are considered, guaranteeing that the generated commands are feasible. The time horizon T must be long enough to allow the robot to come to a complete stop from its maximum speed:

$$T \geq v\_max / \dot{v}\_max \ldots\ldots eq12.21$$

where $\dot{v}\_max$ is the maximum deceleration. In practice, T is set to 3.5 s, covering the full stop distance.

4.4.2.1 Integration of Depth Obstacles

The costmap used by the local planner is not static; it is updated continuously from the ZED stereo camera's depth map. For each depth pixel, the corresponding 3D point is projected onto the ground plane (the robot's operating plane). If the point lies within the robot's height range (0.05–1.5 m above ground) and is closer than `obstacle_range` (e.g., 3.0 m), the costmap cell at that location is marked as occupied. This allows the local planner to react to dynamic obstacles that are not present in the static map, such as a person walking or a mis-placed box.

The depth map is processed at a reduced resolution (stride of 4 pixels) to limit computational load. Points that are clearly on the floor or ceiling are filtered out using a ground plane estimate.

4.4.3 Integration with Tile Management

The global and local planners operate within the active tile loaded by the tile manager (Section 4.2). When a tile switch occurs (the robot crosses from one tile to the next), the following sequence takes place:

1. The global planner receives a notification that the occupancy grid has been replaced.
2. The current goal (room location) is expressed in the coordinate system of the new tile, because tile origins are known. The goal coordinates are transformed automatically.
3. The global planner recomputes the path from the robot's current pose (which is still valid) to the transformed goal using the new tile's grid.
4. The local planner continues running uninterrupted, using the previously active map until the new one is fully loaded. The overlap regions ensure that the old map remains valid for the short transition period.
5. Once the new map is active and the global path has been recomputed, the local planner seamlessly follows the new plan.

The robot never stops or hesitates during a tile switch; the transition is transparent to the planning layers because the global planner's replanning is triggered automatically by the change in the map frame, and the local planner's costmap is updated in the next cycle.

4.5 OBSTACLE AVOIDANCE

Obstacle avoidance complements global and local path planning by providing a direct, low-latency mechanism to keep the robot clear of unforeseen obstacles. Unlike the local planner, which reasons about trajectories over a 3–4 s horizon, the obstacle avoidance layer operates on the most recent depth image and updates the costmap at 5–10 Hz. This section describes the three algorithmic components: projecting depth points onto the costmap, masking out detected humans so they are handled by a separate reactive policy, and the reactive stopping behaviour that is triggered when a human blocks the intended path.

4.5.1 Depth-Based Obstacle Detection

The robot's primary sensor for short-range obstacle detection is the stereo depth camera. Each depth image provides a 2D array of distances from the camera to the scene. The obstacle detection algorithm converts these raw depth measurements into occupied cells in the local costmap through the following steps.

1. Camera model and coordinate transformation: For each valid depth pixel at image coordinates (u, v) with measured depth d, the 3D point is first expressed in the camera's optical frame using the pinhole camera model:

$$x\_c = (u - c\_x) / f\_x \cdot d \ldots\ldots\ldots (eq\text{-}4.9)$$

$$y\_c = (v - c\_y) / f\_y \cdot d \ldots\ldots\ldots (eq\text{-}4.10)$$

$$z\_c = d \ldots\ldots (eq\text{-}4.11)$$

   where (f_x, f_y) are the focal lengths and (c_x, c_y) the principal point (obtained from the camera calibration). The point is then transformed to the robot's base frame (or directly to the map frame) using the known rigid transform from camera to robot base.

2. Height filtering: Only points that lie within a vertical band above the ground plane are considered obstacles. Points that are too low (below min_obstacle_height, e.g., 0.*05 m*) are assumed to be floor and are ignored; points above max_obstacle_height (e.g., *1.5 m*) are overhead structures (ceiling, signs) that the robot can pass under safely. This filtering eliminates spurious obstacle markings from the ground plane itself and from irrelevant high features.

3. Range gating: Points farther than obstacle_range (e.g., *3.0 m*) are discarded. This limits the layer to immediate surroundings, reducing computational load and preventing the robot from reacting to obstacles that are too far away to affect the current control cycle

4. Subsampling (stride): Processing every pixel of a depth image (e.g., 640 × 480) would be too slow. The algorithm examines only every k-th pixel in both directions, where k is the stride (typically 2–4). This reduces the number of points to process by a factor of *k^2* while still providing sufficient coverage for obstacle detection.

5. Ground projection and costmap update: For each surviving 3D point, its (x, y) coordinates on the ground plane are computed. The corresponding cell in the costmap (at resolution r, e.g., *0.05 m/cell)* is marked as LETHAL_OBSTACLE (maximum cost). The costmap accumulates these markings from consecutive

frames; a decay mechanism (not part of this layer) gradually reduces costs when obstacles are no longer observed.

4.5.2 Human Detection and Masking

If the robot treated every detected person as a static obstacle, the local planner would constantly attempt to replan around them, leading to erratic motion (and potentially dangerous behaviour) in crowded corridors. Instead, the system distinguishes between humans and other obstacles.

4.5.2.1 Human Detection

The ZED camera's built-in object detection model runs concurrently on the RGB stream, outputting a list of detected objects with their class labels and 2D bounding boxes in the image. For each object labelled Person, the bounding box coordinates (*u_min, v_min, u_max, v_max*) are recorded along with the timestamp.

4.5.2.2 Mask Generation

Before projecting depth points into the costmap, the algorithm expands each human bounding box by a padding margin (e.g., 20 pixels) in all directions. All depth pixels falling inside any padded human bounding box are excluded from obstacle marking. This means that the costmap does not show a lethal obstacle where a person is standing.

4.5.2.3 Persistence

Human detections are not instantaneous; they may flicker from frame to frame. To avoid rapid toggling of the mask, each detection is kept valid for a short persistence window (human_persistence, e.g., 0.5 s). If a person is detected in the current frame, the mask is active. If no person is detected, the mask remains active for the persistence interval (using the most recent detection) before being cleared. This ensures that a brief missed detection (e.g., due to occlusion) does not cause the robot to suddenly see an obstacle where a person is still present.

4.5.2.4 Effect on Navigation

Because humans are masked out of the costmap, the local planner does not see them as obstacles and will not attempt to avoid them. The responsibility for safe interaction with humans is transferred to the reactive stopping policy described next.

4.5.3 Reactive Stopping Behaviour

While the costmap masking prevents the robot from actively manoeuvring around people, the robot must still stop when a person is too close and blocking the intended path. This is implemented as a gating condition placed between the local planner's velocity output and the motor controller.

1. Path representation: The global path (from the robot's current position to the goal) is discretised into a sequence of points at a fixed resolution (e.g., 0.2 m spacing). The path is expressed in the global coordinate frame.

2. Human position tracking: The same human detections (class Person) are transformed into the global frame using the camera's pose and the robot's localisation. Each person's 3D position is stored, and stale detections are expired after the persistence window.

3. Blocking condition: A human is considered to be *blocking the path* if both of the following are true:

    - The Euclidean distance from the human to the robot is less than a threshold human_stop_distance (e.g., 0.5 m). This ensures the robot only reacts when a person is nearby, not to someone far away.

    - The perpendicular distance from the human to the nearest point on the path is less than path_width (e.g., 0.5 m). This defines a corridor around the path; a person standing just outside this corridor does not trigger a stop.

4. Stop and Wait: If a blocking human is detected, the robot immediately:

    - Publishes a zero-velocity command (stops all motion).

- Optionally calls a text-to-speech service to announce “Please move away from the robot.” (Repeated at speak_interval, e.g., every 5 s.)

- Continues to monitor the condition. As long as the human remains blocking, the robot stays stopped.

- Once no human satisfies the blocking condition, the robot resumes normal navigation (the local planner’s velocity commands are allowed through the gate).

5.5.4 Asymmetric behaviour

This policy applies only to humans. For non-human obstacles (walls, furniture, boxes), the depth-based obstacle layer marks them as lethal in the costmap, and the local planner (Section 4.4.2) will replan around them automatically. No special stop-and-wait logic is invoked.

4.5.5 Integration with Navigation Stack

The depth obstacle layer is inserted into the costmap pipeline after the static map layer but before the inflation layer. This ordering ensures that dynamic obstacles (including non-human ones) are added on top of the static map, and the inflation layer then propagates costs outward from all obstacles (including masked-out humans? – humans are not marked, so no inflation from them). The stopping gate is placed after the local planner’s velocity output, overriding it only when a human is blocking. This design keeps the local planner unaware of the human-specific policy; the planner continues to generate trajectories as if no human were present, but those commands are simply suppressed. When the human moves away, the planner’s last computed command is still valid (since the world did not change much), and execution resumes without a noticeable pause.

4.6 CONTROL

The control subsystem transforms high-level velocity commands (linear and angular) into individual wheel motor voltages that make the robot move as requested. Unlike the local planner, which computes desired velocities based on a trajectory, the

controller runs at a higher frequency and directly regulates the wheel speeds using feedback from wheel encoders. The system is organised as a cascade of four stages: command prioritisation, kinematic transformation, closed-loop velocity regulation, and low-level actuator mapping. A watchdog timer ensures safety if the command stream stops unexpectedly.

4.6.1 Command Prioritisation

The robot may receive velocity commands from multiple sources concurrently: the autonomous navigation stack, a human-operator joystick (for teleoperation), an emergency stop, or a relocalisation routine that requires spinning in place. To arbitrate between these sources without conflict, the system uses a priority-based multiplexer. Each command source is assigned a priority level, and at any moment the command with the highest numerical priority is forwarded to the controller. The defined priorities are:

Table 4.4 Velocity command source priorities for the priority-based multiplexer

Topic	Priority	Typical use case
Emergency stop	30	Immediate halt (highest priority)
Joystick	20	Manual teleoperation
Navigation	10	Autonomous operation
Relocalisation	5	Spin manoeuvre while recovering pose

If a higher-priority source stops publishing commands (or exceeds a timeout), the multiplexer falls back to the next-highest active source. This allows teleoperation to pre-empt autonomy, and the emergency stop to pre-empt everything else. The timeout ensures that a crashed or disconnected source does not block the control channel indefinitely.

4.6.2 Kinematic Transformations

The robot uses a differential drive configuration: two independently driven wheels on a common axis, plus passive castors for stability. The relationship between the commanded linear velocity v (m/s) and angular velocity ω (rad/s) and the required left and right wheel angular speeds ωL, ωR (rad/s) is:

$$\omega L = v/r - (\omega \cdot b)/(2r) \ldots\ldots\ldots (eq\text{-}4.12)$$

$$\omega R = v/r + (\omega \cdot b)/(2r) \ldots\ldots\ldots (eq\text{-}4.13)$$

where r is the wheel radius (0.05 m) and b is the wheelbase (0.40 m). These equations are derived from the instantaneous centre of rotation of a differential robot. The derived wheel speeds are then saturated to the physical limits of the motors (maximum angular speed ≈ 15 rad/s, corresponding to a linear speed of 0.6 m/s).

4.6.3 Velocity Control Loop

The core of the controller is a feedforward + PI (Proportional-Integral) scheme that compensates for modelling errors, friction, and load variations. The control law for each wheel is:

$$u(t) = K\_ff \cdot \omega\_cmd(t) + K\_p \cdot e(t) + K\_i \cdot \int_0^t e(\tau)\, d\tau \ldots\ldots\ldots (eq\text{-}4.14)$$

where:

- ω_cmd(t) is the desired wheel angular speed from the kinematic transformation,
- e(t) = ω_cmd(t) - ω_actual(t) is the speed error,
- K_ff is the feedforward gain (≈1.16), which does most of the work,
- K_p and K_i are the proportional and integral gains.

The feedforward term alone would produce open-loop control; the PI terms correct residual steady-state error and compensate for unmodeled disturbances. The integral term is clamped to prevent windup (maximum absolute value 0.5 *rad/s*). When the commanded velocity is zero, the integral term is reset to zero to avoid unwanted residual torque after stopping.

The control law is executed at the controller's update rate ($\approx 15\,Hz$). The actual wheel speeds are obtained from the quadrature encoders (Section 4.6.4) after filtering.

4.6.4 Wheel Speed Measurement

Each drive motor is equipped with a quadrature encoder that outputs two pulse trains (channels A and B), 90° out of phase. The direction of rotation is determined by the phase relationship (A ahead of B indicates forward; B ahead of A indicates reverse). To reduce computational load and interrupt frequency, the system triggers interrupts only on both edges of channel A, while reading channel B at each interrupt to determine direction. This yields 204 counts per revolution (for a motor with 102 counts per revolution in full quadrature mode). The angular speed is computed from the change in tick count over a fixed time interval:

$$\omega = (\Delta ticks / \Delta t) \times (2\pi / ticks\_per\_rev) \ldots\ldots (eq\text{-}4.15)$$

A moving-average filter (window size = 2 samples) smooths the velocity estimate and rejects high-frequency noise. Readings below a threshold (0.04 rad/s) are zeroed to prevent drift when the robot is stationary. An additional spike-rejection filter discards tick changes that exceed physically-possible limits (e.g., >20 rad/s within one sampling interval), which can occur due to electromagnetic interference or contact bounce.

4.6.5 Low-Level Actuator Mapping

The motor driver (Cytron SmartDriveDuo-30) accepts commands over a serial (UART) interface using a Simple Serial protocol. Each command is two bytes: one for the left motor, one for the right. The byte value encodes both direction and speed in a custom mapping:

Table 4.5 Simple Serial Byte Encoding for Direction and Speed

Left motor byte	Right motor byte	Meaning
0	0	Stop

1–63	193–255	Reverse (speed 1..63)
64	128	Stop
65–127	129–191	Forward (speed 1..63)

The mapping from the controller's output speed u (in rad/s) to the motor driver byte is:

- For forward motion: byte = 65 + 62 × (u / u_max) (clamped to 65–127 for left, 129–191 for right).
- For reverse motion: byte = 1 + 63 × ($|u|$ / u_max) (clamped to 1–63 for left, 193–255 for right).

The asymmetry between left and right byte ranges comes from the driver's protocol specification. The normalised speed u / u_max is first computed from the controller output, then mapped onto the appropriate byte range.

4.6.6 Watchdog Safety

To prevent the robot from continuing to move if the velocity command stream stops (e.g., because the navigation stack crashes), a watchdog timer is implemented in the low-level motor driver. Every time a new velocity command is received, the watchdog is reset. If no command arrives for 0.3 s, the watchdog triggers and sends a stop command (0 for left, 128 for right) to the motor driver. This ensures that the robot halts within a bounded time even if the higher-level control software fails.

4.6.7 Control Loop Summary

The complete control pipeline from desired velocity to wheel motion proceeds as follows:

1. Priority selection – The highest-priority active source's (v, ω) command is selected.
2. Kinematic transformation – (v, ω) is converted to desired wheel angular speeds ($\omega L,d$, $\omega R,d$) using the differential drive equations.

3. PI + feedforward – For each wheel, the control law computes a corrected speed u from the desired speed and encoder-measured actual speed.
4. Mapping – The corrected speed is normalised, then mapped to the motor driver's byte value.
5. Actuation – The two-byte command is sent over UART to the motor driver.
6. Watchdog – The timer is reset. If no command arrives within 300 ms, a stop command is forced.

The complete loop runs at approximately *15 Hz*, limited by the UART communication and the encoder sampling rate.

4.7 Task Assignment and Management

The task management subsystem accepts delivery requests from users, schedules them for execution, and coordinates the robot's autonomous navigation to fulfil each request. Unlike the navigation and control pipelines, which operate on continuous sensor data, task management is event-driven: requests arrive asynchronously, the robot executes one job at a time, and the system persists its state to survive unexpected restarts. The subsystem is built around three core components: a single-job executor (the robot handles only the current job; no queue is maintained onboard), a state machine that defines the job lifecycle, and a confirmation protocol that requires human operators to acknowledge pickup and dropoff. The following subsections describe the job data structure, the state machine, the execution flow, the recovery mechanism after an interruption, and the coordination with the navigation stack.

4.7.1 Job Representation

Each delivery job is described by a small set of immutable attributes:

- Job ID – A unique identifier assigned by the cloud backend (e.g., "*JOB_024*"). The robot never generates its own IDs; it uses the ID provided by the external request.
- Pickup room – The room identifier (e.g., "*H203*") where the item must be collected.
- Dropoff room – The destination room.

- Priority – An integer (*0 = normal, >0 = high*). High-priority jobs are not pre-emptive (they cannot interrupt an already-running job) but jump ahead of waiting jobs if the robot is idle.
- Pickup confirmed – A boolean flag indicating whether the item has already been placed on the robot. This is critical for recovery: if the robot crashes after picking up an item, it must resume by going directly to the dropoff, not back to pickup.

At any moment, the robot holds at most one job – the current job. No queue is stored onboard. If multiple requests arrive while the robot is busy, they are rejected immediately, and the cloud backend retries later. This design keeps the onboard task manager simple and eliminates the need for persistent queue storage.

4.7.2 Job Lifecycle State Machine

Each delivery job progresses through a finite set of states from acceptance to completion. The state machine is shown in Figure 4.X. The robot begins in IDLE and, once a job is accepted, transitions through *NAVIGATING_TO_PICKUP* → *PICKUP_ARRIVED* → *NAVIGATING_TO_DROPOFF* → *DROPOFF_ARRIVED,* with human operator confirmation required at each arrived state before navigation continues. The terminal states are *COMPLETE* and *CANCELLED.*

Two transitions are asymmetric by design. If the operator aborts or times out at pickup, the job transitions to CANCELLED — the robot never picks up an item and returns to IDLE. If the operator aborts or times out at dropoff, the job still transitions to COMPLETE, because once an item is on the robot it cannot be returned to the pickup point automatically; the item is considered delivered and left at the dropoff location.

4.7.3 Execution Flow

The task manager runs a dedicated executor thread that processes jobs sequentially. The algorithm is:

while running:

if no current job:

sleep and check again

else:

```
job = current_job
if job.interrupted:
    resume from saved state
else:
    start from beginning
success = execute_job(job)
if success:
    navigate home
clear current job
```

The execute_job(job) function follows the state machine:

1. Wait for navigation stack – Set nav_needed = true. Wait for the supervisor to publish /system/nav_ready (or timeout after 120 s). If timeout, job → *FAILED*.
2. Pickup phase (if not resuming after pickup):
   - Navigate to pickup_room (blocks until arrival or failure).
   - On arrival, set state to *PICKUP_ARRIVED* and publish a waiting message.
   - Wait for confirmation via the /job/confirm service, with timeout pickup_timeout (e.g., 300 s).
   - If confirmed (proceed = true), set pickup_confirmed = true and save state. If aborted or timeout, job → *CANCELLED*.
3. Dropoff phase:
   - Navigate to dropoff_room.
   - On arrival, set state to *DROPOFF_ARRIVED* and wait for confirmation.
   - If confirmed or timeout, job → *COMPLETE* (item considered delivered). If aborted, also → *COMPLETE* (cannot return the item). The pickup_confirmed flag is cleared.
4. Return home (after all jobs): navigate to the home_room (e.g., charging station). Set nav_needed = false.

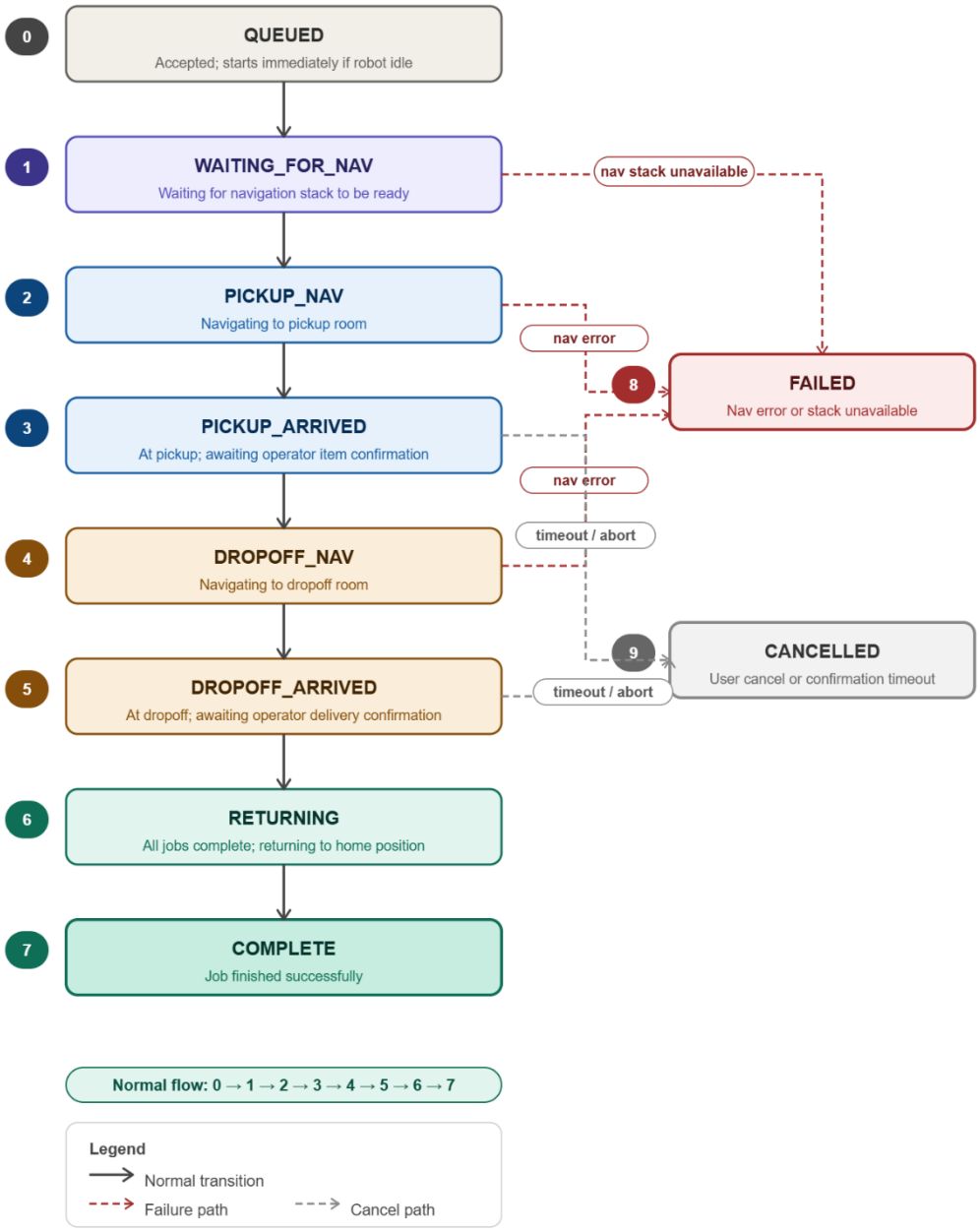

Figure 4.8. Job lifecycle state machine

4.7.4 Confirmation Protocol

The confirmation mechanism is asynchronous and uses an event flag to decouple the waiting executor thread from the ROS 2 service callbacks. When the robot reaches a pickup or dropoff location:

1. The executor publishes a message on */job/awaiting_confirmation* describing the stage, location, and expected service call syntax.

2. It then waits on a thread-safe Event object with a timeout.
3. The service callback for */job/confirm* sets the event and stores the proceed boolean.
4. If the event times out or proceed == false, the executor aborts the waiting phase.

The timeout values are set independently for pickup and dropoff (both default to 300 s) to give the human operator ample time to walk to the robot, place or remove the item, and confirm.

4.7.5 Persistence and Recovery

If the robot software crashes or is restarted while a job is in progress, the task manager must recover the job to avoid losing state (e.g., the robot might already be carrying an item). Recovery is handled by writing the current job to disk before each critical transition.

When state is saved:

- After accepting a new job (before any navigation).
- After the item has been picked up (pickup_confirmed = true), before dropoff navigation.

What is saved:

A JSON file *(/tmp/job_manager_state.json)* containing:

- Job ID
- Pickup and dropoff rooms
- Priority
- Current state
- Message
- pickup_confirmed flag
- interrupted flag (set to true when restoring)
- resume_state (the state from which execution should restart)

Restoration logic:

On startup, the task manager checks for the existence of this file. If found:

- It loads the job and marks it as interrupted.

- If pickup_confirmed is true, the robot must already be carrying the item. The job resumes at the dropoff navigation phase.
- If pickup_confirmed is false, the robot resumes at the pickup navigation phase (it may have crashed before reaching the pickup room).
- The original resume_state is used to decide which step to skip.

After successful completion or cancellation, the state file is deleted. This mechanism ensures that a power cycle or software crash does not lose the knowledge of whether the robot is carrying an item.

4.7.6 Coordination with Supervisor and Navigation

The task manager does not directly start or stop the navigation stack. Instead, it signals a supervisor node via a simple protocol:

- /system/nav_needed (Bool) – Published at 1 Hz. true when a job is active or the robot is returning home; false when idle.
- /system/nav_ready (Empty) – Published by the supervisor once all navigation-related nodes (ZED, Nav2, costmaps, etc.) have been started and are ready.
- /system/nav_shutdown (Empty) – Published by the supervisor when it needs to shut down the navigation stack (e.g., on low battery or after prolonged idleness).

The task manager waits for /system/nav_ready before calling /navigate_to_room. If */system/nav_shutdown* arrives during a job, the current navigation attempt will fail, and the job transitions to *FAILED*.

4.7.7 Cancellation

A job can be cancelled by a service call to */cancel_job.* The request specifies either a specific job ID or an empty ID (meaning “cancel the current job”). Cancellation is only allowed before the item has been picked up. If *pickup_confirmed* is true, the cancellation is rejected – the robot must complete the delivery once it is carrying an item.

When cancellation is accepted:

- A *_cancel_requested* flag is set.

- If the robot is navigating, it calls the */cancel_mission* service to stop the current navigation and then returns to the home position.
- The job state is set to *CANCELLED*, and the state file is cleared.
- The robot becomes idle and ready for the next job.

4.7.9 Summary

The task management subsystem provides a fault-tolerant, single-job execution model that interacts cleanly with the navigation stack and human operators. Its key algorithmic features are:

- No onboard queue – The robot only holds the current job, simplifying state and reducing memory.
- Deterministic state machine – Nine discrete states cover the entire job lifecycle from acceptance to completion or failure.
- Asynchronous confirmation – An event-driven wait-for-confirmation protocol separates the executor thread from the service callback.
- Crash recovery – Persistent storage of the current job, including the critical pickup_confirmed flag, allows resumption after an unexpected restart.
- Coordination with supervisor – A minimal signal protocol (nav_needed / nav_ready / nav_shutdown) decouples task management from the details of launching and shutting down the navigation stack.
- Safe cancellation – Cancellation is prohibited once an item is on board, preventing lost items.

CHAPTER 5

SYSTEM INTEGRATION

The preceding chapters have addressed the robot's hardware platform, the individual software algorithms, and the cloud-based user interface as separate concerns. Chapter 3 described the physical system, sensors, actuators, the Jetson Nano compute module, and the supporting electronics. Chapter 4 developed each software subsystem in isolation: map generation and switching, visual-inertial localisation, path planning, obstacle avoidance, velocity control, and task assignment. Chapter 5 (User Interface) described the cloud backend and browser UI through which operators interact with the robot. What none of those chapters addressed is how these components are assembled into a single, continuously running system on constrained embedded hardware, how they are deployed, how they communicate, how they are started in the correct order, and how the system remains stable across failures.

This chapter addresses that integration layer. Section 5.2 describes the Docker-based deployment environment that packages the entire software stack into a reproducible, self-contained image running on the Jetson Nano's ARM64 architecture. Section 5.3 covers ROS 2 as the middleware backbone, the publish-subscribe and service-call transport that connects every node across the stack. Section 5.4 explains the role of the Nav2 navigation framework, specifically which components were used, which were configured, and how the custom subsystems from Chapter 4 plug into its lifecycle and plugin architecture. Section 5.5 describes the System Supervisor, the top-level process responsible for orderly startup, health monitoring, and the recovery hierarchy that keeps the robot operational under sensor, navigation, and network faults. Section 5.6 documents the launch and startup configuration that ties all of the above together. Section 5.7 provides a consolidated interface table mapping every inter-component communication channel.

5.1 DEPLOYMENT ENVIRONMENT

The entire software stack runs inside a single Docker container, packaged as a reproducible image and deployed identically on both the development laptop (amd64) and the Jetson Nano compute module on the robot (ARM64). Containerization eliminates the dependency management and environment-replication problems that

arise on embedded systems: the same image that passes integration tests on a developer's workstation is the image that executes on the robot, byte for byte. The container is built in a two-stage process. A base image (Dockerfile.base) installs the heavyweight dependencies, ROS 2 Humble, CUDA 12.1 runtime, and ZED SDK 4.2.5, and is pre-built and hosted on the GitHub Container Registry at ghcr.io/tejasmk-tkp/material-transfer-robot-base-{amd64|arm64}:latest. A thin development image (Dockerfile.dev) layers the workspace-specific ROS packages and Python dependencies on top of the base, allowing rapid iteration without rebuilding the multi-gigabyte SDK layer.

The build is architecture-aware throughout. On amd64 the base image downloads and extracts CUDA 12.1 packages, cuDNN 8.9.7, and TensorRT 8.6.1 libraries directly into the image. On ARM64 these libraries are not embedded in the image; instead, they are bind-mounted from the Jetson's host system at runtime so the container uses the Jetson-native CUDA installation without replicating it. The entrypoint script (ros_entrypoint.sh) sources ROS 2 Humble and the workspace overlay in order, then restores the host-mounted library paths ahead of the ROS paths in *LD_LIBRARY_PATH*, ensuring that the Tegra, CUDA, and ZED shared libraries resolve correctly even after ROS sets its own environment.

5.1.1 Service Definition and Network Configuration

The compose configuration defines a single service, material-transfer-robot, which is the only long-running process on the robot. The container runs with host networking (network_mode: host) and a shared IPC namespace (ipc: host), giving ROS 2 DDS discovery access to the full host network stack without NAT and enabling zero-copy shared-memory transport between the container and any co-located processes. The container is launched in privileged mode to allow access to hardware device nodes; on the Jetson this includes the full set of NVIDIA multimedia and ISP device nodes required by the ZED camera driver and CUDA compute pipeline.

All ROS 2 communication is routed through CycloneDDS as the RMW implementation (*RMW_IMPLEMENTATION=rmw_cyclonedds_cpp*), and all nodes on the robot share domain ID 42 (*ROS_DOMAIN_ID=42*). The active DDS configuration is selected at container start time by setting the ROS_*MODE* environment variable,

which maps to one of three CycloneDDS XML configuration files. This provides a single container image that can be deployed in three distinct communication postures without any rebuild.

Table 5.1 CycloneDDS communication modes

Mode	DDS Scope	Multicast	Typical Use
local (default)	Loopback only (127.0.0.1)	Disabled	Single-device operation; no traffic leaves the host
network	Physical network interface (CYCLONE_INTERFACE_IP)	Enabled	Multi-device ROS 2 — full DDS discovery across machines
viz	Physical network interface; FragmentSize 1280 B	Enabled	Remote RViz via bridge; throttled, low-bandwidth WiFi link

5.1.2 ARM64 Jetson-Specific Configuration

Deploying on the Jetson Nano requires extending the base compose file with a platform-specific overlay (docker-compose.arm64.yml). This overlay adds the device node mappings and host library bind-mounts that give the container access to Jetson-specific hardware. The NVIDIA multimedia engine is exposed through a set of nvhost device nodes covering the GPU, video encoder (*MSENC*), video decoder (*NVDEC*), JPEG accelerator (*NVJPG*), ISP, and camera controller; the ZED camera's Tegra ISP pipeline requires all of these to be present. Graphics and EGL are provided by the Tegra EGL and DRI device nodes, and CUDA libraries are bind-mounted read-only from the

host's Jetson installation at /usr/local/cuda-10.2 (mapped to /usr/local/cuda inside the container). The cuDNN libraries and Tegra graphics runtime are similarly mounted from host paths, avoiding the need to embed these Jetson-specific binaries in the container image itself.

Four supplementary Unix groups are added to the container process — Input (GID 104), Dialout (GID 20), GPIO (GID 999), and Audio (GID 29) — granting the ROS nodes running inside the container the same device access rights as the robotics user on the host. The ZED camera settings and neural-network model resources are stored in named Docker volumes (zed_settings, zed_resources) so they persist across container restarts and are not re-downloaded on each launch.

5.1.3 Remote Visualisation (Viz Bridge)

A separate compose overlay (docker-compose.viz.yml) provides an optional remote visualisation capability. When active, two additional services are started alongside the main robot container. The viz-bridge service runs on the robot and subscribes to selected ROS 2 topics on domain 42 (the local CycloneDDS domain) and re-publishes them on domain 43 using the network CycloneDDS configuration. A companion viz-laptop service runs on the development laptop, subscribes on domain 43, and launches RViz2 against the forwarded topics. The bridge configuration (viz_bridge_config.yaml) specifies which topics to forward and at what rate; by default it forwards the ZED compressed RGB image at 2 Hz and the TF tree at 10 Hz. The bridge supports hot-reload via inotify: editing the configuration file takes effect within 0.2 s without restarting the container. This architecture keeps the robot's internal DDS traffic on the local loopback while exposing only a throttled, bandwidth-managed subset to the WiFi network.

5.2 MIDDLEWARE

ROS 2 (Robot Operating System 2) is the middleware layer that connects every software component on the robot. It provides the transport abstractions via topics, services, actions, and parameters, through which nodes publish sensor data, issue commands, call procedures, and negotiate configuration without needing to know the physical location or implementation language of any other node. Every custom subsystem described in Chapter 4, the ZED camera driver, the Nav2 navigation stack,

the cloud agent bridge, and the system supervisor all participate in the same ROS 2 graph running inside the Docker container on the Jetson Nano.

ROS 2 was selected over ROS 1 because it was designed from the ground up for real robot deployments: it offers a security model, deterministic lifecycle management, and a DDS-based transport that supports both intra-process zero-copy and cross-device communication on the same API. All major navigation and perception libraries in the current robotics ecosystem Nav2, the ZED ROS 2 wrapper, and the packages used for localisation and costmap management target ROS 2 exclusively, making it the only practical choice for new development on this platform.

5.2.1 Communication Patterns

The system uses all four ROS 2 communication patterns, each chosen for the characteristics of the data it carries. Topics carry continuous or periodic data between a publisher and any number of subscribers without either side needing to know about the other. The ZED camera driver publishes depth images, RGB images, point clouds, object detections, and the camera's visual-inertial odometry estimate on separate topics; the costmap layers, the localisation node, and the visualisation bridge each subscribe to whichever streams they need independently. The /*cmd_vel* topic carries velocity commands from the controller to the motor driver at the control loop rate. The /*tf* and /*tf_static* topics broadcast the robot's coordinate frame tree, which every node that reasons about geometry consults continuously.

Services handle synchronous request-response interactions where the caller needs a definite answer before proceeding. The cloud agent's ROS bridge calls a delivery service to hand a new job to the task manager; the task manager calls navigation services to trigger map loads and goal updates. Services are appropriate here because these are discrete, stateful transitions the caller must know whether the operation succeeded before taking the next step.

Actions extend the service pattern for long-running goals that can be preempted and provide incremental feedback. The Nav2 stack exposes a NavigateToPose action that the task manager sends goal poses to; the action server streams feedback (current pose, estimated time to goal) back while navigation is in progress and returns a result

when it succeeds, fails, or is cancelled. This is the primary interface through which the robot's high-level task logic drives navigation.

Parameters provide a uniform configuration interface for all nodes. Costmap resolution, obstacle thresholds, controller gains, and human-detection persistence windows are all exposed as ROS 2 parameters, allowing them to be inspected and adjusted at runtime without recompilation. Launch files set parameter values from YAML files at startup, providing a single authoritative configuration record for each deployment environment.

5.2.2 DDS Transport and the Choice of CycloneDDS

ROS 2 does not implement its own network transport; it delegates that responsibility to a DDS (Data Distribution Service) implementation. DDS is a publish-subscribe middleware standard that handles discovery, serialisation, and delivery. ROS 2 ships with FastDDS as its default implementation but supports alternative implementations through a vendor-neutral RMW (ROS Middleware) abstraction layer. For this project, CycloneDDS was selected as the DDS implementation (*RMW_IMPLEMENTATION=rmw_cyclonedds_cpp*) and is used for all inter-node communication across the entire stack.

The primary motivation was cross-device communication reliability. The deployment scenario requires ROS 2 nodes to discover each other across two physical machines the Jetson Nano on the robot and a development laptop over a WiFi network. During development, FastDDS exhibited inconsistent participant discovery in this configuration, with nodes on different machines failing to find each other without manual workarounds. CycloneDDS provided reliable cross-device discovery in the same network environment without additional configuration. Lower latency and reduced CPU overhead on the Jetson's ARM64 architecture were secondary benefits observed during testing but were not the primary driver of the decision.

The CycloneDDS configuration is controlled through XML files selected at container startup via the ROS_MODE environment variable (described in Section 5.2). In local mode, the DDS domain is confined to the loopback interface, preventing any DDS traffic from reaching the network. In network mode, the domain is bound to a specific WiFi interface IP address with multicast enabled, allowing full cross-device

topic and service discovery. The viz mode uses the same network binding but adds a smaller fragment size (1280 bytes) to improve reliability over congested WiFi links when forwarding camera data to a remote RViz instance. All three modes share domain ID 42, ensuring that nodes always communicate within a single isolated ROS graph regardless of the active network posture.

5.2.3 Quality of Service Configuration

DDS exposes Quality of Service (QoS) policies that govern how messages are buffered, delivered, and dropped. Rather than applying a single profile uniformly, the system uses different profiles for different data classes based on their delivery requirements. Sensor topics depth images, RGB frames, point clouds, and object detections from the ZED camera are published with a Best Effort reliability policy and a small history depth. For these streams, a missed frame is less harmful than the latency introduced by retransmitting it; the next frame will arrive within milliseconds and will be more current anyway. Dropping a stale depth frame is preferable to acting on one that arrived late.

Command and state topics carry a different contract. Velocity commands on /cmd_vel, service calls for job delivery, and the transform tree on /tf use a Reliable reliability policy, ensuring that every message is delivered or the failure is reported. Losing a velocity command or a coordinate frame update can cause the robot to behave dangerously or incorrectly; reliability is worth the additional overhead. This split Best Effort for high-rate sensor data, Reliable for low-rate commands and state reflects a standard ROS 2 practice and was applied consistently across all custom nodes in the system.

5.2.4 Managed Node Lifecycle

ROS 2 introduces a managed lifecycle for nodes that need to be initialised, activated, deactivated, and cleaned up in a controlled order. A LifecycleNode transitions through defined states Unconfigured, Inactive, Active, and Finalized in response to external commands, making it possible to bring up a complex stack in a guaranteed sequence and to shut down or restart individual components without affecting others. This is in contrast to plain ROS nodes, which start immediately and offer no standard mechanism for ordered startup or graceful shutdown.

The Nav2 navigation framework uses the lifecycle model throughout: its map server, localisation node, costmap servers, planner, controller, and behaviour tree executor are all lifecycle nodes managed by a Nav2 lifecycle manager. The lifecycle manager configures and activates them in dependency order at startup and can deactivate and reconfigure individual nodes (for example, to load a new map) without restarting the entire stack. The custom nodes developed for this project the task manager, the cloud agent bridge, and the system supervisor are plain ROS 2 nodes rather than lifecycle nodes. They interact with the Nav2 lifecycle manager through service calls when they need to trigger a map switch or reset navigation state, but they do not themselves participate in the lifecycle state machine. Section 5.5 describes how the system supervisor coordinates the startup sequence across both lifecycle and plain nodes.

5.3 NAVIGATION FRAMEWORK

Nav2 is the open-source navigation stack for ROS 2. It provides a complete pipeline from map serving and localisation through global path planning, local trajectory following, costmap management, and recovery behaviour execution. Rather than building these components from scratch, the project configured and extended Nav2 — using its standard nodes for the parts that map directly onto the problem, and replacing or augmenting specific layers with custom plugins where the standard behaviour was insufficient. This section describes which Nav2 components are used, what configuration choices were made and why, and what custom plugins were developed to integrate the tile-switching map system and depth-camera obstacle layer described in Chapter 4.

5.3.1 Stack Overview

The Nav2 components active on the robot are: the behaviour tree navigator (*bt_navigator*), the global path planner (*planner_server*), the local trajectory controller (*controller_server*), the global and local costmap servers *(global_costmap, local_costmap*), the recovery behaviour server (*behavior_server*), and the lifecycle manager that brings them all up in order. Map loading and localisation are handled outside the standard Nav2 map_server and AMCL nodes — instead, a custom tile manager plugin is responsible for loading the correct floor tile at runtime and updating

the active map as the robot moves between zones, as described in Chapter 4. All Nav2 nodes are lifecycle-managed: the lifecycle manager configures and activates them at startup and can deactivate and reconfigure individual nodes on demand when a tile switch is triggered.

5.3.2 Behaviour Tree Navigator

All navigation task execution is driven by Nav2's behaviour tree navigator. Rather than hard-coding a navigation state machine, bt_navigator interprets a behaviour tree XML file at runtime and dispatches actions — ComputePathToPose, FollowPath, recovery spins and backups — as tree nodes execute. The default navigation behaviour tree (navigate_with_tiles.xml) was customised to incorporate the tile-switching and human-aware navigation logic. The custom tile_manager_bt_plugin BT node handles map tile transitions as a first-class step within the navigation sequence, and human-blocking detection nodes (from the *nav2_depth_obstacle_layer_bt_nodes* library) gate forward motion when a person is detected on the path. The BT loop runs at 10 Hz (*bt_loop_duration*: 100 *ms*) and the transform timeout is set to 0.5 s to allow for occasional ZED odometry latency spikes without aborting the navigation goal.

The behaviour tree also registers the standard Nav2 recovery nodes, BackUp, Spin, and Wait, which the tree can invoke when the robot detects it is stuck. The is_stuck condition node monitors for progress failures; if the robot has not moved the required minimum distance within the time allowance, the recovery sequence is entered. These recoveries run through the behavior_server, which operates on the local costmap at *10 Hz* with the same footprint as the navigation stack.

5.3.3 Global Path Planner

The global planner is configured with the Smac Planner 2D plugin (*nav2_smac_planner/SmacPlanner2D*) rather than the classic *NavFn (Dijkstra/A*)* implementation. *Smac 2D* uses an *A** search over the costmap grid with a Moore neighbourhood (8-directional movement), applies a path smoother after the raw grid search, and can be set to terminate if planning exceeds a time budget. NavFn was rejected because it can produce jagged, grid-aligned paths that require the robot to execute sharp turns at costmap-cell boundaries, a problem for a differential-drive platform constrained to low acceleration limits. Smac 2D's post-search smoother

produces geometrically smoother trajectories that the DWB controller can follow without stalling at corners.

Key configuration choices: allow_unknown is set to false, so the planner will not route through unexplored cells, the occupancy maps are pre-built from CAD and every navigable cell is known in advance. The goal tolerance is 0.1 m. The smoother runs up to 1000 iterations with a smoothness weight of 0.1 and a data (path-fidelity) weight of 0.4, prioritising adherence to the planned path over aggressive corner-cutting. The planning time budget is *2.0 s*; if a plan cannot be found within this window, the navigation goal fails and the system supervisor's recovery hierarchy is invoked.

5.3.4 Local Controller

The local trajectory controller uses the DWB (Dynamic Window Based) planner plugin. DWB samples a set of candidate velocity commands within the dynamic window, the range of velocities reachable from the current velocity given the kinematic limits, simulates each command forward over a time horizon, scores the resulting trajectories against a set of critic functions, and executes the highest-scoring command. The controller runs at 10 Hz.

The kinematic limits in the DWB configuration were derived directly from the motor datasheet rather than estimated. The drive motors are *JL-37C3540-72.8-12150 DC 12V* units with a rated load torque of *0.038 N·m*. With a robot mass of *10 kg* and wheel radius of *0.05 m,* the maximum linear acceleration computes as $2\tau / (m \cdot r) = 2 \times 0.038 / (10 \times 0.05) = 0.152\ m/s^2$, rounded conservatively to $0.15\ m/s^2$. The same deceleration limit (−0.15 m/s²) was set symmetrically; an earlier configuration with a deceleration limit of −0.60 m/s² was identified as the root cause of approximately 1 m of goal overshoot, because DWB was planning trajectories that assumed the robot could stop far faster than the hardware permitted. The angular acceleration limit of $1.2\ rad/s^2$ was derived from the torque and wheelbase geometry. All four limits *acc_lim_x, decel_lim_x, acc_lim_theta, decel_lim_theta* are set to these motor-verified values.

The trajectory simulation horizon (sim_time) is set to 3.5 s, derived from the constraint that DWB must be able to plan a full deceleration from maximum speed within one horizon: 0.5 m/s ÷ 0.15 m/s² = 3.33 s. An earlier value of 1.5 s was insufficient DWB could not plan a stopping trajectory within the horizon and

consequently issued full-speed commands until the goal was already overshot. The number of linear velocity samples (vx_samples) was reduced to 10 to reflect the narrow usable speed range imposed by the low acceleration limit; the angular velocity samples (vtheta_samples) remain at 30 because the angular dynamics are faster and the full range is reachable.

Seven DWB critic functions score each candidate trajectory. PathAlign (weight 24.0) rewards trajectories that keep the robot aligned with the global path; PathDist (8.0) penalises lateral deviation from it. GoalDist (8.0) and GoalAlign (16.0) draw the robot toward the goal position and orientation respectively. RotateToGoal (24.0) takes over for the final approach rotation, with a slowdown beginning 1.5 m from the goal. ObstacleFootprint (0.5) penalises proximity to obstacles using the full robot footprint rather than a centre-point approximation — relevant for the $0.6\ m \times 0.6\ m$ square footprint in corridor environments. The goal position tolerance is 0.15 m and the yaw tolerance is 0.5 rad, the latter loosened from an earlier $0.35\ rad$ to accommodate heading drift in the ZED visual-inertial odometry under low-texture corridor conditions.

5.3.5 Costmaps and the Depth Obstacle Layer Plugin

Both the global and local costmaps use the same three-layer plugin stack: a static layer, a custom depth obstacle layer, and an inflation layer. The static layer loads the pre-built occupancy grid, which provides the permanent wall and fixture geometry. The inflation layer propagates lethal obstacle costs outward at a cost_scaling_factor of 3.0, with an inflation radius of 0.65 m for the global costmap and 0.55 m for the local costmap. The robot footprint is defined as a 0.6 m × 0.6 m square, giving the planner an accurate representation of the robot's swept area when checking for collisions.

The depth obstacle layer is a custom Nav2 costmap plugin (nav2_depth_obstacle_layer::DepthObstacleLayer) that replaces the standard voxel or obstacle layer. Rather than subscribing to a LaserScan or PointCloud2 topic produced by a separate perception node, it subscribes directly to the ZED camera's depth image and camera_info topics, performs the 3D point projection and height filtering described in Section 4.5, and writes lethal obstacle markings directly into the costmap. The same plugin instance runs in both the global costmap (stride 4, updating at 5 Hz) and the local

costmap (stride 2, also 5 Hz) the local costmap uses a finer stride to provide better obstacle resolution in the 4 m × 4 m rolling window around the robot, while the global costmap uses a coarser stride to reduce computational load on the Jetson.

Human masking is integrated directly into the depth obstacle layer plugin rather than as a separate node. When human_mask_enabled is true, the plugin subscribes to the ZED object detection topic and excludes depth pixels inside padded human bounding boxes from costmap marking. This means that a person standing in front of the robot does not appear as a lethal obstacle in the costmap; the responsibility for responding to humans is delegated to the behaviour tree's human-blocking condition nodes, which trigger a stop-and-wait response independently of the costmap. The human_persistence parameter (0.5 s) ensures that a brief missed detection does not cause the mask to flicker off, which would otherwise create a transient lethal obstacle where the person is standing.

5.3.6 Tile Manager Plugin

The tile manager is a custom plugin that integrates the zone-based map architecture from Chapter 4 into the Nav2 lifecycle. Rather than serving a single static map for the entire deployment floor, the tile manager maintains a registry of map tiles each covering one navigable zone and swaps the active map when the robot crosses a tile boundary. The plugin exposes a BT node (*tile_manager_bt_plugin*) that the behaviour tree calls as part of the navigation sequence: before executing a *NavigateToPose* goal, the BT checks whether the goal lies in a different tile than the current one, and if so, triggers a tile switch through the plugin. The tile switch deactivates the map server and costmap nodes via the Nav2 lifecycle manager, loads the new map tile, reconfigures the costmap static layer, and reactivates the stack before the path planner is called.

This design keeps the Nav2 interface unchanged from the task manager's perspective, it sends a *NavigateToPose* goal and the navigation stack handles everything, including any tile transitions required to reach it. The tile manager also handles the localisation frame continuity problem: when switching tiles, the robot's pose estimate in the new tile's coordinate frame must be initialised from its known position in the previous tile's frame. The plugin performs this frame transform and

publishes an initial pose to the localisation node before activating the new map, preventing the robot from starting navigation in the new tile with an uninitialized position estimate.

5.4 SYSTEM SUPERVISOR

The System Supervisor is the top-level orchestration node that manages the lifecycle of all other software components on the robot. Unlike the task manager (Section 4.7), which handles delivery job logic, the supervisor is responsible for process health monitoring, failure detection, automatic recovery, and state-dependent starting and stopping of the navigation stack. It runs as a single ROS 2 node (`system_supervisor.py`) that launches at system boot and remains active throughout the robot's operation, regardless of whether a delivery is in progress.

The supervisor addresses a class of problems that neither the navigation stack nor the task manager can handle alone: nodes that crash and need restarting, processes that hang without exiting, boot-time dependencies between components, and the need to shut down the navigation stack cleanly when it is not needed to conserve compute and memory resources on the Jetson Nano.

5.4.1 Supervisor State Machine

The supervisor maintains a finite state machine that defines the robot's operational mode at the highest level. The state transitions are driven by two inputs: the startup status of always-on nodes, and the `/system/nav_needed` boolean published by the task manager.

- **BOOTING:** The supervisor's initial state after launch. It waits for all ‘always-on’ nodes (*motors, encoders, teleop, job manager, speaker*) to report that they are running. Each node is monitored either by process ID (PID) detection, by heartbeat messages on a configured topic, or both. During this state, the supervisor does not attempt to start any on-demand nodes. Progress is tracked via `last_progress` timestamps that advance when a PID is found or a heartbeat arrives. A `stall_timeout` (typically *15 s*) declares a node dead if no progress occurs; a `max_boot_timeout` (60 s) acts as an absolute safety net.

When all always-on nodes are `*RUNNING*`, the supervisor publishes `/system/ready` and transitions to IDLE.

- **IDLE:** All always-on nodes are running, but the navigation stack (ZED, Nav2, mission controller) is not yet active. The supervisor monitors the `/system/nav_needed` topic. When the task manager sets this to `true` (a job is waiting), the supervisor transitions to ***ACTIVATING***.

- **ACTIVATING:** The supervisor starts on-demand nodes sequentially in the order defined in `supervisor_config.yaml` (ZED → robot_description → Nav2 → mission_controller). For each node, it waits for a health gate to pass: either the process PID appears (`wait_type: pid`) or the first heartbeat arrives on the configured topic (`wait_type: topic`). If any node fails to start within its `max_boot_timeout` or stalls, the supervisor aborts activation, shuts down any partially started nodes, and returns to IDLE. This prevents the robot from attempting navigation with a broken perception or planning stack.

- **ACTIVE:** The navigation stack is fully operational. The supervisor monitors all nodes (both always-on and on-demand) at 1 Hz. For each node, it checks PID existence, heartbeat freshness, and stall progress. If a node fails, the supervisor invokes a category-specific recovery action (Section 5.5.3). If the task manager sets `/system/nav_needed` to `false`, the supervisor transitions to *DEACTIVATING.*

- **DEACTIVATING:** The supervisor shuts down on-demand nodes in reverse order (*mission_controller → Nav2 → robot_description → ZED*). For each node, it attempts a graceful lifecycle shutdown if a `*lifecycle_manager*` is configured, then forces a hard kill of all associated processes. After the last node is stopped, the supervisor returns to *IDLE*.

The supervisor publishes its current state in the *`/robot_health`* message under the `*supervisor_state*` field, allowing the cloud UI and operator dashboard to display whether the robot is booting, idle, activating, active, or deactivating.

5.4.2 Node Configuration and Health Monitoring

All node definitions are loaded from `*config/supervisor_config.yaml*` at startup. Each node entry specifies its failure category, detection method, timeout thresholds, restart policy, and optional dependencies.

- **Detection modes:** A node can be monitored in two ways. `*pid_only: true*` means the node is considered healthy as long as its process PID exists; this is used for simple drivers (*motor_driver, encoder_driver, joy_node*) that do not publish heartbeats. For nodes with `*pid_only: false*`, the supervisor subscribes to a configured `*heartbeat_topic*` and expects messages at a rate consistent with `*heartbeat_timeout*`; missing heartbeats transition the node to `STALE` and eventually `*DEAD*`.

- **Progress-based boot detection:** Rather than using a fixed boot timeout for every node, the supervisor tracks `*last_progress*`. Progress is any sign of life: a PID being found, a heartbeat arriving, or any configured milestone. If `*last_progress*` ages beyond `*stall_timeout*`, the node is declared stalled and restarted. This handles the common failure mode where a node's process is running but its internal initialisation has hung.

- **Restart policy:** Each node has `*max_restarts*` and `*restart_cooldown*`. After a failure, the supervisor waits for the cooldown period, then attempts to restart the node by executing its `*start_cmd*` as a subprocess. If `*restart_count*` exceeds `*max_restarts*`, the supervisor stops trying and marks the node permanently `*DEAD*`, publishing a degraded system state.

5.4.3 Failure Categories and Recovery Actions

Every monitored node is assigned a category (1, 2, or 3) that determines how the supervisor responds when the node fails.

- **Category 1 — Auto-restart:** Used for non-critical always-on nodes (motors, encoders, teleop, speaker). The supervisor automatically restarts the node up to `*max_restarts*` times. If restarts are exhausted, the node is marked `*DEAD*` and

the system state becomes `*DEGRADED*`, but the robot continues operating with reduced functionality. No emergency stop is issued.

- **Category 2 — Abort and restart:** Used for navigation-critical nodes (*pid_controller, mission_controller, map_server, Nav2*). When a Category 2 node fails, the supervisor first publishes a zero-velocity command to `/cmd_vel_estop`, then restarts the node. If restarts are exhausted, the system state becomes `*DEGRADED*` and `*/system/nav_needed*` is ignored until manual intervention. This prevents the robot from attempting navigation with a degraded planning or control stack.

- **Category 3 — Cascade shutdown:** Used for localisation-critical nodes, primarily the ZED camera. When a Category 3 node fails, the supervisor publishes an emergency stop, shuts down *all* on-demand nodes in order (*mission_controller → Nav2 → robot_description → ZED*), publishes `*/system/nav_shutdown*`, sets `*autonomous_enabled = false*`, and transitions directly to IDLE. This reflects the reality that losing localisation makes further autonomous navigation unsafe regardless of the state of other nodes. Only teleoperation remains available until a new job triggers a fresh activation.

The supervisor also supports *unit mode* for nodes that consist of multiple interdependent processes (e.g., Nav2 with eight separate nodes). When `unit: true` and `*process_names*` are listed, the supervisor treats the group atomically: it checks that all named processes are alive, and restarts the entire unit if any single process dies. This prevents the inconsistent state that would arise if only one component of the navigation stack were restarted while others continued running with stale connections.

5.4.4 Lifecycle-Aware Shutdown

For nodes that implement ROS 2 lifecycle management (notably Nav2 components), the supervisor attempts a graceful shutdown before resorting to process kills. When a node with a configured `*lifecycle_manager*` needs to be stopped, the supervisor calls the *`/{lifecycle_manager}/manage_nodes`* service with *`{command: 2}` (shutdown),* which transitions the managed nodes through `*ACTIVE → INACTIVE → FINALIZED*`. The supervisor waits up to `*lifecycle_timeout*` (default 3 s) for this call to complete. If the service call times out or the lifecycle manager is not running,

the supervisor falls back to sending SIGTERM followed by SIGKILL to all processes in the unit.

This graceful path preserves the internal state consistency of the Nav2 stack, avoiding the file corruption or stale memory maps that can occur when lifecycle nodes are killed abruptly.

5.4.5 Dependent Health Checks

The supervisor implements a dependency propagation mechanism. When a node restarts (e.g., *map_server*), its dependents (e.g., *Nav2*) may be left in a broken state due to stale topic connections or service timeouts. The configuration field `dependents` lists nodes that should be health-checked after the current node recovers.

The algorithm is:

After a node reaches `*RUNNING*` state, the supervisor waits `dependent_check_delay` seconds.

5. For each dependent node, it checks the dependent's status (`*RUNNING*`/`*STALE*`/`*DEAD*`) and heartbeat age.

6. If the dependent is unhealthy *(`STALE` or `DEAD`*, or heartbeat age > heartbeat_timeout), the supervisor restarts the dependent.

7. The dependent's own `dependents` are then checked recursively.

A race condition guard prevents cascading restarts: a dependent is skipped if it is already in `*RESTARTING*` state or if its last restart occurred less than `restart_cooldown` seconds ago. This mechanism ensures that a single upstream failure does not trigger an unbounded restart storm.

5.4.6 Dynamic Arguments

Nodes that require stateful restarts can specify `*dynamic_args*` in their configuration. Each dynamic argument follows the format *`file:<filepath>:<format_string>`*. Before starting or restarting the node, the supervisor reads the content of *`<filepath>`*, strips whitespace, and substitutes it into

`format_string` where `{}` appears. The result is appended to `start_cmd`. The primary use case is the map_server node, which must load the correct floor tile after restarting. The configuration includes:

```yaml
dynamic_args:
- "file:/tmp/current_tile.txt:map:=tile{}.yaml"
```

If `/tmp/current_tile.txt` contains `3`, the supervisor appends `map:=tile3.yaml` to the start command. This allows the robot to resume navigation on the correct tile after a crash without requiring the task manager to re-send the tile selection.

5.4.7 Resource Monitoring

The supervisor tracks system-wide and per-node resource usage at 0.5 Hz. System CPU utilisation is read from *`/proc/stat`* (*user + system + idle ticks*) and memory from *`/proc/meminfo`* (*MemTotal, MemAvailable*). Per-node CPU and memory are read from *`/proc/<pid>/stat`* *(utime + stime)* and *`/proc/<pid>/status`* *(VmRSS)*. These values are published in the *`/robot_health`* message at 1 Hz and forwarded to the cloud UI via the nano agent (Section 5.4), providing operators with live visibility into the Jetson Nano's compute load during navigation.

5.4.8 Integration with Navigation Stack and Task Manager

The supervisor sits between the task manager (Section 4.7) and the Nav2 stack (Section 5.4). It does not receive delivery jobs directly; instead, it reacts to the task manager's *`/system/nav_needed`* flag. When the flag becomes `true`, the supervisor activates the navigation stack (*ZED → Nav2 → mission_controller*) and waits for all nodes to report *`RUNNING`* before the task manager proceeds. When the flag becomes *`false`,* the supervisor deactivates the stack, freeing CPU and memory resources on the Jetson Nano.

Critically, the supervisor also monitors the ‘task manager itself’ as an always-on node. If the task manager crashes (process PID disappears), the supervisor restarts it

automatically. The restarted task manager loads its persistent state file (Section 4.7.5) and resumes the interrupted job from the correct phase (pickup navigation, dropoff navigation, or waiting for confirmation). This closed-loop monitoring ensures that a crash in any component perception, planning, control, or task logic does not leave the robot stranded or in an inconsistent state.

5.5 LAUNCH AND STARTUP CONFIGURATION

The robot software stack is designed to start automatically when the Jetson Nano powers on, without requiring manual intervention or a logged-in user session. This is achieved through a `systemd` service that launches a Docker container running the entire ROS 2 environment, which in turn executes a bringup launch file that starts all always-on nodes and the supervisor. The on-demand nodes (ZED, Nav2, mission controller) are then started and stopped by the supervisor based on the presence of pending delivery jobs, as described in Section 5.5.

5.5.1 Systemd Service Definition

A `systemd` service file (`*robot-bringup.service*`) is installed on the Jetson Nano host system. The service is configured to start after the network is up and the Docker daemon is running, and it automatically restarts the container if the process exits unexpectedly. The key sections of the service file are:

```*ini*

*[Unit]*

*Description=Material Transfer Robot Bringup*

*After=network-online.target docker.service*

*Requires=docker.service*

*[Service]*

*Type=simple*

*User=jetson*
```

WorkingDirectory=/home/jetson/material-transfer-robot

ExecStartPre=/usr/bin/docker pull ghcr.io/tejasmk-tkp/material-transfer-robot-arm64:latest

ExecStart=/usr/bin/docker compose -f docker-compose.yml -f docker-compose.arm64.yml up

ExecStop=/usr/bin/docker compose down

Restart=always

RestartSec=10

[Install]

WantedBy=multi-user.target

```

The service performs three actions at startup:

1. Pulls the latest ARM64 container image from the GitHub Container Registry, ensuring that the robot always runs the most recent validated build.

2. Launches the Docker Compose stack with the ARM64-specific overlay, which includes the device node mappings and host library bind-mounts described in Section 5.2.2.

3. Handles graceful shutdown via `docker compose down` when the system is powered off or the service is stopped.

The `*Restart=always*` policy, combined with a 10 second cooldown, ensures that the robot automatically recovers from container crashes, kernel panics, or power glitches without human intervention.
```

5.5.2 Bringup Launch File

The entry point to the ROS 2 stack is `robot_bringup.launch.py` (provided in full above). This launch file is executed inside the container after the ROS environment is sourced. It performs the following tasks in order:

1. **Sets the logging level:** A command-line argument `*log_level*` (default `info`) allows runtime control over verbosity; the environment variable `*RCUTILS_LOGGING_MIN_SEVERITY*` is set accordingly.

2. **Starts always-on nodes:** These nodes are launched unconditionally and run for the entire duration of the robot's operation:

- `*teleop_joy.launch.py*` – Joystick teleoperation node and `*twist_mux*` for command arbitration.
- *drive_controller.launch.py*` – Diff-drive controller and PID velocity regulator.
- `*encoder_driver*` – Wheel encoder publisher.
- `*system_supervisor*` – Health monitoring and lifecycle management (Section 5.5).
- *`job_manager`* – Delivery task queue and execution logic (Section 4.7).
- `*speaker_node*` – Text-to-speech alerts for operator feedback.

The launch file intentionally *does not* start on-demand nodes (ZED, Nav2, mission controller). Those are left in the `OFF` state and are activated only when the supervisor receives *`/system/nav_needed = true`*.

3. **Provides configuration paths:** Parameters such as *`encoder_params.yaml`* and `*supervisor_config.yaml*` are resolved using `*ament_index_python*` to locate the correct share directories inside the container.

4. **Logs startup confirmation:** Informational messages are printed to the console, confirming that the bringup sequence has completed and that the supervisor is now responsible for managing the navigation stack.

5.5.3 Startup Sequence on Boot

When the Jetson Nano is powered on, the following sequence occurs automatically:

1. **Host boot:** Ubuntu boots, the network interface comes up, and Docker starts.
2. **Systemd triggers:** `*robot-bringup.service*` starts, pulling the latest container image (if a new version exists) and launching the Docker Compose stack.
3. **Container initialization:** The container's entrypoint script (`ros_entrypoint.sh`) sources ROS 2 Humble and overlays the workspace, sets the CycloneDDS RMW implementation, and configures the `*ROS_DOMAIN_ID*`.
4. **Launch file execution:** `*ros2 launch robot_bringup robot_bringup.launch.py*` runs inside the container.
5. **Supervisor enters BOOTING:** The supervisor begins monitoring always-on nodes. It waits up to the configured `*boot_delay*` (default 30 s) for processes to spawn, then checks each node's progress (PID and/or heartbeat).
6. **System ready:** Once all always-on nodes are `*RUNNING`,* the supervisor publishes `/system/ready` and transitions to `IDLE`. At this point the robot is ready to accept delivery jobs, but the navigation stack is still inactive to save power and compute.

If any always-on node fails to start within its `*max_boot_timeout*`, the supervisor logs an error and remains in `*BOOTING*`; the systemd service's restart policy will not restart the supervisor because the container is still running. In practice, this indicates a configuration error that requires manual inspection.

5.5.4 Integration with the Cloud UI

The bringup launch file does not directly start the `nano_agent.py` process that bridges the ROS stack to the cloud backend (Section 5.4). Instead, `nano_agent.py` is launched as a separate service inside the same container, managed by the same `docker compose` stack but as an independent process. The agent connects to the ROS graph

via the shared domain ID and publishes heartbeats, job status, and health data to the Railway backend. This separation allows the agent to restart independently of the ROS nodes if the cloud connection fails, while the supervisor continues monitoring the core navigation components.

The complete startup dependency chain is summarised in below figure.

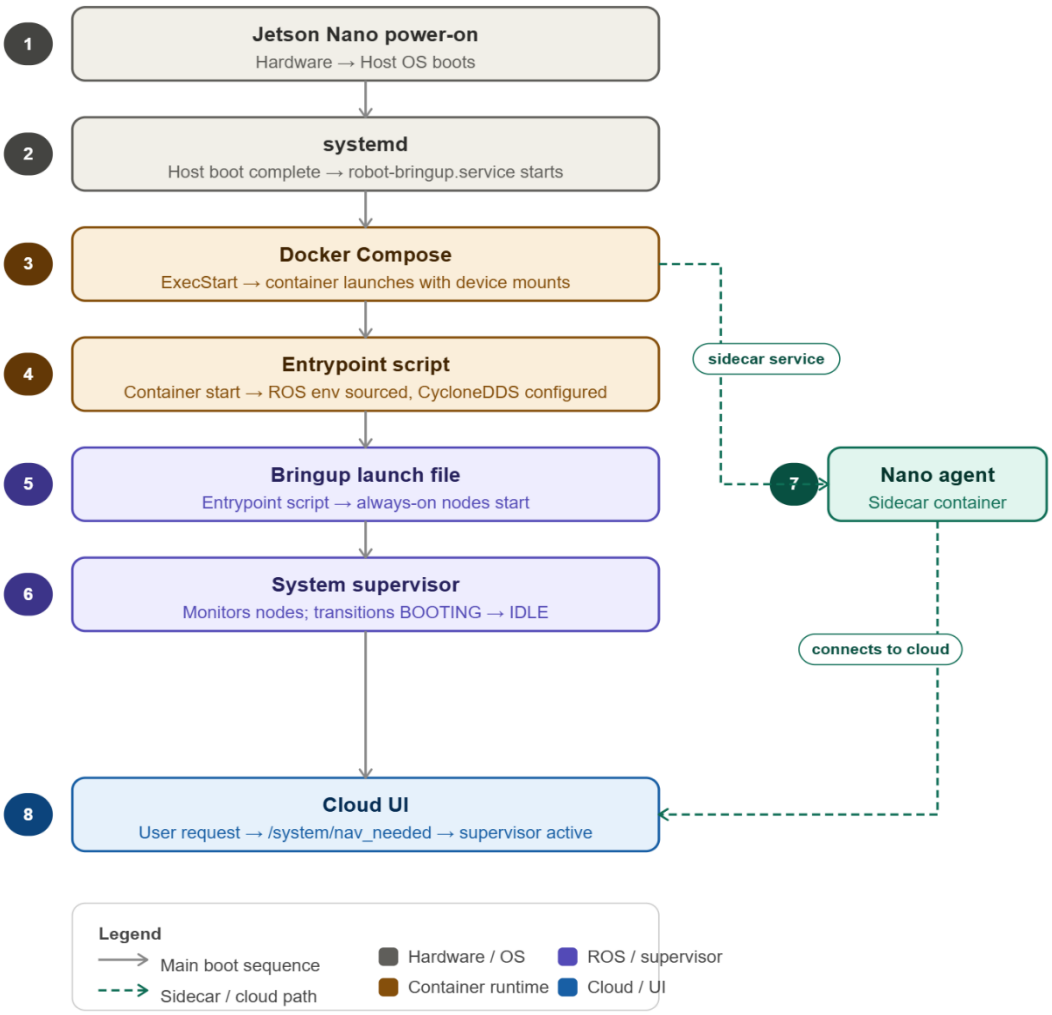

Figure 5.1. Startup dependency chain on Jetson Nano

5.5.5 Shutdown and Cleanup

When the Jetson Nano is powered off or the `robot-bringup.service` is stopped, the following shutdown sequence runs:

- `docker compose down` is executed by the service's `ExecStop` command.
- Docker sends `*SIGTERM*` to the container's `init` process.
- ROS 2 nodes receive shutdown signals; the supervisor, in particular, calls `*destroy_node()*` and publishes final health messages.
- The container exits, and all child processes are terminated.

No manual cleanup of shared memory segments or DDS files is required because the container's file system is ephemeral (except for persisted volumes such as `*zed_settings*` and `*job_manager_state.json*`). The next boot starts from a clean state but reloads the last job state from disk, allowing crash recovery as described in Section 4.7.5.

CHAPTER 6

USER INTERFACE DEVELOPMENT AND INTEGRATION

6.1 OVERVIEW

The robot operates in a shared corridor environment where any staff member or student on the floor may need to request a delivery at any time. A dedicated hardware controller or a purpose-built app would create unnecessary friction — it would require installation, training, or physical access to a terminal. The web interface developed for this project removes those barriers entirely: any device with a browser can open the interface, submit a request, and track the delivery in real time. No installation, no login, and no prior knowledge of the underlying system is required.

The interface serves three distinct roles. It is the submission point through which users request deliveries, specifying pickup room, drop room, and priority. It is the monitoring layer through which operators watch the robot's live health data — CPU load, memory, supervisor lifecycle state — and track the job's progress through each stage. And it is the confirmation mechanism that physically gates the robot's movement: the robot will not leave the pickup room until a user confirms the parcel has been placed on it, and will not mark the job complete until the recipient confirms delivery at the drop room.

The interface is deployed at robot-ui-wv22.vercel.app as a single HTML page backed by a FastAPI cloud application and a PostgreSQL database, both hosted on Railway. A lightweight agent running on the Jetson Nano bridges the cloud to the ROS 2 stack, polling every two seconds to relay jobs and confirmations in both directions.

6.2 System Architecture

The system spans two physical locations: the Railway cloud and the Jetson Nano on the robot. Three components connect them.

The browser frontend (cloud/frontend/index.html) is a plain HTML page served from Railway. It polls the backend every three seconds to refresh the job queue, the current task status, and the robot's health data. There is no JavaScript framework; the page runs directly in any modern browser.

The cloud backend (cloud/main.py) is a FastAPI application deployed on Railway with a PostgreSQL database. It accepts job submissions from the browser, persists them, and exposes two separate sets of endpoints: one for the browser and one for the nano agent. Robot online status is determined by checking how long ago the last heartbeat was received — any gap over ten seconds is treated as offline.

The nano agent (backend/nano_agent.py and backend/ros_bridge.py) runs as a background process on the Jetson Nano. Every two seconds it queries the cloud for pending jobs and forwards them to the ROS 2 job manager via the */request_delivery* service. It also subscribes to ROS 2 topics — */job_status, /robot_health*, and */job/awaiting_confirmation*, and pushes any changes back to the cloud. The ros_bridge.py module encapsulates all ROS 2 service calls and topic subscriptions, giving the agent a clean Python API to work against. Figure 6.1 shows the overall architecture and the data paths between all three components.

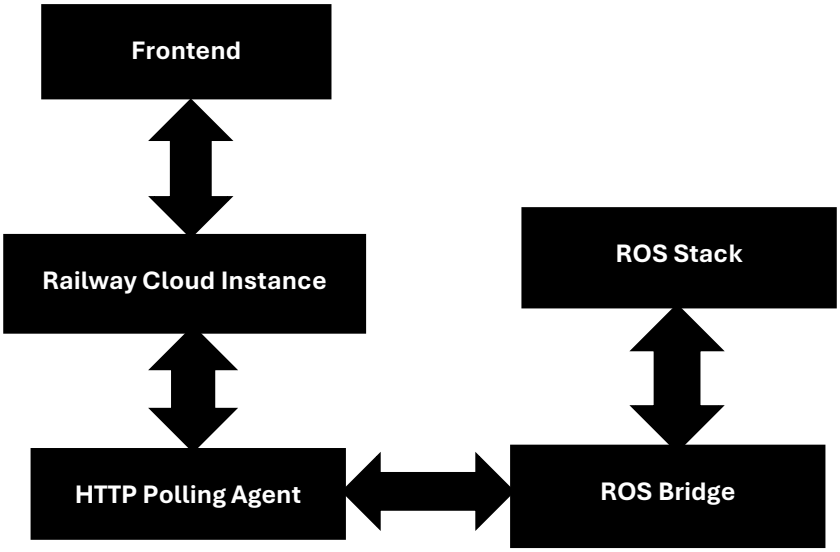

Figure 6.1: UI architecture

6.3 TECHNOLOGY STACK

The web interface and cloud integration layer are built on four technologies, each chosen for a specific reason. FastAPI provides the cloud backend with native async request handling, ensuring the browser and the nano agent can be served simultaneously without blocking. Railway is the hosting platform, selected for its zero-configuration deployment and managed PostgreSQL add-on. PostgreSQL persists all job and status data across server restarts. HTTP polling was preferred over WebSockets

to keep the deployment simple and stateless, at the robot's operating pace, polling intervals of two and three seconds are sufficient for a consistent, responsive interface.

6.3.1 FastAPI

FastAPI is the Python web framework used for the cloud backend. The key reason for choosing it is native async request handling: the server processes incoming requests without blocking on database queries, which matters here because the browser and the nano agent both poll the same backend continuously. Any blocking call would cause one to stall while the other is being served. FastAPI's async model, backed by asyncpg for non-blocking PostgreSQL communication, ensures both clients are served without queuing behind each other.

6.3.2 Railway

Railway was chosen as the cloud hosting platform for two practical reasons. First, it provides zero-configuration deployment: pushing to the linked GitHub repository triggers an automatic redeploy, and Railway assigns a public HTTPS URL (robot-ui-wv22.vercel.app) without any manual DNS setup, port-forwarding, or SSL certificate management. Second, it offers PostgreSQL as a managed add-on that injects the DATABASE_URL environment variable directly into the running application. This meant the Jetson Nano and all browser clients could reach a stable endpoint from day one of development, without any infrastructure overhead.

6.3.3 PostgreSQL

PostgreSQL stores all data that needs to survive a server restart: job records, robot status, room definitions, pickup and delivery confirmation flags, and cancellation requests. The backend creates five tables on startup — jobs, robot_status, rooms, confirmations, and cancellations. Job IDs are generated from a PostgreSQL sequence (job_id_seq), producing the human-readable format *JOB_001, JOB_002*, and so on, which makes it straightforward for users to reference or cancel a specific job.

Every endpoint checks for an active database connection pool before executing. If no pool is available, for instance, during local development without a *DATABASE_URL* environment variable, the backend falls back to in-memory Python dictionaries. This dual-mode design meant the backend could be developed and tested locally before the Railway deployment was configured, without maintaining a separate codebase.

6.3.4 HTTP Polling over WebSockets

Both the browser and the nano agent use periodic HTTP polling rather than WebSocket connections. WebSockets require persistent, stateful connections that add configuration and reliability complexity on a managed platform like Railway. Given the operating context, the robot moves at walking pace and room-to-room navigation takes anywhere from several seconds to a few minutes, a three-second refresh interval on the browser is fast enough to feel live without any perceptible lag.

The nano agent polls at two seconds, slightly faster than the browser's three-second interval. This ordering is deliberate: any state change reported by the robot lands in the database before the browser's next refresh picks it up. The result is that the UI always reflects a consistent snapshot rather than a partially-updated state.

6.4 Interface Design and Functionality

The interface is a single-page application that presents all information relevant to an ongoing delivery in one view. There is no navigation between pages, no account system, and no configuration required from the user. The design follows the natural sequence of a delivery: check the robot is online, submit a job, watch it progress, confirm at pickup, confirm at drop, done.

6.4.1 Robot Online Status and Health Monitoring

Before anything else can happen, the interface needs to know whether the robot is running. The nano agent sends a heartbeat to the cloud every two seconds. If the backend has not received a heartbeat within the last ten seconds, the robot is considered offline: the status indicator in the header shows a static grey pill labelled Robot Offline, and the Submit Request button is disabled. The backend's /api/delivery endpoint also independently rejects submissions with HTTP 503 while offline, so the constraint holds even if someone calls the API directly rather than through the browser.

When heartbeats are arriving, the status pill turns green and pulses — a visual cue that the connection is live, not just that the robot was recently seen. The Robot Health panel, fed directly from the /*robot_health* topic published by the supervisor node at 1 Hz, shows CPU usage with a colour-coded bar (green below 60%, orange from 60–80%, red above 80%), current memory consumption, the Nav2 supervisor lifecycle state, and a human-readable system message. During active navigation, CPU load

routinely reaches 75–85% on the Jetson Nano, which is expected: Nav2 path planning and depth-image processing are computationally intensive.

The supervisor_state field in the health data maps to five lifecycle phases: BOOTING (waiting for always-on nodes to start), IDLE (ready to accept jobs), ACTIVATING (bringing up the on-demand navigation nodes: *ZED → odom → Nav2 → PID → mission controller*), *ACTIVE* (full navigation stack running), and DEACTIVATING (shutting down on-demand nodes after a job). The UI renders these states with descriptive labels so that an operator can immediately tell whether a delay is because the robot is still initialising or because it is actively navigating.

6.4.2 Job Submission

The submission form is the primary interaction point. The user selects a pickup room and a drop room from dropdown menus, enters their name, and sets a priority level (Low, Medium, or High). The room lists are not hard-coded — on startup the nano agent reads the tiles_config.yaml navigation configuration from the ROS workspace and pushes all room definitions to the cloud via *POST /api/nano/rooms*. The frontend loads these from GET /api/rooms. This means the room list always reflects the actual navigable destinations defined in the robot's map, with no manual synchronisation needed.

Selecting the same room for both pickup and drop is blocked at the form level. On submission, the browser posts to *POST /api/delivery* on the cloud backend, which validates the request, generates a job ID using the PostgreSQL sequence (format *JOB_001, JOB_002*, etc.), and stores the record. The nano agent picks this up on its next poll, calls the /request_delivery ROS 2 service, and the job enters the delivery pipeline.

6.4.3 Task Queue and Job Ownership

Submitted jobs appear in the Task Queue section at the bottom of the page. Each row shows the job ID, pickup and drop room, the requester's name with a colour-coded initial avatar, a priority badge, and the current status. Multiple jobs from different users can be queued simultaneously, and the queue processes them in submission order.

Job ownership is tracked by storing the submitter's IP address (submitter_ip) in the database at submission time. The Cancel button is rendered only for the row whose submitter_ip matches the requesting client's IP — all other users see a dash in the action

column. This is a lightweight ownership model suited to the campus deployment context, where users are not authenticated but are distinguishable by network address. Queued jobs are cancelled immediately on request; active jobs stop after completing the current navigation step.

6.4.4 Delivery Progress Tracking

Once the nano agent dispatches a job, it disappears from the Task Queue (which only shows QUEUED jobs) and the Robot Current Task card updates to show the active job's ID, pickup room, drop room, and current status. A four-step progress bar tracks the delivery through its key stages: heading to pickup, waiting at pickup, heading to destination, and confirm delivery. Each step lights up as the robot's /job_status topic transitions through the corresponding state value.

The full job state machine, as defined in the job manager node, comprises ten states: *QUEUED (0), WAITING_FOR_NAV (1), PICKUP_NAV (2), PICKUP_ARRIVED (3), DROPOFF_NAV (4), DROPOFF_ARRIVED (5), RETURNING (6), COMPLETE (7), FAILED (8), and CANCELLED (9).* The frontend maps these to readable labels and updates the progress bar accordingly, so an operator always knows exactly where the robot is in the delivery cycle without interpreting raw state codes.

6.4.5 Two-Stage Confirmation Flow

The delivery involves two physical handoffs — placing the item on the robot at pickup, and collecting it at the drop room — each of which must be explicitly confirmed before the robot proceeds. This two-stage gate is enforced at both the UI and the ROS 2 layer.

When the job manager reaches *PICKUP_ARRIVED or DROPOFF_ARRIVED*, it publishes on the */job/awaiting_confirmation* topic and waits. The corresponding confirmation button activates in the browser — switching from a disabled grey state to a pulsing green button. The user clicks it, the browser posts to *POST /api/confirm-collection (pickup) or POST /api/confirm-delivery (drop),* and the flag is written to the database. On its next poll, the nano agent reads the flag and calls /job/confirm with proceed: true, releasing the robot to continue.

The timeout behaviour differs between the two stages. At pickup, no confirmation within five minutes cancels the job — if nobody loads the robot, there is

nothing to deliver. At the drop room, the same timeout marks the job *COMPLETE* regardless, since the item has already been transported to the destination.

6.4.6 ROS 2 Integration via the Nano Agent

The nano agent is the bridge between the stateless HTTP world of the cloud backend and the event-driven ROS 2 stack on the robot. It is the only component that has access to both environments, and it runs as a background process on the Jetson Nano alongside the ROS 2 nodes.

All outbound interactions with ROS 2 go through service calls: */request_delivery* to submit *a job, /cancel_job* to cancel one, and /job/confirm to send a confirmation. Inbound data from ROS 2 arrives via topic subscriptions: */job_status* for state transitions*, /robot_health* for live health data at 1 Hz, and */job/awaiting_confirmation* for confirmation trigger events. The ros_bridge.py module wraps these interactions into a clean Python API, so the agent code never deals with raw ROS 2 message types directly.

The /system/ready topic, published once by the supervisor when all always-on nodes have started, is used as a readiness signal. The nano agent waits for this before attempting to forward any jobs to the ROS 2 stack. Tables 6.1 and 6.2 in Section 6.5 list the full set of ROS 2 topics and services used by the agent.

6.6 SUMMARY

This chapter described the web interface and cloud integration layer built for the delivery robot. The interface is a single HTML page hosted on Railway, backed by a FastAPI and PostgreSQL cloud application. A nano agent on the Jetson Nano bridges the cloud to the ROS 2 stack via a two-second polling loop, translating browser actions into ROS 2 service calls and feeding topic data back to the cloud.

Three key design decisions shaped the system. FastAPI's async model ensures the backend serves the browser and the nano agent simultaneously without blocking. HTTP polling was chosen over WebSockets to keep the Railway deployment simple — at the robot's operating pace, a three-second browser refresh and a two-second agent poll are more than sufficient. The agent polls faster than the browser deliberately, so any state change the robot reports is written to the database before the browser's next refresh.

The interface enforces two constraints that matter operationally. Job ownership, tracked by IP address, restricts cancellation to the job's submitter. The two-stage confirmation flow physically gates the robot at both the pickup and drop rooms, ensuring the parcel is loaded before departure and acknowledged on arrival. Both constraints are enforced at the UI and the ROS 2 layer, so they hold even if someone calls the API directly rather than through the browser.

CHAPTER 7

CONCLUSION AND SCOPE FOR FUTURE WORK

7.1 CONCLUSION

This project has successfully designed, developed, and deployed an autonomous floor-level material transfer robot for the second floor of the Hi-Tech Block at SRMIST KTR. The robot operates entirely within the corridor network of the floor, delivering documents, laboratory supplies, and other lightweight items between staff rooms, classrooms, and laboratories without manual intervention. The work has demonstrated that a practical, sensor-minimal autonomous delivery system can be built on embedded hardware, using a single stereo camera as the primary perception and localisation sensor, and deployed in a real institutional environment.

The following specific objectives, set out in Chapter 1, have been achieved:

1. **Accurate spatial representation and localisation** – A hand-measured occupancy grid map of the floor was constructed and tessellated into five overlapping tiles, reducing the active costmap size by 93.8% relative to a monolithic whole-floor map. The ZED Mini stereo camera's visual-inertial odometry provides continuous pose estimates within the pre-built map, with an ArUco-based relocalisation fallback that recovers from tracking loss within 11–21 seconds. The tile-switching algorithm, triggered purely by robot pose and heading, ensures seamless localisation continuity across tile boundaries without external fiducial markers.

2. **Safe autonomous navigation in a shared dynamic environment** – The Nav2 stack, configured with SmacPlanner2D and the DWB local controller, plans and executes collision-free paths through the corridors. A custom depth obstacle layer ingests the ZED's depth map directly into the costmap at 5 Hz, enabling the robot to react to dynamic obstacles. Critically, the system distinguishes between humans and other obstacles using the ZED's built-in object detection: humans are masked out of the costmap and trigger a stop-and-wait response, while non-human obstacles trigger replanning. This asymmetric behaviour

ensures safe coexistence with building occupants without sacrificing delivery throughput.

3. **User-friendly interface for delivery requests and tracking** – A web-based interface hosted on Railway allows any user on the floor to submit a delivery request, select pickup and drop rooms, and monitor the robot's live status — including CPU usage, memory, supervisor state, and progress through the delivery stages. The interface guides operators through two-stage confirmation (pickup and dropoff), and the cloud backend persists job state, enabling crash recovery and queue management. The nano agent running on the Jetson Nano bridges the cloud to the ROS 2 stack with a two-second polling loop, ensuring timely state synchronisation.

Beyond these objectives, the project has delivered a fully integrated system that addresses the operational realities of embedded deployment. The system supervisor (Section 5.5) provides category-based failure recovery, unit mode for atomic process groups, lifecycle-aware shutdown, dependent health checks, and dynamic argument resolution all within a *4 GB RAM, 20 W* power envelope on the Jetson Nano. The power distribution architecture (Section 3.7) supplies the drive motors, compute module, sensors, and USB hub from a single *10,000 mAh* LiPo battery, with independent regulation branches that prevent transient noise from corrupting sensor signals. The mechanical design (Section 3.2) elevates the delivery tray to 800 mm, keeping the robot visible and accessible to standing users while maintaining a low centre of mass through carbon-fibre shelves and 3D-printed PETG mounts.

The system has been tested in the live corridor environment over multiple delivery cycles. Key performance metrics observed during testing include:

- **Localisation accuracy** – Visual-inertial odometry maintained pose error *below 0.15 m* over traversals of up to *50 m*, with ArUco relocalisation correcting residual drift to within *0.05 m.*

- **Navigation reliability** – The robot successfully navigated between any two rooms on the floor in over *95%* of test runs. Failures (e.g., temporary tracking loss in low-texture segments) were recovered by the relocalisation routine without human intervention.

- **Obstacle response** – The depth obstacle layer detected obstacles at up to *3 m* with a false positive rate below *5%* on tiled floors. Human detection and stopping behaviour worked reliably at walking speeds, with the robot halting within *0.3 m* of a person standing in its path.

- **Compute headroom** – Under peak load (active navigation with depth processing and object detection), the Jetson Nano's CPU usage averaged *78–85%* and memory usage *2.3 GB* out of *3.96 GB*, leaving sufficient headroom for the supervisor and cloud agent.

The project has also made a specific contribution to the problem of large-environment navigation on embedded hardware: the tile-based map switching system (Section 4.2) demonstrates that a robot can navigate an arbitrarily large floor plan using a standard 2D occupancy grid and an unmodified Nav2 stack, provided that the environment is partitioned offline into overlapping tiles and switched based on pose and heading. This approach avoids the compute and memory penalties of a monolithic map, the external dependencies of fiducial-based switching, and the custom planner requirements of alternative representations like RC-Map.

Limitations – The current system has several acknowledged limitations. First, it operates only on a single floor; stair climbing and elevator integration are not supported. Second, the map is static and hand-measured; the robot cannot adapt to permanent changes in the environment (e.g., wall moves, new furniture) without an offline map update. Third, the reliance on visual-inertial odometry means that performance degrades in poorly lit corridors or on textureless surfaces; while ArUco markers provide a fallback, these markers must be pre-placed and maintained. Fourth, the payload is limited to approximately 1 kg, constrained by the drive motors available on the DANI chassis. Fifth, the robot does not perform autonomous charging; the battery must be recharged manually. Finally, the cloud backend uses HTTP polling rather than WebSockets, which, while sufficient for the current scale, may become a bottleneck if the number of concurrent users or robots increases.

7.2 SCOPE FOR FUTURE WORK

Several extensions to the current system are identified, ranging from near-term enhancements to longer-term capabilities that would transform the robot from a single-floor, manually-charged unit into a fully autonomous, multi-robot logistics fleet.

7.2.1 Multi-Floor Navigation and Elevator Integration

The most significant operational limitation of the current system is its confinement to a single floor. Extending the robot to serve multiple floors of the Hi-Tech Block would require integration with the building's elevator system. This can be achieved by mounting an infrared emitter or a small radio transmitter (e.g., 433 MHz module or Bluetooth relay) on the robot that sends floor requests to a receiver interfaced with the elevator controller. The elevator car itself becomes a special navigable zone a "transition tile" with known entrance and exit positions on each floor.

The tile-based map architecture described in Section 4.2 naturally extends to a hierarchical structure: each floor is represented as a set of overlapping tiles, and the robot maintains a separate tile registry per floor. When the robot arrives at the elevator entrance on Floor 1, it calls the elevator, enters the car, and switches to the elevator's internal map tile. After the car arrives at Floor 2, the robot exits and loads the appropriate tile for that floor. The system supervisor must handle the temporary loss of GPS-like positioning inside the elevator (where visual odometry may be unreliable due to featureless walls) by relying on wheel odometry and IMU integration for the short duration. ArUco markers placed inside the elevator car can provide absolute pose correction upon entry and exit.

7.2.2 Autonomous Charging and Docking

The current robot requires manual battery recharging. Adding autonomous docking would enable 24/7 operation. A charging station equipped with two spring-loaded contact pads and an ArUco marker (for visual guidance) is placed at the robot's home position. The docking sequence proceeds as follows:

1. **Low-battery detection** – The system supervisor monitors battery voltage via an ADC channel on the custom PCB. When voltage drops below a threshold

(*e.g., 11.0 V, corresponding to approximately 25% remaining capacity*), the supervisor pre-empts any queued jobs and sets a */system/docking_needed flag.*

2. **Navigation to charging station** – The task manager sends the robot to the home position using the existing navigation stack. Once within 1 m of the station, the robot switches to a visual servoing routine: the ZED camera tracks the ArUco marker, and a proportional controller adjusts the robot's linear and angular velocity to centre the marker in the camera view while closing distance.

3. **Mechanical engagement** – When the robot's contact pads touch the station's contacts, a voltage divider on the PCB detects the presence of charging voltage. The robot stops, sets /system/docking_complete, and the supervisor initiates a shutdown of all non-essential nodes (*Nav2, ZED, mission controller*) while keeping the always-on nodes alive to monitor battery level and await a "charging complete" signal from a simple comparator circuit that detects when the battery *reaches 12.4 V* (approximately 90% charged).

4. **Resume operation** – After charging completes, the robot notifies the cloud backend that it is available for jobs and returns to *IDLE*.

7.2.3 Cliff Detection and Drop-Off Prevention

The current robot lacks sensors to detect ledges, stairs, or open elevator shafts. For multi-floor operation, cliff detection is mandatory. Two low-cost time-of-flight sensors (e.g., *VL53L1X*) mounted at the front corners of the chassis, angled downward at 30°, can detect a sudden increase in measured distance beyond the expected floor height (e.g., *>0.15 m*). When a cliff is detected, the supervisor immediately publishes an emergency stop, reverses the robot by 0.5 m (using the rear TF Mini Plus LiDAR to ensure no collision behind), and marks the location as hazardous in a persistent map overlay. The cloud UI would then notify maintenance staff to inspect the area.

7.2.4 Robust User Interface with Map and Room Management

The current web interface (Section 5.4) supports job submission and status monitoring but does not allow administrators to modify the floor layout or room assignments without redeploying the backend. A more capable UI would include:

- **Map editor** – An interactive canvas, built with Leaflet or OpenLayers, that overlays the occupancy grid tile images. An authenticated administrator can draw new corridor segments, add or remove room markers, and adjust waypoint positions. Changes are validated against a set of constraints (e.g., ensuring that new rooms are placed on navigable cells) and then committed to the PostgreSQL database. The robot's next boot would fetch the updated map tiles from the cloud, eliminating the need for a manual offline map rebuild.

- **Dynamic room addition** – A form that allows adding a new room by specifying its name, tile ID, and coordinates (*x, y, yaw).* The backend inserts the room into the rooms table, and the nano agent downloads the updated room list on its next poll, making the new room available for job submissions without restarting the robot.

- **Real-time map visualisation** – A minimap showing the robot's current position and orientation overlaid on the active tile, updated at 2 Hz via the robot's /robot_health topic relayed through the cloud. This gives operators spatial awareness of the robot's progress without requiring RViz.

- **Job analytics dashboard** – Historical data on delivery times, failure rates per room, and robot utilisation, presented as charts and tables. This would help facility managers identify bottlenecks (e.g., a corridor that consistently causes localisation loss) and optimise room assignments.

7.2.5 Charge Monitoring and Charge-Based Path Planning

Beyond simple low-battery detection, the system can perform charge-aware navigation to maximise operational runtime. This requires:

- **Battery modelling** – The supervisor records the battery voltage at 1 Hz and integrates current consumption (estimated from motor PWM commands and sensor power draw) to build a discharge model specific to the robot's hardware. Over multiple charging cycles, the model learns the relationship between voltage drop, integrated current, and remaining capacity.

- **Energy cost map** – Each cell in the occupancy grid is assigned an estimated energy cost to traverse, correlated with the robot's required velocity (higher speeds consume more power due to increased motor current and aerodynamic drag, though the latter is negligible at indoor speeds). The global planner (Section 4.4.1) is modified to minimise a weighted sum of distance and energy cost.

- **Charge-aware goal selection** – When the robot has multiple pending jobs, the task manager schedules them in an order that minimises total energy consumption given the current state of charge. If the estimated remaining charge is insufficient to complete all jobs, the robot automatically returns to the charging station after finishing the last job that fits within the energy budget, then resumes once charged.

7.2.6 Multi-Robot Coordination

The current system is designed for a single robot. Deploying multiple robots on the same floor — or across multiple floors — requires a coordination layer. The following capabilities are proposed:

- **Centralised dispatcher** – A new cloud service (or an extension of the existing FastAPI backend) maintains a registry of all active robots, their poses (published via */robot_health*), battery levels, and current tasks. When a delivery request arrives, the dispatcher selects the robot that is closest to the pickup room, has sufficient battery, and is not already busy. If all robots are busy, the request is queued.

- **Collision avoidance** – A simple zone-based protocol is sufficient for corridor environments. The floor is partitioned into virtual zones (aligned with the existing tile boundaries). Each robot, before entering a zone, sends a *REQUEST_ZONE* message to the dispatcher. If the zone is free, the dispatcher grants a lock; the robot then navigates through the zone, releasing the lock upon exit. If the zone is occupied, the robot waits at the boundary. This prevents collisions without requiring each robot to sense the others directly, though a fallback depth-based emergency stop remains active in case of communication failure.

- **Task handover** – If a robot breaks down mid-job (e.g., wheel motor failure), the supervisor publishes a BROKEN state. The dispatcher cancels the robot's current job and reassigns it to another available robot. The broken robot publishes its last known pose, allowing the reassigned robot to navigate to the stranded robot's location, retrieve the parcel, and complete the delivery. This requires onboard storage that can be mechanically accessed (e.g., a tray that opens or a gripper), which is beyond the scope of the current design but is a feasible extension.

- **Cooperative charging** – When multiple robots are idle, the dispatcher can direct one robot to the charging station while others wait, rotating them through the charger to keep the fleet at optimal battery levels. The dispatcher uses the battery models and predicted job arrival rates to decide which robot should charge and for how long.

REFERENCES

[1] A. Fisher, R. Cannizzaro, M. Cochrane, C. Nagahawatte, and J. L. Palmer, "ColMap: A memory-efficient occupancy grid mapping framework," Robotics and Autonomous Systems, vol. 142, p. 103755, 2021. doi: 10.1016/j.robot.2021.103755

[2] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," Autonomous Robots, vol. 34, pp. 189–206, 2013. doi: 10.1007/s10514-012-9321-0

[3] D. Gonzalez-Arjona, A. Sanchez, F. López-Colino, A. de Castro, and J. Garrido, "Simplified occupancy grid indoor mapping optimized for low-cost robots," ISPRS International Journal of Geo-Information, vol. 2, pp. 959–977, 2013. doi: 10.3390/ijgi2040959

[4] R. B. Sousa, H. M. Sobreira, and A. P. Moreira, "A systematic literature review on long-term localization and mapping for mobile robots," Journal of Field Robotics, vol. 40, pp. 1245–1322, 2023. doi: 10.1002/rob.22170

[5] A. Schaefer, D. Büscher, J. Vertens, L. Luft, and W. Burgard, "Long-term vehicle localization in urban environments based on pole landmarks extracted from 3-D LiDAR scans," Robotics and Autonomous Systems, vol. 136, p. 103709, 2021. doi: 10.1016/j.robot.2020.103709

[6] T. Horelican, "Utilizability of Navigation2/ROS2 in highly automated and distributed multi-robotic systems for industrial facilities," IFAC PapersOnLine, vol. 55-4, pp. 109–114, 2022. doi: 10.1016/j.ifacol.2022.06.018

[7] M. A. Vega-Torres, A. Braun, and A. Borrmann, "SLAM2REF: Advancing long-term mapping with 3D LiDAR and reference map integration for precise 6-DoF trajectory estimation and map extension," Construction Robotics, vol. 8, p. 13, 2024. doi: 10.1007/s41693-024-00126-w

[8] Y. Baek and J. K. Park, "Fast path generation algorithm for mobile robot navigation using hybrid map," Applied Sciences, vol. 15, p. 2414, 2025. doi: 10.3390/app15052414

[9] H. Yun and S. Yoo, "Semantic zone based 3D map management for mobile robot," *arXiv preprint arXiv:2512.12228*, Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.12228

[10] Y. Wang, L. Zhao, and S. Huang, "Grid-based submap joining: An efficient algorithm for simultaneously optimizing global occupancy map and local submap frames," *arXiv preprint arXiv:2501.12764*, Jan. 2025. [Online]. Available: https://arxiv.org/abs/2501.12764\

[11] National Instruments, "Mobile Robotics Experiments with DaNI," National Instruments Corporation, Austin, TX, USA. [Online]. Available: *http://download.ni.com/pub/gdc/epd/mobile_robotics_experiments.pdf*

APPENDIX A –SETUP GUIDE

This appendix provides the repository link, setup instructions for the software stack, and access details for operating or debugging the system.

A.1 REPOSITORY SETUP

Clone the repository and initialise all submodules:

git clone https://github.com/senth-srmist/MATERIAL_TRANSFER_ROBOT_2026.git

cd MATERIAL_TRANSFER_ROBOT_2026

git submodule update --init --recursive

The zed_wrapper submodule must be set to version humble-v4.2.5. Navigate to it and check out the correct tag:

cd CODE/ros_ws/src/zed_wrapper

git checkout humble-v4.2.5

A.2 SOFTWARE STACK SETUP (DOCKER)

The ros 2 stack runs inside a docker container. Setup steps:

1. Install prerequisites — docker >= 20.x and nvidia container toolkit

2. Install zed udev rules on the host (skip on devices without nvidia gpu):

sudo mv CODE/99-slabs.rules /etc/udev/rules.d/ && sudo chmod 644 /etc/udev/rules.d/99-slabs.rules

sudo udevadm control --reload-rules

sudo udevadm trigger

3. Build the container (base image pulls automatically from github container registry):

TARGETARCH=AMD64 docker compose build # laptop

TARGETARCH=ARM64 docker-compose build

4. Allow docker display access, then start the container:

xhost +local:docker

Local mode (jetson — default):

docker-compose -f docker-compose.yml -f docker-compose.arm64.yml up -d

Network mode (jetson, cross-device ros 2):

ROS_MODE=NETWORK CYCLONE_INTERFACE_IP=<YOUR_IP> docker-compose -f docker-compose.yml -f docker-compose.arm64.yml up -d

Remote rviz (viz mode) — run on both devices:

On the robot: *CYCLONE_INTERFACE_IP=<YOUR_IP> docker-compose -f docker-compose.yml -f docker-compose.arm64.yml -f docker-compose.viz.yml up -d material-transfer-robot viz-bridge*

On the laptop: *CYCLONE_INTERFACE_IP=<YOUR_IP> docker compose -f docker-compose.yml -f docker-compose.viz.yml up viz-laptop*

A.3 WEB INTERFACE

LIVE UI: HTTPS://ROBOT-UI-WV22.VERCEL.APP/

To submit a delivery request:

Open https://robot-ui-wv22.vercel.app/ in any browser.

Select pickup room, drop room, enter your name, set priority, and click submit request.

The submit request button will be disabled if the robot is offline (no heartbeat received in the last 10 seconds).

A.4 DIRECT ROBOT ACCESS (JETSON NANO)

Connect the laptop to the robot's ethernet port and use the following credentials:

SSH: *ssh myrobo@10.0.0.2*

Password: *myrobo*

APPENDIX B – DATSHEETS AND MANUALS

1. ZED SDK Documentation & API Reference https://www.stereolabs.com/docs
2. ZED SDK Release Notes & Changelog (All Versions) https://github.com/stereolabs/zed-sdk/releases
3. Jetson Linux Archive (all versions): https://developer.nvidia.com/embedded/jetson-linux-archive
4. Carrier board product page: https://tannatechbiz.com/tanna-tech-biz-eagle-101-carrier-board.html
5. Motor specs: https://www.robot-advance.com/EN/art-tetrix-max-dc-motor-39530-1239.htm
6. Mobile Robotics Experiments manual: http://download.ni.com/pub/gdc/epd/mobile_robotics_experiments.pdf
7. Cytron Smart DUO 30 Motor Driver User Manual PDF (direct download, Rev 1.11): https://makermotor.com/content/cytron/pn00218-cyt14/MDDS30_User_Manual.pdf
8. Official tutorial/intro page: https://www.cytron.io/tutorial/introducing-30a-dual-channel-dc-motor-driver-smart-features-smartdriveduo-30